

## ***Curs 1. Introducere in baze de date***

### **Ce este o bază de date?**

O **bază de date** este o colecție de date de dimensiuni mari, stocată pe o perioadă îndelungată de timp în scopul analizei ulterioare. De asemenea, o bază de date modelează aspecte ale lumii reale sau conceptuale folosind un *model de date*.

Cu alte cuvinte, bazele de date sunt utile în stocarea și gestionarea datelor. Bazele de date sunt disponibile pe orice tip de dispozitiv de calcul/sistem de operare și accesarea datelor dintr-o bază de date reprezintă una dintre cele mai frecvente utilizări a unui calculator.

Utilizăm zi de zi o mare diversitate de baze de date fără a conștientiza acest lucru. Ori de câte ori căutăm un număr de telefon, realizăm o tranzacție într-un cont bancar, platim un produs sau serviciu cu cardul sau achiziționăm un produs de pe internet, folosim o bază de date.

Mai mult decât atât, bazele de date nu reprezintă un concept legat de un sistem de calcul. Ca exemple de baze de date “*necomputerizate*” amintim: cartea de telefon, dicționarele, almanahurile etc. (deși în accepțiunea curentă acestea sunt exemple de rapoarte în format imprimat generate dintr-o bază de date).

### **Model de date**

Un **model de date** este o colecție de concepte utilizate în descrierea datelor. Aceste concepte sunt utilizate în definirea structurii, semanticii, constrângerilor de consistență a datelor sau pentru definirea relațiilor cu alte date.

O **structură/schemă** este descrierea componentelor unei colecții de elemente/date utilizând un model particular. Structura este în general definită la începutul dezvoltării unei aplicații software și este rareori modificată ulterior.

O **instanță** a unei baze de date este formată din datele/valorile efective stocate într-o structură definită. Datele se pot schimba frecvent și pot fi comparate cu “*variabilele cu tip*” din limbajele de programare de nivel înalt.

*Exemple:*

- Modelul de date este un *graf*. De exemplu, nodurile pot reprezenta orașe iar arcurile dintre noduri rute aeriene
- Modelul de date este un *document XML*. De exemplu, documentul poate conține lista cărților cu cod ISBN ca identificator, titlu și numele autorului/autorilor ca sub-elemente
- Modelul de date este o *mulțime de înregistrări*. De exemplu, înregistrările pot conține cod de student, nume, adresă, listă cursuri, fotografie.

Primul model de date important a fost *Modelul Ierarhic*. El a fost definit spre finalul anilor ‘60 și a reprezentat o extindere a unui sistem de stocare/procesare a fișierelor. O structură ce utilizează acest model organizează datele într-o structură arborescentă.

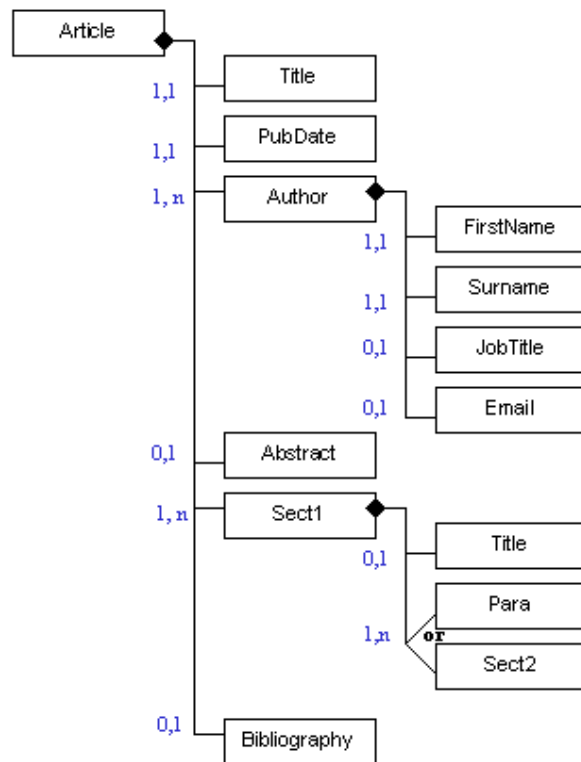


Figura 1.1 Structura unei entități *Article* folosind *Modelul Ierarhic*

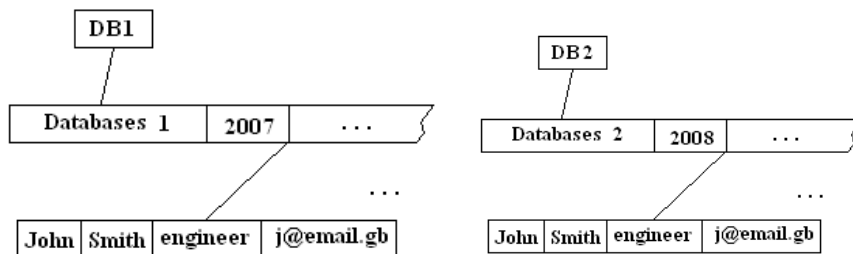


Figura 1.2 Două instanțe particulare ale structurii din Fig. 1.1

Un alt model de date important este *Modelul Rețea* care reprezintă o extensie a *Modelului Ierarhic*, organizând datele într-o structură de tip graf. Figura 1.3 prezintă structura unei baze de date folosind *Modelul Rețea*.

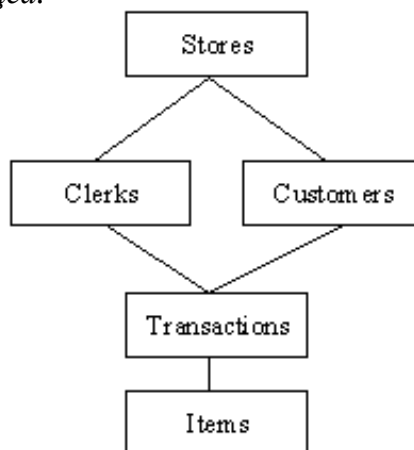


Figura 1.3.

La începuturile anilor '70 Ted Codd invetează *Modelul Relational* și conceptul de abstractizare a datelor. Modelul relațional este cel mai popular model de date în zilele noastre și va reprezenta principalul subiect al cursului curent.

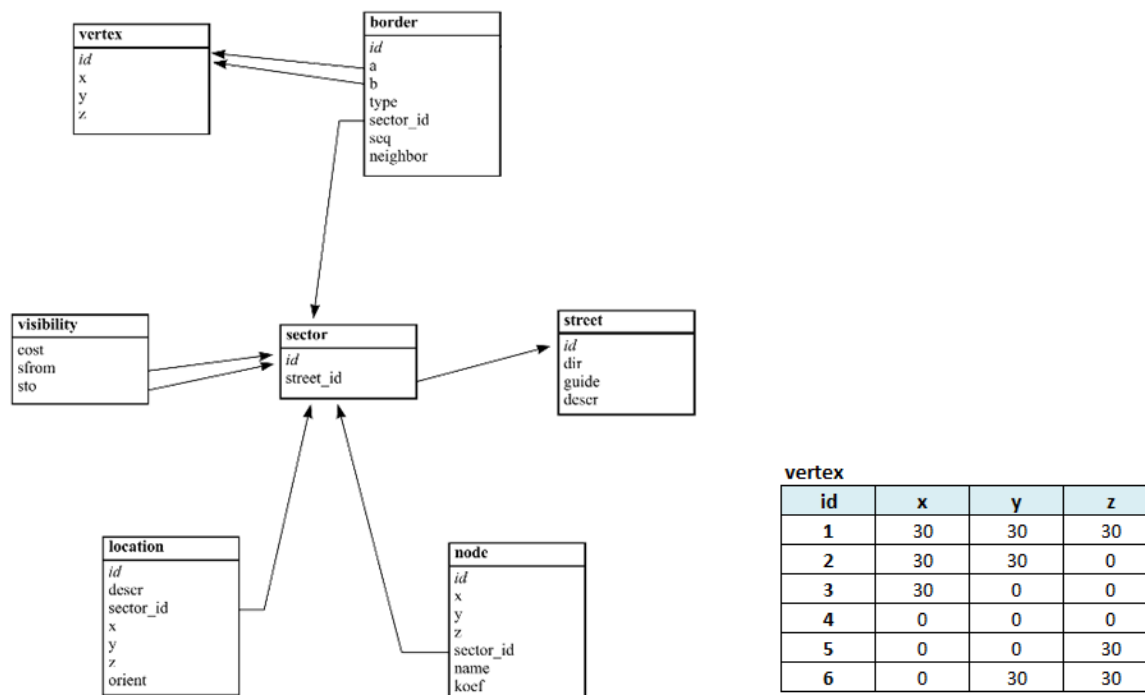


Figura 1.4 Exemplu de model relațional

*Modelul Orientat-Obiect* – introduce concepte (ca clasă, atribut, metodă) și relații între acestea (asocieri, agregări, moștenire). Modelul orientat-obiect este foarte popular ca filozofie de analiză, proiectare și dezvoltare de soft. În baze de date, din motive de eficiență, el prezintă doar un interes “științific”.

## Structură vs. Date

### Structura bazelor de date

- Se modifică rar;
- Denumită și *metadata* (= date reprezentând date);

### Starea bazei de date

- Se modifică frecvent;
- Sistemul de gestiune a bazelor de date asigură că starea fiecărei baze de date este una *validă*.

**Instanța bazei de date** se referă în general la combinația dintre structură și stare

## Sisteme de gestiune a bazelor de date

În timp ce o bază de date reprezintă o colecție structurată de date, aplicațiile care utilizează informațiile conținute în bazele de date sunt de obicei create pe baza unor instrumente furnizate de un mediu de dezvoltare a bazelor de date. Aceste medii de dezvoltare diferă de mediile tradiționale de programare prin faptul că oferă instrumente specifice care permit utilizatorilor să gestioneze cu ușurință datele, fără a se preocupa de detaliile de nivel coborât asociate în mod normal programării tradiționale (cum ar fi gestionarea memoriei etc.).

Un astfel de mediu de dezvoltare a bazelor de date face parte dintr-un așa-numit Sistem de Gestiune a Bazelor de Date (SGBD).

Un SGBD este o colecție integrată de **instrumente** pentru

- crearea unei baze de date și specificare structurii acesteia;
- interogarea și modificarea eficientă a datelor;
- securizarea datelor;
- controlul accesului la date de către *mai mulți* utilizatori la *un moment dat*;

Când este util să folosim baze de date? Atunci când nevoile aplicației noastre implică:

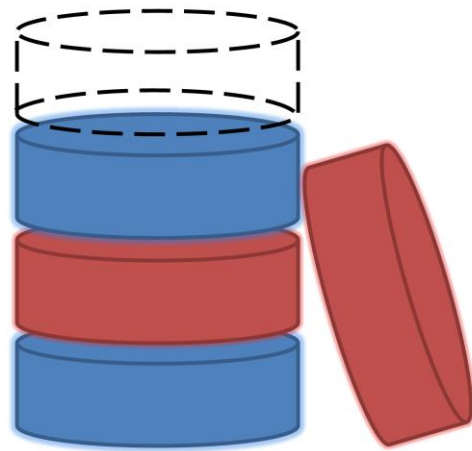
- Persistență
- Cantitate mare de date
- Date structurate
- Acces distribuit și concurrent la date
- Integritate
- Securitate
- Partajarea datelor cu alte aplicații

Când NU utilizăm baze de date?

- Investiția inițială e prea mare
- Prea mult efort
- Dezvoltăm o aplicație foarte simplă, bine-definită și care nu presupune modificări ulterioare
- Nu este necesar accesul mai multor utilizatori la date.

Alternativa: fișiere text.

# Modelul Relațional



2

# Modele de date

- Modelul ierarhic (1965)
- Modelul rețea (1965)
- **Modelul relațional (1NF)** (1970s)
- Model relațional imbricat (1970s)
- Obiecte complexe (1980s)
- Model obiectual (1980)
- Model relațional-obiectual (1990s)
- XML (DTD), XML Schema (1990s)

# Model relațional - idei

- Utilizează o structură de date simplă: *Tabela*
  - simplu de înțeles
  - utilă în modelarea multor situații/entități din lumea reală
  - conduc la interogări de o complexitate redusă
- Utilizează matematica în descrierea/reprezentarea înregistrărilor și a colecțiilor de înregistrări: *Relația*
  - pot fi modelate formal
  - permit utilizarea de limbaje de interogare formale
  - au proprietăți ce pot fi modelate și demonstrate matematic

# Relația – definiție formală

- O **relație** sau **structura unei relații R** este o listă de **nume de attribute**  $[A_1, A_2, \dots, A_n]$ .
- **Domeniu** = mulțime de valori scalare (tipuri atomice - întreg, text, dată, etc)
- $D_i = \text{Dom}(A_i)$  - domeniul lui  $A_i$ ,  $i=1..n$
- **Instanța unei relații** ( $[R]$ ) e o submulțime a
$$D_1 \times D_2 \times \dots \times D_n$$



# Relația – definiție formală

- **Grad (aritate)** = numărul tuturor atributelor din structura unei relații
- **Tuplu** = un element al instanței unei relații, o înregistrare. Toate tuplurile unei relații sunt distincte!
- **Cardinalitate** = numărul tupluri unei relații

# Exemplu de relație

- Students(**sid**:integer; **name**:string;**email**:string; **age**:integer; **gr**:integer)

*field name*

*field type  
(domain)*

| <b>sid</b> | <b>name</b> | <b>email</b>                 | <b>age</b> | <b>gr</b> |
|------------|-------------|------------------------------|------------|-----------|
| 2833       | Jones       | <u>jones@scs.ubbcluj.ro</u>  | 19         | 231       |
| 2877       | Smith       | <u>smith@scs.ubbcluj.ro</u>  | 20         | 232       |
| 2976       | Jones       | <u>jones@math.ubbcluj.ro</u> | 21         | 233       |
| 2765       | Mary        | <u>mary@math.ubbcluj.ro</u>  | 22         | 233       |

*relation  
schema*

*relation  
instance*

*relation  
tuple*

- cardinalitate = 4, grad = 5, toate tuplurile distincte !

# Baze de date relaționale

- O **bază de date** este o mulțime de relații
- **Structura** unei baze de date este mulțimea structurilor relațiilor acesteia
- **Instanța (starea)** unei baze de date este mulțimea instanțelor relațiilor acesteia

# Reprezentarea grafică a relațiilor

Students(*sid*:string, *name*:string, *email*:string, *age*:integer, *gr*:integer)

Courses(*cid*: string, *cname*: string, *credits*:integer)

Enrolled(*sid*:string, *cid*:string, *grade*:double)

Teachers(*tid*:integer; *name*: string; *sal* : integer)

Teaches(*tid*:integer; *cid*:string)

| Students                                                                          |       |
|-----------------------------------------------------------------------------------|-------|
|  | sid   |
|  | name  |
|  | email |
|  | age   |
|  | gr    |

| Courses                                                                             |         |
|-------------------------------------------------------------------------------------|---------|
|  | cid     |
|  | cname   |
|  | credits |

| Teachers                                                                            |      |
|-------------------------------------------------------------------------------------|------|
|  | tid  |
|  | name |
|  | sal  |

| Teaches                                                                             |     |
|-------------------------------------------------------------------------------------|-----|
|  | tid |
|  | cid |

| Enrolled                                                                              |       |
|---------------------------------------------------------------------------------------|-------|
|  | sid   |
|  | cid   |
|  | grade |

# Constrângeri de integritate (CI)

- **CI**: sunt condiții ce trebuie să fie îndeplinite de către *orice* instanță a unei baze de date
  - specificate la momentul definirii structurii relației
  - verificate la modificarea conținutului relației
- O instanță a unei relații că este *legală* dacă satisface toate CI specificate
  - SGBD nu va permite instanțe *ilegale*

# Constrângeri de integritate - exemple

- Students(*sid:string*, *name:string*, *email:string*, *age:integer*, *gr:integer*)
  - Constrângere de domeniu: *gr:integer*
  - Constrângere de interval:  $18 \leq \textit{age} \leq 70$
- TestResults(*sid:string*, *TotalQuestions:integer*, *NotAnswered:integer*, *CorrectAnswers:integer*, *WrongAnswers:integer*)
  - $\textit{TotalQuestions} = \textit{NotAnswered} + \textit{CorrectAnswers} + \textit{WrongAnswers}$  – **nu e o CI!**

# Chei Primare

- O mulțime de attribute reprezintă o **cheie** a unei relații dacă:
  1. Nu există două tuple care au aceleași valori pentru toate attributele

**ȘI**

  2. Acest lucru nu este adevărat pentru nici o submulțime a cheii
- Dacă a 2-a afirmație este falsă → **super cheie**
- Dacă există >1 cheie pentru o relație → **chei candidat**
- Una dintre cheile candidat este selectată ca **cheie primară**

## Chei străine (externe)

- O **cheie străină (externă)** este o mulțime de câmpuri a unei relații utilizate pentru a `referi` un tuplu al unei alte relații (un fel de `pointer logic`).
  - Aceasta trebuie să corespundă cheii primare din a doua relație.

De exemplu pentru

*Enrolled* (*sid*: string, *cid*: string, *grade*: double)

*sid* este cheie externă referind *Students*



# Integritate referențială

■ **Integritate referențială** = nu sunt permise valori pentru cheia străină care nu se regăsesc în tabela referită.

Exemplu de model de date fără integritate referențială:

*Link-uri HTML*



# Integritate referențială

- Fie *Students* și *Enrolled*; *sid* in *Enrolled* este o cheie străină ce referă o înregistrări din *Students*.
- Adaugarea in *Enrolled* a unui tuplu cu un id de student inexistent, acesta va fi respins de SGBD.

*Enrolled*

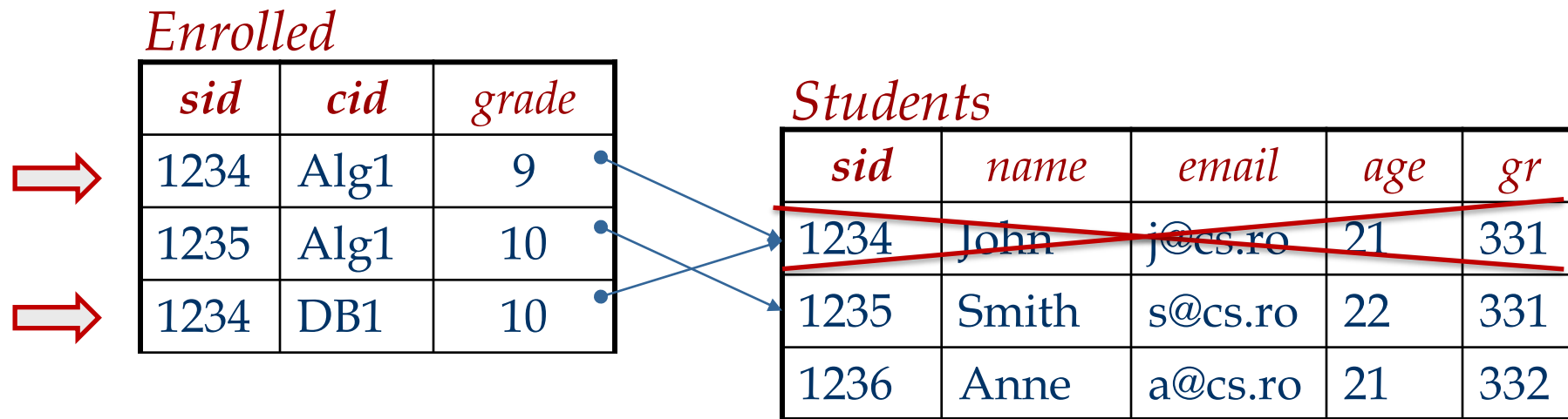
| <i>sid</i>      | <i>cid</i>     | <i>grade</i> |
|-----------------|----------------|--------------|
| 1234            | Alg1           | 9            |
| 1235            | Alg1           | 10           |
| 1234            | DB1            | 10           |
| <del>1237</del> | <del>DB2</del> | <del>9</del> |

*Students*

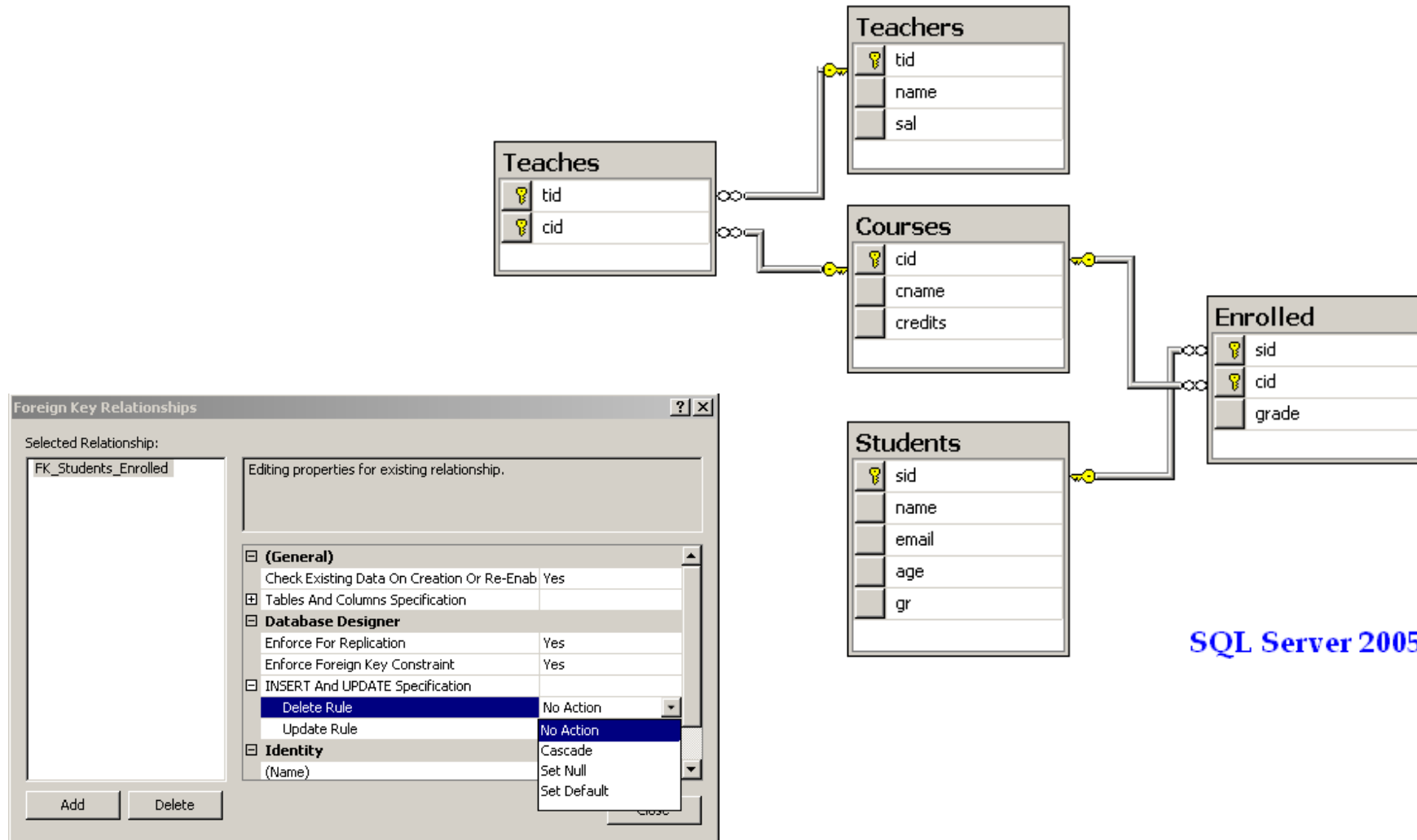
| <i>sid</i> | <i>name</i> | <i>email</i> | <i>age</i> | <i>gr</i> |
|------------|-------------|--------------|------------|-----------|
| 1234       | John        | j@cs.ro      | 21         | 331       |
| 1235       | Smith       | s@cs.ro      | 22         | 331       |
| 1236       | Anne        | a@cs.ro      | 21         | 332       |

# Integritate referențială

- Dacă o înregistrare din *Students* este ștearsă dar ea este referită din *Enrolled*:
  - se șterg toate înregistrările ce o refera din *Enrolled*.
  - nu se permite ștergerea înregistrării din *Students*
  - sid din *Enrolled* va avea asignată o valoare implicită.
  - sid din *Enrolled* va avea asignată valoarea *null*.



# Reprezentarea grafică a CI



SQL Server 2005

# Cum apar CI?

- CI se bazează pe semantica entităților din lumea reală / conceptuală modelate.
- Putem verifica dacă o CI este încălcată de instanța unei tabele, însă **NU** vom putea deduce dacă o CI este adevărată doar consultând o singură instanță.
  - O CI se referă la *toate instanțele* posibile ale unei tabele
- Cheile primare și externe sunt cele mai comune CI;

# Interogări

- Posibile informații pe care dorim să le obținem din baza de date anterioară (*Faculty Database*) :
  - Care este numele studentului cu *sid* = 2833?
  - Care este salariul profesorilor care predau cursul *Alg100*?
  - Câți studenți sunt înscriși la cursul *Alg100*?
- Astfel de întrebări referitoare la datele stocate într-un SGBD se numesc *interogări*.
- → *limbaj de interogare*

# Limbaje SGBD

- *Data Definition Language (DDL)*
  - Definesc structura **conceptuală**
  - Descriu **constrângerile de integritate**
  - Influențează **structura fizică** (în anumite SGBD-uri)
- *Data Manipulation Language (DML)*
  - Operații aplicate instanțelor unei baze de date
  - DML procedural (**cum?**) vs. DML declarative (**ce?**)
- *Limbaj gazdă*
  - Limbaj de programare obișnuit ce permite utilizatorilor să includă comenzi DML în propriul cod

# Limbaje de interogare pentru BD relaționale

SQL (Structured Query Language)

**SELECT** *name* **FROM** *Students* **WHERE** *age* > 20

Algebra

$\pi_{name}(\sigma_{age > 20}(Students))$

Domain Calculus

$\{ \langle X \rangle \mid \exists V \exists Y \exists Z \exists T : Students(V, X, Y, Z, T) \wedge Z > 20 \}$

T-tuple Calculus

$\{ X \mid \exists Y : Y \in Students \wedge Y.age > 20 \wedge X.name = Y.name \}$

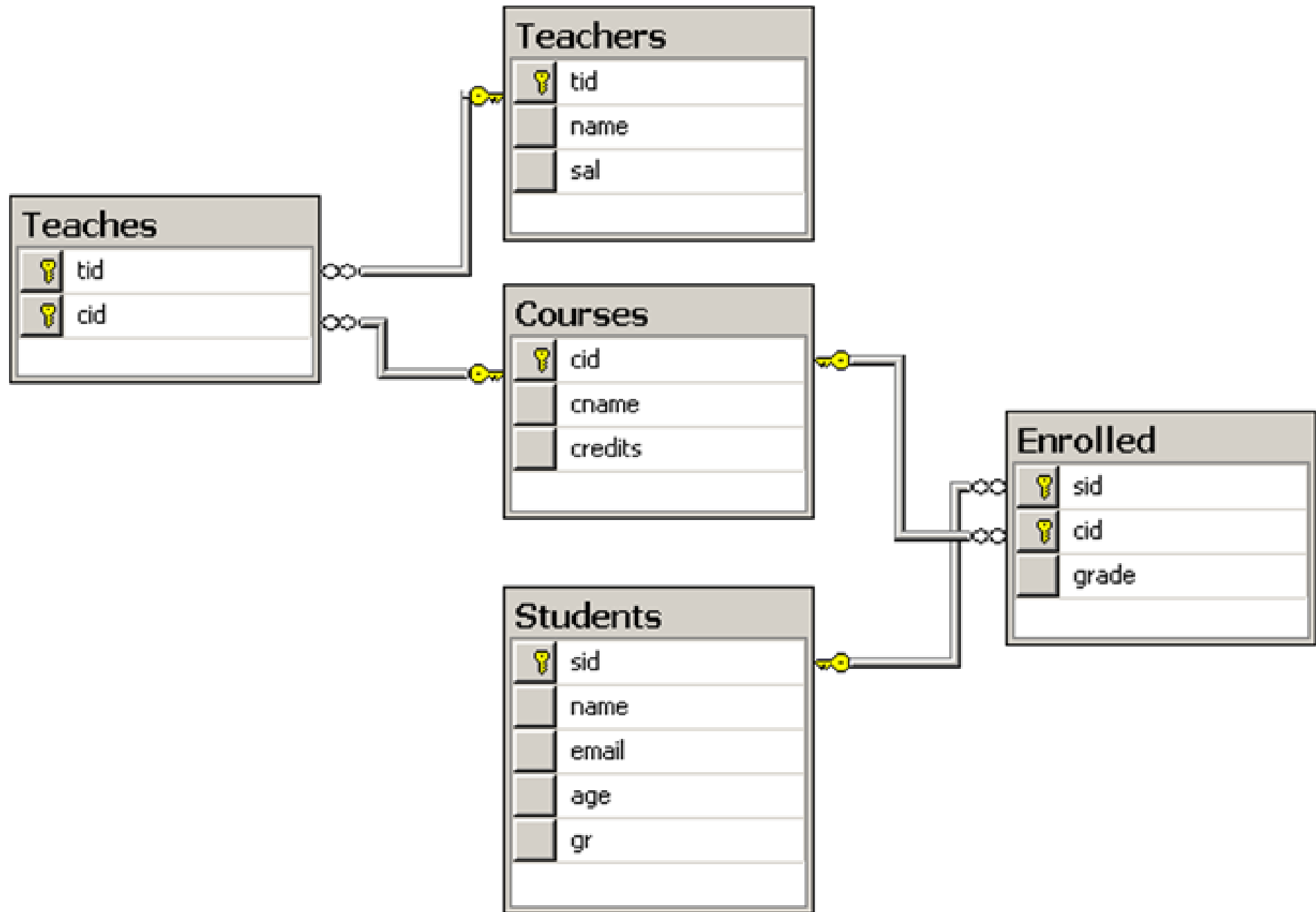


# Structured Query Language (SQL)

- Dezvoltat de IBM (*system R*) în anii 1970
- Ulterior a apărut nevoia de standardizare
- Standarde (ANSI):
  - SQL-86
  - SQL-89 (minor revision)
  - SQL-92 (major revision) - *1,120 pagini*
  - SQL-99 (major extensions) - *2,084 pagini*
  - SQL-2003 (secțiuni SQL/XML) - *3,606 pagini*
  - SQL-2006
  - SQL-2008
  - SQL-2011

# Nivele SQL

- Data-definition language (DDL):
  - Creare / stergere / modificare *tabele* și *views*.
  - Definire *constrangeri de integritate* (CI's).
- Data-manipulation language (DML)
  - Permit formularea de interogari
  - Inserare / ștergere / modificare înregistrări.
- *Controlul accesului:*
  - Asignează sau elimină drepturi de acces si modificare a *tabelelor* și a *view-urilor*.



## *Students*

| <i>sid</i> | <i>name</i> | <i>email</i> | <i>age</i> | <i>gr</i> |
|------------|-------------|--------------|------------|-----------|
| 1234       | John        | j@cs.ro      | 21         | 331       |
| 1235       | Smith       | s@cs.ro      | 22         | 331       |
| 1236       | Anne        | a@cs.ro      | 21         | 332       |

## *Enrolled*

| <i>sid</i> | <i>cid</i> | <i>grade</i> |
|------------|------------|--------------|
| 1234       | Alg1       | 9            |
| 1235       | Alg1       | 10           |
| 1237       | DB2        | 9            |

## *Courses*

| <i>cid</i> | <i>cname</i> | <i>credits</i> |
|------------|--------------|----------------|
| Alg1       | Algorithms1  | 7              |
| DB1        | Databases1   | 6              |
| DB2        | Databases2   | 6              |

# SELECT

Studentii cu vârsta de 21 de ani:

```
SELECT  *  
FROM    Students S  
WHERE   S.age = 21
```

| <i>sid</i> | <i>name</i> | <i>email</i> | <i>age</i> | <i>gr</i> |
|------------|-------------|--------------|------------|-----------|
| 1234       | John        | j@cs.ro      | 21         | 331       |
| 1236       | Anne        | a@cs.ro      | 21         | 332       |

Returnează doar numele și adresele de e-mail:

```
SELECT  S.name, S.email  
FROM    Students S  
WHERE   S.age = 21
```

| <i>name</i> | <i>email</i> |
|-------------|--------------|
| John        | j@cs.ro      |
| Anne        | a@cs.ro      |

# Interogare SQL simplă

```
SELECT [DISTINCT] target-list
FROM relation-list
WHERE qualification
```

- *relation-list* - lista de nume de relații/tabele.
- *target-list* - listă de attribute ale relațiilor din  
*relation-list*
- *qualification* - comparații logice (*Attr op const* sau *Attr1 op Attr2*, unde *op* is one of  $<$ ,  $>$ ,  $=$ ,  $\leq$ ,  $\geq$ ,  $\neq$ ) combinate cu AND, OR sau NOT.
- *DISTINCT* (optional) - indică faptul că rezultatul final nu conține duplicate.

# Evaluare conceptuală

```
SELECT [DISTINCT] target-list  
FROM relation-list  
WHERE qualification
```

- Calcul produs cartezian al tabelelor din *relation-list*.
- Filtrare înregistrări ce nu verifică *qualifications*.
- Ștergere attribute ce nu aparțin *target-list*.
- Dacă **DISTINCT** e prezent, se elimină înregistrările duplicate.

1. PRODUS CARTEZIAN

2. ELIMINA LINII

3. ELIMINA COLOANE

4. ELIMINA DUPLICATE

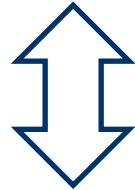


Această strategie e **doar**  
la nivel *conceptual*!

Modul actual de evaluare  
a unei interogări  
e **mult** optimizat

# Utilizare *alias* pentru

```
SELECT S.name, E.cid  
FROM Students S, Enrolled E  
WHERE S.sid=E.sid AND E.grade=10
```



```
SELECT name, cid  
FROM Students, Enrolled  
WHERE Students.sid=Enrolled.sid  
AND grade=10
```

Interogare: *Studentii care au cel puțin o notă*

```
SELECT S.sid  
FROM Students S, Enrolled E  
WHERE S.sid=E.sid
```

- Rezultatul e diferit cu **DISTINCT**?
- Ce efect are înlocuirea *S.sid* cu *S.sname* în clauza **SELECT**?  
Rezultatul e diferit cu **DISTINCT** în acest caz?

# Expresii și string-uri

- *Obține triplete (cu vârsta studenților + alte două expresii) pentru studenții al căror nume începe și se termină cu B și conține cel puțin trei caractere.*

```
SELECT S.age, age1=S.age-5, 2*S.age AS age2  
FROM Students S  
WHERE S.name LIKE 'B_%B'
```

- **AS** și **=** sunt două moduri de redenumire a câmpurilor în rezultat.
- **LIKE** e folosit pentru comparații pe siruri de caractere. **'\_'** reprezintă orice caracter și **'%'** stands reprezintă 0 sau mai multe caractere arbitrare.

# INNER JOIN

```
SELECT S.name, C.cname
FROM Students S,
Enrolled E, Courses C
WHERE S.sid = E.sid
AND E.cid = C.cid
```



```
SELECT S.name, C.cname
FROM Students S
INNER JOIN Enrolled E ON
S.sid = E.sid,
INNER JOIN Courses C ON
E.cid = C.cid
```

*Students*

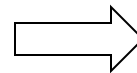
| <i>sid</i> | <i>name</i> | <i>email</i> | <i>age</i> | <i>gr</i> |
|------------|-------------|--------------|------------|-----------|
| 1234       | John        | j@cs.ro      | 21         | 331       |
| 1235       | Smith       | s@cs.ro      | 22         | 331       |
| 1236       | Anne        | a@cs.ro      | 21         | 332       |

*Courses*

| <i>cid</i> | <i>cname</i> | <i>credits</i> |
|------------|--------------|----------------|
| Alg1       | Algorithms1  | 7              |
| DB1        | Databases1   | 6              |
| DB2        | Databases2   | 6              |

*Enrolled*

| <i>sid</i> | <i>cid</i> | <i>grade</i> |
|------------|------------|--------------|
| 1234       | Alg1       | 9            |
| 1235       | Alg1       | 10           |
| 1237       | DB2        | 9            |



| <i>name</i> | <i>cname</i> |
|-------------|--------------|
| John        | Algorithms1  |
| Smith       | Algorithms1  |

# LEFT OUTER JOIN

- Daca dorim sa regasim și studentii fără nici o notă la vreun curs:

```
SELECT S.name, C.cname
FROM Students S
LEFT OUTER JOIN Enrolled E
ON S.sid = E.sid,
LEFT OUTER JOIN Courses C
ON E.cid = C.cid
```

*Students*

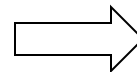
| <i>sid</i> | <i>name</i> | <i>email</i> | <i>age</i> | <i>gr</i> |
|------------|-------------|--------------|------------|-----------|
| 1234       | John        | j@cs.ro      | 21         | 331       |
| 1235       | Smith       | s@cs.ro      | 22         | 331       |
| 1236       | Anne        | a@cs.ro      | 21         | 332       |

*Courses*

| <i>cid</i> | <i>cname</i> | <i>credits</i> |
|------------|--------------|----------------|
| Alg1       | Algorithms1  | 7              |
| DB1        | Databases1   | 6              |
| DB2        | Databases2   | 6              |

*Enrolled*

| <i>sid</i> | <i>cid</i> | <i>grade</i> |
|------------|------------|--------------|
| 1234       | Alg1       | 9            |
| 1235       | Alg1       | 10           |
| 1237       | DB2        | 9            |



| <i>name</i> | <i>cname</i> |
|-------------|--------------|
| John        | Algorithms1  |
| Smith       | Algorithms1  |
| Anne        | <b>NULL</b>  |

# RIGHT OUTER JOIN

- Pentru a gasi notele asiguate unor studenti inexistenti:

```
SELECT S.name, C.cname
FROM Students S
RIGHT OUTER JOIN Enrolled E
ON S.sid = E.sid,
INNER JOIN Courses C ON
E.cid = C.cid
```

*Students*

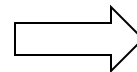
| <i>sid</i> | <i>name</i> | <i>email</i> | <i>age</i> | <i>gr</i> |
|------------|-------------|--------------|------------|-----------|
| 1234       | John        | j@cs.ro      | 21         | 331       |
| 1235       | Smith       | s@cs.ro      | 22         | 331       |
| 1236       | Anne        | a@cs.ro      | 21         | 332       |

*Courses*

| <i>cid</i> | <i>cname</i> | <i>credits</i> |
|------------|--------------|----------------|
| Alg1       | Algorithms1  | 7              |
| DB1        | Databases1   | 6              |
| DB2        | Databases2   | 6              |

*Enrolled*

| <i>sid</i> | <i>cid</i> | <i>grade</i> |
|------------|------------|--------------|
| 1234       | Alg1       | 9            |
| 1235       | Alg1       | 10           |
| 1237       | DB2        | 9            |



| <i>name</i> | <i>cname</i> |
|-------------|--------------|
| John        | Algorithms1  |
| Smith       | Algorithms1  |
| NULL        | Databases2   |

# FULL OUTER JOIN

- LEFT+RIGHT OUTER JOIN
- In majoritatea SGBD OUTER e optional

```
SELECT S.name, C.cname
FROM Students S
FULL OUTER JOIN Enrolled E
ON S.sid = E.sid,
FULL OUTER JOIN Courses C
ON E.cid = C.cid
```

*Students*

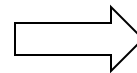
| <i>sid</i> | <i>name</i> | <i>email</i> | <i>age</i> | <i>gr</i> |
|------------|-------------|--------------|------------|-----------|
| 1234       | John        | j@cs.ro      | 21         | 331       |
| 1235       | Smith       | s@cs.ro      | 22         | 331       |
| 1236       | Anne        | a@cs.ro      | 21         | 332       |

*Courses*

| <i>cid</i> | <i>cname</i> | <i>credits</i> |
|------------|--------------|----------------|
| Alg1       | Algorithms1  | 7              |
| DB1        | Databases1   | 6              |
| DB2        | Databases2   | 6              |

*Enrolled*

| <i>sid</i> | <i>cid</i> | <i>grade</i> |
|------------|------------|--------------|
| 1234       | Alg1       | 9            |
| 1235       | Alg1       | 10           |
| 1237       | DB2        | 9            |

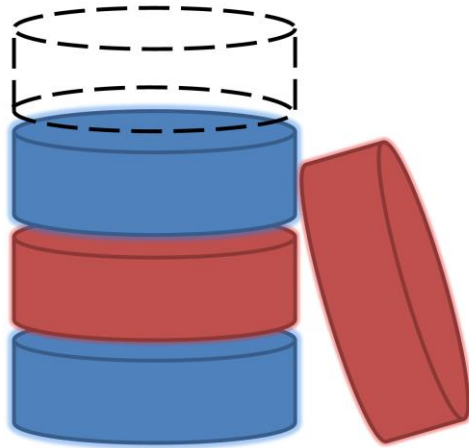


| <i>name</i> | <i>cname</i> |
|-------------|--------------|
| John        | Algorithms1  |
| Smith       | Algorithms1  |
| NULL        | Databases2   |
| NULL        | Databases1   |
| Anne        | NULL         |



# SQL DML (cont)

GROUP BY, HAVING  
ORDER BY



# Valoarea NULL

- În anumite situații valorile particulare ale unor attribute (câmpuri) pot fi *necunoscute* sau *inaplicabile* temporar.
  - SQL permite utilizarea unei valori speciale null pentru astfel de situații.
- Prezența valorii *null* implică unele probleme suplimentare:
  - E necesară implementarea unei logici cu 3 valori: *true*, *false* și *null* (de exemplu o condiție de tipul *rating*>8 va fi întotdeauna evaluată cu *false* dacă valoarea câmpului *rating* este *null*)
  - E necesară adăugarea unui operator special IS NULL / IS NOT NULL.

# Operatori de agregare

COUNT (\*)  
COUNT ([DISTINCT] A)  
SUM ([DISTINCT] A)  
AVG ([DISTINCT] A)  
MAX (A)  
MIN (A)

*atribut*

```
SELECT COUNT (*)  
FROM Students S
```

```
SELECT AVG (S.age)  
FROM Students S  
WHERE S.gr=921
```

```
SELECT COUNT (DISTINCT S.gr)  
FROM Students S  
WHERE S.name='Bob'
```

```
SELECT S.name  
FROM Students S  
WHERE S.age = ANY  
      (SELECT MAX(S2.age)  
       FROM Students S2)
```

# GROUP BY / HAVING

For  $i = 221, 222, 223, 224 \dots$ :

```
SELECT MIN(S.age)
FROM   Students S
WHERE  S.gr =  $i$ 
```

# GROUP BY / HAVING

```
SELECT [DISTINCT] target-list
FROM    relation-list
WHERE   qualification
GROUP BY grouping-list
HAVING  group-qualification
```

# GROUP BY / HAVING

```
SELECT [DISTINCT] target-list
FROM   relation-list
WHERE  qualification
GROUP BY grouping-list
HAVING   group-qualification
```

- Un *grup* este o mulțime de tupluri care au aceeași valoare pentru toate attributele din *grouping-list*.
- *target-list* conține (i) nume de atribut sau (ii) termeni ce utilizează operatori de agregare (e.g., MIN (*S.age*)).
  - numele de atribut (i) trebuie să fie o submulțime a *grouping-list*.
  - Intuitiv, fiecare tuplu din rezultat corespunde unui *grup*, și toate attributele vor avea o singură valoare per grup.

Numarul studentilor cu nota la cursurile cu 6 credite si media notelor acestora

```
SELECT  C.cid, COUNT (*) AS scount, AVG(grade)
FROM    Enrolled E, Courses C
WHERE   E.cid=C.cid AND C.credits=6
GROUP BY C.cid
```

### *Courses*

| <i>cid</i> | <i>cname</i> | <i>credits</i> |
|------------|--------------|----------------|
| Alg1       | Algorithms1  | 7              |
| DB1        | Databases1   | 6              |
| DB2        | Databases2   | 6              |

### *Students*

| <i>sid</i> | <i>name</i> | <i>email</i> | <i>age</i> | <i>gr</i> |
|------------|-------------|--------------|------------|-----------|
| 1234       | John        | j@cs.ro      | 21         | 331       |
| 1235       | Smith       | s@cs.ro      | 22         | 331       |
| 1236       | Anne        | a@cs.ro      | 21         | 332       |

### *Enrolled*

| <i>sid</i> | <i>cid</i> | <i>grade</i> |
|------------|------------|--------------|
| 1234       | Alg1       | 9            |
| 1235       | Alg1       | 10           |
| 1234       | DB1        | 10           |
| 1234       | DB2        | 9            |
| 1236       | DB1        | 7            |

### *Enrolled*

### *Courses*

| <i>sid</i> | <i>cid</i> | <i>grade</i> | <i>cid</i> | <i>cname</i> | <i>credits</i> |
|------------|------------|--------------|------------|--------------|----------------|
| 1234       | Alg1       | 9            | Alg1       | Algorithms1  | 7              |
| 1234       | Alg1       | 9            | DB1        | Databases1   | 6              |
| 1234       | Alg1       | 9            | DB2        | Databases2   | 6              |
| 1235       | Alg1       | 10           | Alg1       | Algorithms1  | 7              |
| 1235       | Alg1       | 10           | DB1        | Databases1   | 6              |
| 1235       | Alg1       | 10           | DB2        | Databases2   | 6              |
| 1234       | DB1        | 10           | Alg1       | Algorithms1  | 7              |
| 1234       | DB1        | 10           | DB1        | Databases1   | 6              |
| 1234       | DB1        | 10           | DB2        | Databases2   | 6              |
| 1234       | DB2        | 9            | Alg1       | Algorithms1  | 7              |
| 1234       | DB2        | 9            | DB1        | Databases1   | 6              |
| 1234       | DB2        | 9            | DB2        | Databases2   | 6              |
| 1236       | DB1        | 7            | Alg1       | Algorithms1  | 7              |
| 1236       | DB1        | 7            | DB1        | Databases1   | 6              |
| 1236       | DB1        | 7            | DB2        | Databases2   | 6              |

```
SELECT C.cid,  
COUNT(*)AS scount,  
AVG(grade)AS average  
FROM Enrolled E,  
Courses C  
WHERE E.cid=C.cid  
AND C.credits=6  
GROUP BY C.cid
```



| <i>Enrolled</i> |            |              | <i>Courses</i> |               |                |
|-----------------|------------|--------------|----------------|---------------|----------------|
| <i>sid</i>      | <i>cid</i> | <i>grade</i> | <i>cid</i>     | <i>cname</i>  | <i>credits</i> |
| 1234            | Alg1       | 9            | Alg1           | Algoritmics 1 | 7              |
| 1235            | Alg1       | 10           | Alg1           | Algoritmics 1 | 7              |
| 1234            | DB1        | 10           | DB1            | Databases1    | 6              |
| 1234            | DB2        | 9            | DB2            | Databases2    | 6              |
| 1236            | DB1        | 7            | DB1            | Databases1    | 6              |

```

SELECT C.cid,
COUNT(*)AS scount,
AVG(grade)AS average
FROM    Enrolled E,
        Courses C
WHERE   E.cid=C.cid
AND
        C.credits=6
GROUP BY C.cid

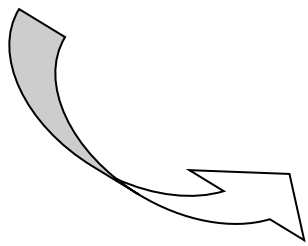
```

| <i>sid</i> | <i>cid</i> | <i>grade</i> | <i>cid</i> | <i>cname</i> | <i>credits</i> |
|------------|------------|--------------|------------|--------------|----------------|
| 1234       | DB1        | 10           | DB1        | Databases1   | 6              |
| 1234       | DB2        | 9            | DB2        | Databases2   | 6              |
| 1236       | DB1        | 7            | DB1        | Databases1   | 6              |

```

SELECT C.cid
COUNT(*) AS scount,
AVG(grade) AS average
FROM   Enrolled E,
       Courses C
WHERE  E.cid=C.cid
AND
      C.credits=6
GROUP BY C.cid
HAVING MAX(grade) = 10

```



| <i>cid</i>     | <i>scount</i> | <i>average</i> |
|----------------|---------------|----------------|
| DB1            | 2             | 8.5            |
| <del>DB2</del> | <del>1</del>  | <del>9</del>   |

# Sortarea rezultatului interogărilor

- `ORDER BY column [ ASC | DESC] [, ...]`

```
SELECT cname, sname, grade
FROM Courses C
      INNER JOIN Enrolled E ON C.cid = E.cid
      INNER JOIN Students S ON E.sid = S.sid
ORDER BY cname, grade DESC , sname
```

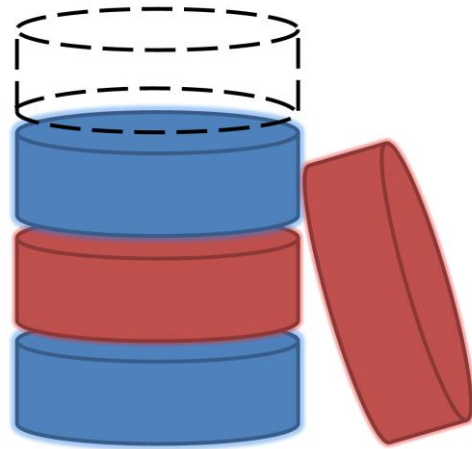
# Sortarea rezultatului interogărilor

- Rezultatul e sortat după orice câmp din clauza SELECT, inclusiv expresii sau agregări:

```
SELECT gr, Count(*) as StudNo  
FROM Students C  
GROUP BY gr  
ORDER BY StudNo
```

# Rafinarea structurii bazelor de date

(Dependențe funcționale)



3

Structura bazei de date

=

Structura relațiilor

+

Constrângeri

## Exemplu: relația *MovieList*

| <i>Title</i>            | <i>Director</i> | <i>Cinema</i>  | <i>Phone</i> | <i>Time</i> |
|-------------------------|-----------------|----------------|--------------|-------------|
| The Hobbit              | Jackson         | Florin Piersic | 441111       | 11:30       |
| The Lord of the Rings 3 | Jackson         | Florin Piersic | 441111       | 14:30       |
| Adventures of Tintin    | Spielberg       | Victoria       | 442222       | 11:30       |
| The Lord of the Rings 3 | Jackson         | Victoria       | 442222       | 14:00       |
| War Horse               | Spielberg       | Victoria       | 442222       | 16:30       |

### Constrângeri:

- Fiecare film are un regizor
- Fiecare cinematograf are un număr de telefon
- Fiecare cinematograf începe proiecția unui singur film al un moment dat

# Proiectare defectuoasă!

| <i>Title</i>            | <i>Director</i> | <i>Cinema</i>  | <i>Phone</i> | <i>Time</i> |
|-------------------------|-----------------|----------------|--------------|-------------|
| The Hobbit              | Jackson         | Florin Piersic | 441111       | 11:30       |
| The Lord of the Rings 3 | Jackson         | Florin Piersic | 441111       | 14:30       |
| Adventures of Tintin    | Spielberg       | Victoria       | 442222       | 11:30       |
| The Lord of the Rings 3 | Jackson         | Victoria       | 442222       | 14:00       |
| War Horse               | Spielberg       | Victoria       | 442222       | 16:30       |

**Anomalie de inserare**

**Anomalie de ștergere**

**Anomalie de actualizare**



# Rafinarea unei structuri defectuoase prin *descompunerea* în mai multe structuri “bune”

## *Movies*

| <i>Title</i>            | <i>Director</i> |
|-------------------------|-----------------|
| The Hobbit              | Jackson         |
| The Lord of the Rings 3 | Jackson         |
| Adventures of Tintin    | Spielberg       |
| War Horse               | Spielberg       |

## *Screens*

| <i>Cinema</i>  | <i>Time</i> | <i>Title</i>            |
|----------------|-------------|-------------------------|
| Florin Piersic | 11:30       | The Hobbit              |
| Florin Piersic | 14:30       | The Lord of the Rings 3 |
| Victoria       | 11:30       | Adventures of Tintin    |
| Victoria       | 14:00       | The Lord of the Rings 3 |
| Victoria       | 16:30       | War Horse               |

## *Cinema*

| <i>Cinema</i>  | <i>Phone</i> |
|----------------|--------------|
| Florin Piersic | 441111       |
| Victoria       | 442222       |

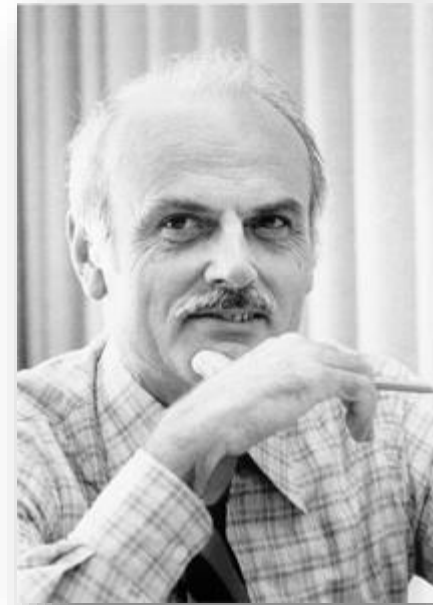
- ✓ Anomalie de **inserare**
- ✓ Anomalie de **ștergere**
- ✓ Anomalie de **actualizare**

Cum determinăm  
dacă o structură este  
“*bună*” sau “*defectuoasă*”?

Cum transformăm  
o structură *defectuoasă*  
într-una *bună*?

Teoria *dependențelor funcționale* furnizează o abordare sistematică a celor două întrebări

Introdusă de  
Edgar Frank Codd în:



*“A relational model for large shared data banks”,  
Com. of the ACM, 13(6), 1970, pp.377-387.*

# Dependențe funcționale

$$\alpha \rightarrow \beta$$

$\alpha, \beta$  sunt submulțimi de attribute ale R

“ $\alpha$  determină funcțional  $\beta$ ”

sau

“ $\beta$  depinde functional de  $\alpha$ ”

# Definiție dependențe funcționale

Dependența funcțională  $\alpha \rightarrow \beta$  este satisfăcută de R dacă și numai dacă

pentru *orice* instanță a lui R,

oricare două tupluri  $t_1$  și  $t_2$  pentru care  
valorile lui  $\alpha$  sunt identice

vor avea de asemenea valori identice pentru  $\beta$ .

O dependență funcțională

$$\alpha \rightarrow \beta$$

este **trivială** dacă

$$\alpha \supseteq \beta.$$

| <i>Title</i>            | <i>Director</i> | <i>Cinema</i>  | <i>Phone</i> | <i>Time</i> |
|-------------------------|-----------------|----------------|--------------|-------------|
| The Hobbit              | Jackson         | Florin Piersic | 441111       | 11:30       |
| The Lord of the Rings 3 | Jackson         | Florin Piersic | 441111       | 14:30       |
| Adventures of Tintin    | Spielberg       | Victoria       | 442222       | 11:30       |
| The Lord of the Rings 3 | Jackson         | Victoria       | 442222       | 14:00       |
| War Horse               | Spielberg       | Victoria       | 442222       | 16:30       |

Dependențe funcționale pentru relația *MovieList*:

1. Title → Director
2. Cinema → Phone
3. Cinema, Time → Title

Fie  $r$  instanța unei relații  $R$

- Spunem că  $r$  **satisfacă DF**  $\alpha \rightarrow \beta$  dacă pentru orice pereche de tupluri  $t_1$  și  $t_2$  din  $r$  astfel încât  $\pi_\alpha(t_1) = \pi_\alpha(t_2)$ , este de asemenea adevărat că  $\pi_\beta(t_1) = \pi_\beta(t_2)$ .

sau

$$\forall t_1, t_2 \in r$$

$$\pi_\alpha(t_1) = \pi_\alpha(t_2) \Rightarrow \pi_\beta(t_1) = \pi_\beta(t_2) *$$

\*  $\pi_\alpha(t)$  este proiecția atributelor  $\alpha$  pentru tuplul  $t$



Fie  $r$  instanța unei relații  $R$

- o DF  $f$  este satisfăcută pe  $R$  dacă și numai dacă orice instanță  $r$  a lui  $R$  satisface  $f$
- $r$  nu respectă o DF  $f$  dacă  $r$  nu satisface  $f$ .
- $r$  este o instanță legală a lui  $R$  dacă  $r$  satisface toate dependențele funcționale definite pentru  $R$ .

## Exemplu: *Movie*(*Title*, *Director*, *Composer*)

| <i>Title</i>        | <i>Director</i> | <i>Composer</i> |
|---------------------|-----------------|-----------------|
| Schindler's List    | Spielberg       | Williams        |
| Saving Private Ryan | Spielberg       | Williams        |
| North by Northwest  | Hitchcock       | Herrmann        |
| Angela's Ashes      | Parker          | Williams        |
| Vertigo             | Hitchcock       | Herrmann        |

- DF *composer* → *director* nu este respectată de relația *Movie*
- *r* satisface DF *director* → *composer*

Acest lucru nu înseamnă că *director*→*composer* e respectat de *Movie*!

# Problema implicației

Putem deduce că o DF  $f$  e respectată de  $R$  pe baza unei mulțimi de DF  $F$  ?

**Exemplu:** în *MovieList*, avem

$$F = \{ \begin{array}{l} \text{Title} \rightarrow \text{Director} \\ \text{Cinema} \rightarrow \text{Phone} \\ \text{Cinema, Time} \rightarrow \text{Title} \end{array} \}$$

- $\text{Time} \rightarrow \text{Director}$  este respectată?
- Dar  $\text{Cinema, Time} \rightarrow \text{Director}$ ?

$F$  implică logic pe  $f$

notat prin

$$F \Rightarrow f$$

daca fiecare instanță  $r$  a relației  $R$   
ce satisface  $F$   
satisfice și  $f$

$F$  &  $G$  : mulțimi de dependențe funcționale  
 $f$  : dependența funcțională

$F$  implică logic  $G$

notat prin

$$\mathbf{F \Rightarrow G}$$

dacă  $F \Rightarrow g$  pentru fiecare  $g \in G$

Închiderea lui  $F$

(notată prin  $F^+$ )

este mulțimea tuturor DF implicate de  $F$

$$F^+ = \{f \mid F \Rightarrow f\}$$

$F$  și  $G$  sunt **echivalente**

(notat prin  $F \equiv G$ )

dacă

$$F^+ = G^+$$

(adică  $F \Rightarrow G$  și  $G \Rightarrow F$ )

# Axiomele lui Armstrong

Fie  $\alpha, \beta, \gamma \subseteq R$

**Reflexivitate:** Dacă  $\beta \subseteq \alpha$ , atunci  $\alpha \rightarrow \beta$

**Augmentare:** Dacă  $\alpha \rightarrow \beta$ , atunci  $\alpha\gamma \rightarrow \beta\gamma$

**Tranzitivitate:** Dacă  $\alpha \rightarrow \beta$  și  $\beta \rightarrow \gamma$ , atunci  $\alpha \rightarrow \gamma$



Sistemul axiomelor lui Armstrong  
este

**Corect**

(Orice FD derivată este implicată de F)

&

**Complet**

(Toate DF din  $F^+$  pot fi derivate)

*Exemplu:* Fie  $R(A, B, C, D, E)$  cu mulțimea

$$F = \{A \rightarrow C; B \rightarrow C; CD \rightarrow E\}.$$

Arătați că  $F \Rightarrow AD \rightarrow E$

*Soluție:*

1.  $A \rightarrow C$  (dat)
2.  $AD \rightarrow CD$  (augmentare cu (1))
3.  $CD \rightarrow E$  (dat)
4.  $AD \rightarrow E$  (tranzitivitate cu (2) și (3))

# Reguli de inferență adiționale

## Reuniunea:

Dacă  $\alpha \rightarrow \beta$  și  $\alpha \rightarrow \gamma$ , atunci  $\alpha \rightarrow \beta\gamma$

## Descompunerea:

Dacă  $\alpha \rightarrow \beta$ , atunci  $\alpha \rightarrow \beta'$  pentru orice  $\beta' \subseteq \beta$

*Exemplu:* Aratati ca  $\{A \rightarrow BCD\} \equiv \{A \rightarrow B; A \rightarrow C; A \rightarrow D\}$

Fie  $F = \{A \rightarrow BCD\}$

Fie  $G = \{A \rightarrow B; A \rightarrow C; A \rightarrow D\}$

Prin regula de descompunere avem

$$F \Rightarrow A \rightarrow B,$$

$$F \Rightarrow A \rightarrow C, \text{ si}$$

$$F \Rightarrow A \rightarrow D$$

Prin urmare  $F \Rightarrow G$

Din regula reuniunii avem

$$\{A \rightarrow B; A \rightarrow C\} \Rightarrow A \rightarrow BC \text{ si}$$

$$\{A \rightarrow BC; A \rightarrow D\} \Rightarrow A \rightarrow BCD$$

Prin urmare  $G \Rightarrow F$ , deci  $F \equiv G$

# Superchei, chei & attribute prime

- O mulțime de attribute  $\alpha$  reprezintă o **supercheie** a relației  $R$  (având mulțimea de DF  $F$ ) dacă

$$F \Rightarrow \alpha \rightarrow R.$$

- O mulțime de attribute  $\alpha$  e o **cheie** a relației  $R$  dacă
  - (1)  $\alpha$  este o supercheie, și
  - (2) nici o submulțime a lui  $\alpha$  nu e supercheie  
(adică, pentru fiecare  $\beta \subset \alpha$ ,  $\beta \rightarrow R \notin F^+$ )

- Un atribut  $A \in R$  se numește atribut **prim** dacă  $A$  face parte dintr-o cheie a lui  $R$ ; în caz contrar,  $A$  se numește atribut **neprim**.

- Considerăm din nou relația

*MovieList* (Title, Director, Cinema, Phone, Time)

cu DF

- (1) Cinema, Time  $\rightarrow$  Title
- (2) Cinema  $\rightarrow$  Phone
- (3) Title  $\rightarrow$  Director

- {Cinema, Time} este singura **cheie** a relației *MovieList*.
- Cinema și Time sunt singurele attribute **prime** din *MovieList*.
- Orice mulțime ce include {Cinema; Time} e **supercheie**  
a *MovieList*.

# Închiderea atributelor

Fie  $\alpha \subseteq R$  și  $F$  o mulțime de DF satisfăcute pe  $R$

■ **Închiderea lui  $\alpha$**  (cu respectarea mulțimii  $F$  de DF), notată cu  $\alpha^+$ , este mulțimea de attribute ce sunt determinate funcțional din  $\alpha$  pe baza dependențelor funcționale din  $F$ ; adică

$$\alpha^+ = \{A \in R \mid F \Rightarrow \alpha \rightarrow A\}$$

■ Se observă că  $F \Rightarrow \alpha \rightarrow \beta$  dacă și numai dacă  $\beta \subseteq \alpha^+$  (cu respectarea DF din  $F$ )

## Algorithm pt determinarea închiderii atributelor

**Input:**  $\alpha, F$

**Output:**  $\alpha^+$  (w.r.t.  $F$ )

Compute a sequence of sets of attrs  $\alpha_0, \alpha_1, \dots, \alpha_k, \alpha_{k+1}$  as follows:

$$\alpha_0 = \alpha$$

$$\alpha_{i+1} = \alpha_i \cup \gamma \text{ such that there is some FD } \beta \rightarrow \gamma \in F \text{ and } \beta \subseteq \alpha_i$$

Terminate the computation once

$$\alpha_{k+1} = \alpha_k \text{ for some } k$$

Return  $\alpha_k$



**Input:**  $\alpha, F$

**Output:**  $\alpha^+$  (w.r.t.  $F$ )

Compute a sequence of sets of attrs  $\alpha_0, \alpha_1, \dots, \alpha_k, \alpha_{k+1}$  as follows:

$$\alpha_0 = \alpha$$

$$\alpha_{i+1} = \alpha_i \cup \gamma \text{ such that there is some FD } \beta \rightarrow \gamma \in F \text{ and } \beta \subseteq \alpha_i$$

Terminate the computation once  $\alpha_{k+1} = \alpha_k$  for some  $k$

Return  $\alpha_k$

Exemple: Fie  $F = \{A \rightarrow C; B \rightarrow C; CD \rightarrow E\}$ , aratati ca  $F \Rightarrow AD \rightarrow E$

| <i>i</i> | $\alpha_i$ | <i>FD folosit</i>  |
|----------|------------|--------------------|
| 0        | AD         | dat                |
| 1        | ACD        | $A \rightarrow C$  |
| 2        | ACDE       | $CD \rightarrow E$ |
| 3        | ACDE       | -                  |

Deci  $AD^+ = ACDE$ . Deoarece  $E \in AD^+$ , rezulta ca  $F \Rightarrow AD \rightarrow E$

# Descompunerea relațiilor

**Descompunerea** unei relații  $R$

este o mulțime de (sub)relații

$$\{R_1, R_2, \dots, R_n\}$$

astfel încât fiecare  $R_i \subseteq R$  și  $R = \cup R_i$

Dacă  $r$  este o instanță din  $R$ ,  
atunci  $r$  se descompune în

$$\{r_1, r_2, \dots, r_n\},$$

unde fiecare  $r_i = \pi_{R_i}(r)$

# Proprietățile descompunerii relațiilor

## 1. Descompunerea trebuie să **păstreze informațiile**

- Datele din relația originală  $\equiv$  Datele din relațiile descompunerii
- Crucial pentru păstrarea consistenței datelor!

## 2. Descompunerea trebuie să **respecte toate DF**

- Dependențele funcționale din relația originală  $\equiv$  reuniunea dependențelor funcționale din relațiile descompunerii
- Facilitează verificarea violărilor DF

# 1. Descompunerea trebuie să păstreze informațiile

Cu alte cuvinte:  
putem reconstrui  $r$   
prin jonctiunea proiecțiilor sale  
 $\{r_1, r_2, \dots, r_n\}$

Observație: dacă  $\{R_1, R_2, \dots, R_n\}$  e o descompunere a  $R$ ,  
atunci pentru orice instanță  $r$  din  $R$ , avem

$$r \subseteq \pi_{R_1}(r) \otimes \pi_{R_2}(r) \otimes \dots \otimes \pi_{R_n}(r)$$

# Descompunerea relațiilor

$$\begin{aligned} &\{ M_1 = (\text{Cinema}, \text{Time}) \\ &\quad M_2 = (\text{Time}, \text{Title}), \\ &\quad M_3 = (\text{Title}, \text{Director}), \\ &\quad M_4 = (\text{Cinema}, \text{Phone}) \} \end{aligned}$$

e o descompunere a:

*MovieList*(Title, Director, Cinema, Phone, Time)

## Exemplu: relația *MovieList*

| <i>Title</i>            | <i>Director</i> | <i>Cinema</i>  | <i>Phone</i> | <i>Time</i> |
|-------------------------|-----------------|----------------|--------------|-------------|
| The Hobbit              | Jackson         | Florin Piersic | 441111       | 11:30       |
| The Lord of the Rings 3 | Jackson         | Florin Piersic | 441111       | 14:30       |
| Adventures of Tintin    | Spielberg       | Victoria       | 442222       | 11:30       |
| The Lord of the Rings 3 | Jackson         | Victoria       | 442222       | 14:00       |
| War Horse               | Spielberg       | Victoria       | 442222       | 16:30       |

### Constrângeri:

- Fiecare film are un regizor
- Fiecare cinematograf are un număr de telefon
- Fiecare cinematograf începe proiecția unui singur film al un moment dat

# *MovieList*(Title, Director, Cinema, Phone, Time)

***M1***

| <i>Cinema</i>  | <i>Time</i> |
|----------------|-------------|
| Florin Piersic | 11:30       |
| Florin Piersic | 14:30       |
| Victoria       | 11:30       |
| Victoria       | 14:00       |
| Victoria       | 16:30       |

***M2***

| <i>Time</i> | <i>Title</i>            |
|-------------|-------------------------|
| 11:30       | The Hobbit              |
| 14:30       | The Lord of the Rings 3 |
| 11:30       | Adventures of Tintin    |
| 14:00       | The Lord of the Rings 3 |
| 16:30       | War Horse               |

***M3***

| <i>Title</i>            | <i>Director</i> |
|-------------------------|-----------------|
| The Hobbit              | Jackson         |
| The Lord of the Rings 3 | Jackson         |
| Adventures of Tintin    | Spielberg       |
| War Horse               | Spielberg       |

***M4***

| <i>Cinema</i>  | <i>Phone</i> |
|----------------|--------------|
| Florin Piersic | 441111       |
| Victoria       | 442222       |

| <i>Title</i>            | <i>Director</i> | <i>Cinema</i>  | <i>Phone</i> | <i>Time</i> |
|-------------------------|-----------------|----------------|--------------|-------------|
| The Hobbit              | Jackson         | Florin Piersic | 441111       | 11:30       |
| The Hobbit              | Jackson         | Victoria       | 442222       | 11:30       |
| The Lord of the Rings 3 | Jackson         | Florin Piersic | 441111       | 14:30       |
| Adventures of Tintin    | Spielberg       | Florin Piersic | 441111       | 11:30       |
| Adventures of Tintin    | Spielberg       | Victoria       | 442222       | 11:30       |
| The Lord of the Rings 3 | Jackson         | Victoria       | 442222       | 14:00       |
| War Horse               | Spielberg       | Victoria       | 442222       | 16:30       |



# Descompunere cu joncțiune fără pierderi (Lossless - Join Decomposition)

O descompunere a  $R$  (având DF  $F$ ) în  
 $\{R_1, R_2, \dots, R_n\}$   
este o

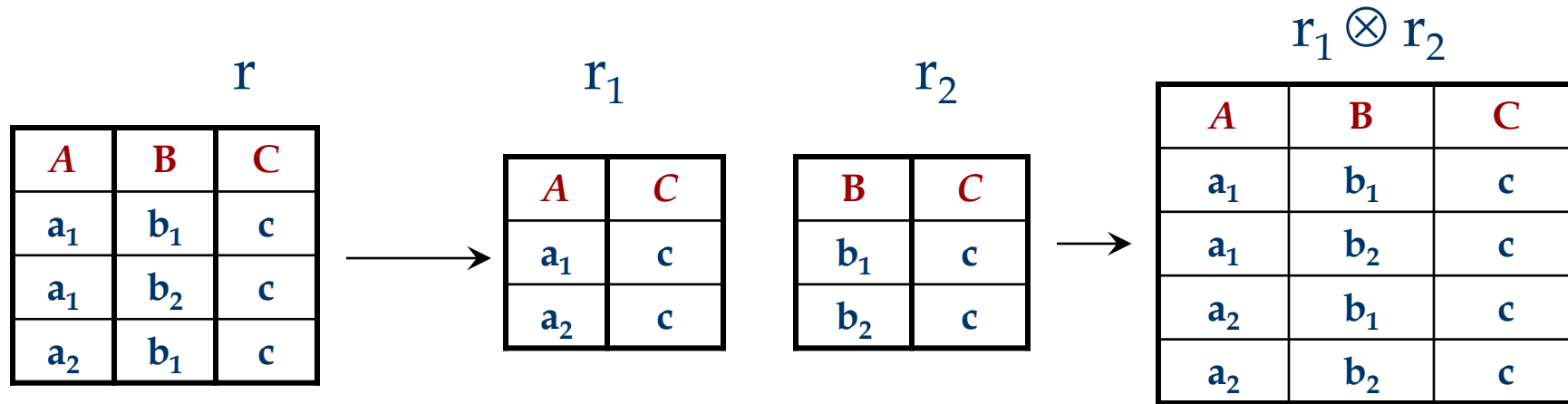
descompunere cu joncțiuni fără pierderi  
cu respectarea mulțimii  $F$

dacă

$$\pi_{R_1}(r) \otimes \pi_{R_2}(r) \otimes \dots \otimes \pi_{R_n}(r) = r$$

pentru orice instanță  $r$  din  $R$  ce satisface  $F$ .

Fie descompunere lui  $R(A,B,C)$   
in  $\{R_1(AC), R_2(BC)\}$



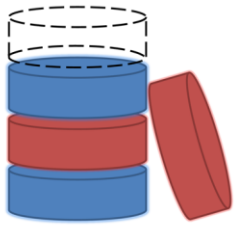
- Deoarece  $r \subset r_1 \otimes r_2$ , descompunerea **nu**  
este cu joncțiuni fără pierderi  
(*lossy decomposition*)

## Întrebarea 1

*Cum determinăm dacă  $\{R_1, R_2\}$   
este o descompunere cu joncțiuni fără  
pierderi a lui  $R$ ?*

## Întrebarea 2

*Cum descompunem  $R$  în  $\{R_1, R_2\}$  astfel  
încât aceasta e  
cu joncțiuni fără pierderi?*



## Rafinarea structurii bazelor de date

(Dependențe funcționale)

### *MovieList*

| <i>Title</i>            | <i>Director</i> | <i>Cinema</i>  | <i>Phone</i> | <i>Time</i> |
|-------------------------|-----------------|----------------|--------------|-------------|
| The Hobbit              | Jackson         | Florin Piersic | 441111       | 11:30       |
| The Lord of the Rings 3 | Jackson         | Florin Piersic | 441111       | 14:30       |
| Adventures of Tintin    | Spielberg       | Victoria       | 442222       | 11:30       |
| The Lord of the Rings 3 | Jackson         | Victoria       | 442222       | 14:00       |
| War Horse               | Spielberg       | Victoria       | 442222       | 16:30       |

### *Screens*

| <i>Cinema</i>  | <i>Time</i> | <i>Title</i>            |
|----------------|-------------|-------------------------|
| Florin Piersic | 11:30       | The Hobbit              |
| Florin Piersic | 14:30       | The Lord of the Rings 3 |
| Victoria       | 11:30       | Adventures of Tintin    |
| Victoria       | 14:00       | The Lord of the Rings 3 |
| Victoria       | 16:30       | War Horse               |

### *Movies*

| <i>Title</i>            | <i>Director</i> |
|-------------------------|-----------------|
| The Hobbit              | Jackson         |
| The Lord of the Rings 3 | Jackson         |
| Adventures of Tintin    | Spielberg       |
| War Horse               | Spielberg       |

### *Cinema*

| <i>Cinema</i>  | <i>Phone</i> |
|----------------|--------------|
| Florin Piersic | 441111       |
| Victoria       | 442222       |

$$\alpha \rightarrow \beta$$

# Descompunerea relațiilor

**Descompunerea** unei relații  $R$

este o mulțime de (sub)relații

$$\{R_1, R_2, \dots, R_n\}$$

astfel încât fiecare  $R_i \subseteq R$  și  $R = \cup R_i$

Dacă  $r$  este o instanță din  $R$ ,  
atunci  $r$  se descompune în

$$\{r_1, r_2, \dots, r_n\},$$

unde fiecare  $r_i = \pi_{R_i}(r)$

# Proprietățile descompunerii relațiilor

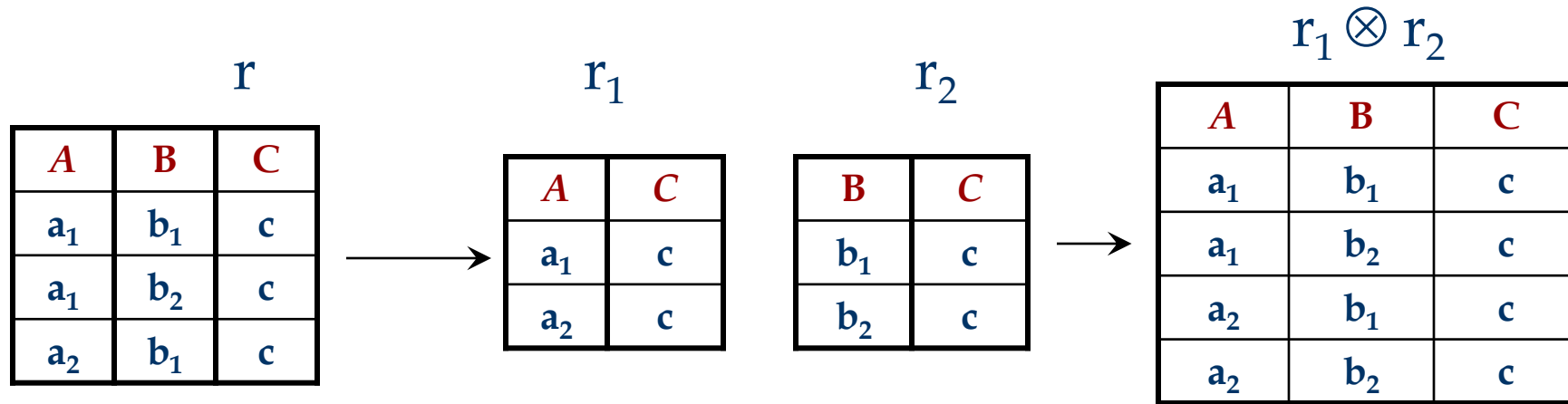
1. Descompunerea trebuie să **păstreze informațiile** (să fie cu joncțiuni fără pierderi)

- Datele din relația originală  $\equiv$  Datele din relațiile descompunerii
- Crucial pentru păstrarea consistenței datelor!

2. Descompunerea trebuie să **respecte toate DF**

- Dependențele funcționale din relația originală  $\equiv$  reuniunea dependențelor funcționale din relațiile descompunerii
- Facilitează verificarea violărilor DF

Fie descompunere lui  $R(A,B,C)$   
in  $\{R_1(AC), R_2(BC)\}$



- Deoarece  $r \subset r_1 \otimes r_2$ , descompunerea **nu**  
este cu joncțiuni fără pierderi  
(*lossy decomposition*)



## Întrebarea 1

*Cum determinăm dacă  $\{R_1, R_2\}$   
este o descompunere cu joncțiuni fără  
pierderi a lui  $R$ ?*

## Întrebarea 2

*Cum descompunem  $R$  în  $\{R_1, R_2\}$  astfel  
încât aceasta e  
cu joncțiuni fără pierderi?*

# Întrebarea 1

*Cum determinăm dacă  $\{R_1, R_2\}$   
este o descompunere cu joncțiuni fără  
pierderi a lui  $R$ ?*

**Teorema:** Descompunerea lui  $R$  (cu mulțimea  $F$  de DF) în  $\{R_1, R_2\}$  este cu joncțiuni fără pierderi cu respectarea mulțimii  $F$  dacă și numai dacă :

$$F \Rightarrow R_1 \cap R_2 \rightarrow R_1$$

sau

$$F \Rightarrow R_1 \cap R_2 \rightarrow R_2$$

## Întrebarea 2

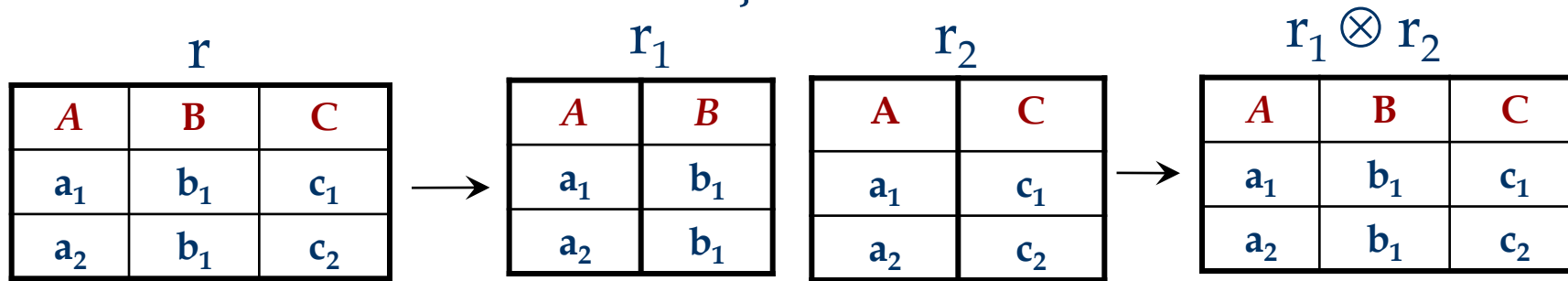
*Cum descompunem  $R$  în  $\{R_1, R_2\}$  astfel încât aceasta e cu joncțiuni fără pierderi?*

**Corolar:** Dacă  $\alpha \rightarrow \beta$  este satisfăcută pe  $R$  și  $\alpha \cap \beta = \emptyset$ , atunci descompunerea lui  $R$  în  $\{R - \beta, \alpha\beta\}$  este o descompunere cu joncțiuni fără pierderi.

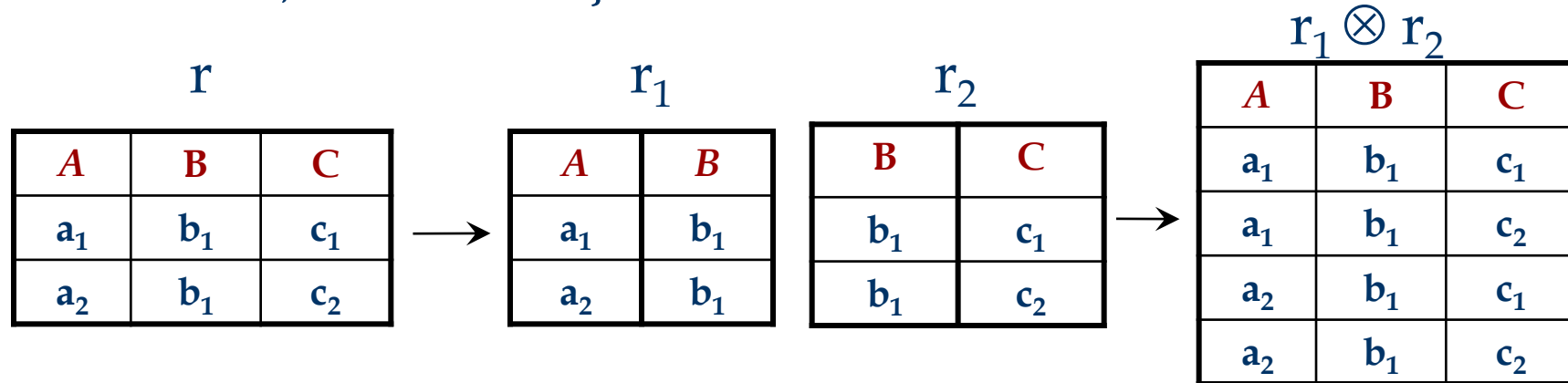
# Exemplu

- Fie  $R(A,B,C)$  cu mulțimea de dependențe funcționale  $F = \{ A \rightarrow B \}$
- Descompunerea  $\{AB, AC\}$  e cu joncțiuni fără pierderi deoarece

$$AB \cap AC = A \quad \text{și} \quad A \rightarrow AB$$



- Descompunerea  $\{AB, BC\}$  nu este cu joncțiuni fără pierderi în raport cu  $F$  deoarece  $AB \cap BC = B$ , iar  $B \rightarrow AB$  și  $B \rightarrow BC$  nu sunt satisfăcute de  $R$



## Teoremă

**Dacă**

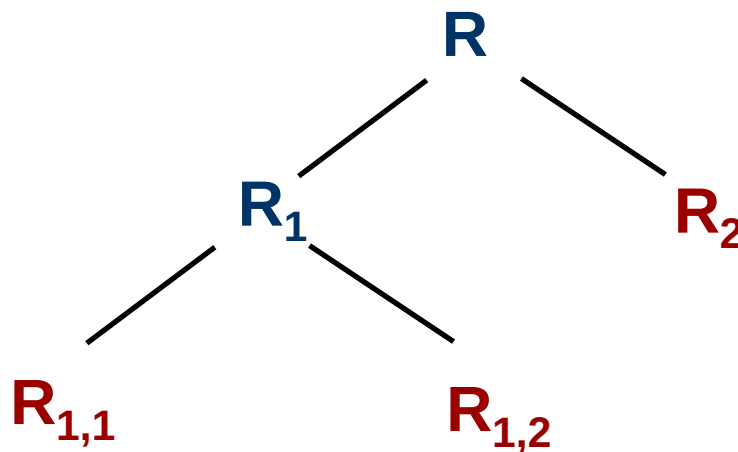
$\{\mathbf{R}_1, \mathbf{R}_2\}$  este o descompunere cu joncțiuni fără pierderi a lui  $\mathbf{R}$ ,

**și dacă**

$\{\mathbf{R}_{1,1}, \mathbf{R}_{1,2}\}$  e o descompunere cu joncțiuni fără pierderi a lui  $\mathbf{R}_1$ ,

**atunci**

$\{\mathbf{R}_{1,1}, \mathbf{R}_{1,2}, \mathbf{R}_2\}$  e o descompunere cu joncțiuni fără pierderi a  $\mathbf{R}$ :



*MovieList*

| <i>Title</i>            | <i>Director</i> | <i>Cinema</i>  | <i>Phone</i> | <i>Time</i> |
|-------------------------|-----------------|----------------|--------------|-------------|
| The Hobbit              | Jackson         | Florin Piersic | 441111       | 11:30       |
| The Lord of the Rings 3 | Jackson         | Florin Piersic | 441111       | 14:30       |
| Adventures of Tintin    | Spielberg       | Victoria       | 442222       | 11:30       |
| War Horse               | Spielberg       | Victoria       | 442222       | 14:00       |
| The Lord of the Rings 3 | Jackson         | Victoria       | 442222       | 16:30       |

*Cinema-Screens*

| <i>Cinema</i> | <i>Phone</i> | <i>Time</i> | <i>Title</i>            |
|---------------|--------------|-------------|-------------------------|
| F. Piersic    | 441111       | 11:30       | The Hobbit              |
| F. Piersic    | 441111       | 14:30       | The Lord of the Rings 3 |
| Victoria      | 442222       | 11:30       | Adventures of Tintin    |
| Victoria      | 442222       | 14:00       | War Horse               |
| Victoria      | 442222       | 16:30       | The Lord of the Rings 3 |

*Movie*

| <i>Title</i>            | <i>Director</i> |
|-------------------------|-----------------|
| The Hobbit              | Jackson         |
| The Lord of the Rings 3 | Jackson         |
| Adventures of Tintin    | Spielberg       |
| War Horse               | Spielberg       |

## *Movie*

| <i>Title</i>            | <i>Director</i> |
|-------------------------|-----------------|
| The Hobbit              | Jackson         |
| The Lord of the Rings 3 | Jackson         |
| Adventures of Tintin    | Spielberg       |
| War Horse               | Spielberg       |

## *Cinema-Screens*

| <i>Cinema</i> | <i>Phone</i> | <i>Time</i> | <i>Title</i>            |
|---------------|--------------|-------------|-------------------------|
| F. Piersic    | 441111       | 11:30       | The Hobbit              |
| F. Piersic    | 441111       | 14:30       | The Lord of the Rings 3 |
| Victoria      | 442222       | 11:30       | Adventures of Tintin    |
| Victoria      | 442222       | 14:00       | War Horse               |
| Victoria      | 442222       | 16:30       | The Lord of the Rings 3 |

## *Screens*

| <i>Cinema</i> | <i>Time</i> | <i>Title</i>         |
|---------------|-------------|----------------------|
| F. Piersic    | 11:30       | The Hobbit           |
| F. Piersic    | 14:30       | Saving Private Ryan  |
| Victoria      | 11:30       | Adventures of Tintin |
| Victoria      | 14:00       | War Horse            |
| Victoria      | 16:30       | Saving Private Ryan  |

## *Cinema*

| <i>Cinema</i> | <i>Phone</i> |
|---------------|--------------|
| F. Piersic    | 441111       |
| Victoria      | 442222       |

# Proprietățile descompunerii relațiilor

## 1. Descompunerea trebuie să **păstreze informațiile**

- Datele din relația originală  $\equiv$  Data din relațiile descompunerii
- Crucial for păstrarea consistenței datelor!

## 2. Descompunerea trebuie să **respecte toate DF**

- Dependențele funcționale din relația originală  $\equiv$  reuniunea dependențelor funcționale din relațiile descompunerii
- Facilitează verificarea violărilor DF



# Proiecția dependențelor funcționale

- Proiecția mulțimii  $F$  pe  $\alpha$  (notată prin  $F_\alpha$ ) este mulțimea acelor dependențe din  $F^+$  care implică doar attribute din  $\alpha$ , adică:

$$F_\alpha = \{ \beta \rightarrow \gamma \in F^+ \mid \beta\gamma \subseteq \alpha \}$$

- Algoritm pentru determinare proiecției DF:

**Input:**  $\alpha, F$

**Output:**  $F_\alpha$

result =  $\emptyset$ ;

for each  $\beta \subseteq \alpha$  do

$T = \beta^+$  (w.r.t.  $F$ )

    result = result  $\cup \{ \beta \rightarrow T \cap \alpha \}$

return result

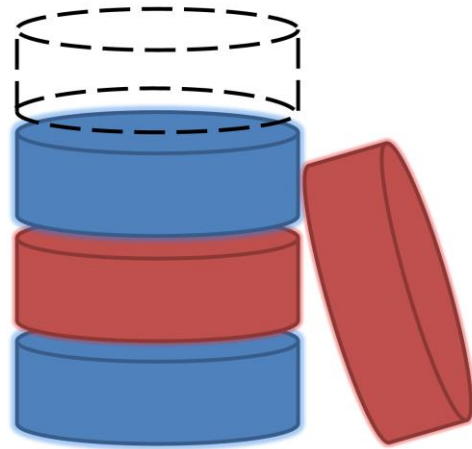
Complexitatea  
e exponențială

## Descompunere cu păstrarea dependențelor

Descompunerea  $\{R_1, R_2, \dots, R_n\}$  a relației  $R$  e cu **păstrarea dependențelor** dacă  $(F_{R_1} \cup F_{R_2} \cup \dots \cup F_{R_n})$  și  $F$  sunt echivalente, adică:

$$(F_{R_1} \cup F_{R_2} \cup \dots \cup F_{R_n}) \Rightarrow F \text{ și}$$
$$F \Rightarrow (F_{R_1} \cup F_{R_2} \cup \dots \cup F_{R_n})$$

# Forme normale



# Redundanța

*Redundanța* este cauza principală a majorității problemelor legate de structura bazelor de date relaționale:

- *spațiu utilizat,*
- *anomalii de inserare / stergere / actualizare*

# Redundanța

- *Dependențele funcționale* pot fi utilizate pentru identificarea problemelor de proiectare și sugerează posibile îmbunătățiri
- Fie relația R cu 3 attribute, ABC.
  - **Nici o DF:** nu avem redundanțe.
  - **Pentru  $A \rightarrow B$ :** Mai multe înregistrări pot avea aceeași valoare pentru A, caz în care avem valori identice pentru B!

## Tehnica de rafinare a structurii: *descompunerea*

Descompunerea trebuie folosită cu "măsură":

- Este necesară o rafinare? Există motive de descompunere a relației?
- Ce probleme pot să apară prin descompunere?

# Forme Normale

■ Dacă o relație se află într-o *formă normală* particulară avem certitudinea că anumite categorii de probleme sunt eliminate/minimizate → ne ajută să decidem dacă descompunerea unei relații este necesară sau nu.

■ Formele normale bazate pe DF sunt:

- *prima formă normală (1NF),*
- *a doua formă normală (2NF),*
- *a treia formă normală (3NF),*
- *forma normală Boyce-Codd (BCNF).*

$$\{BCNF \subseteq 3NF, 3NF \subseteq 2NF, 2NF \subseteq 1NF\}$$

# 1NF

**Definiție.** O relație se află în *Prima Formă Normală* (**1NF**) dacă fiecare atribut al relației poate avea doar valori atomice (deci listele și mulțimile sunt excluse)

(această condiție este implicită conform definiției modelului relațional)

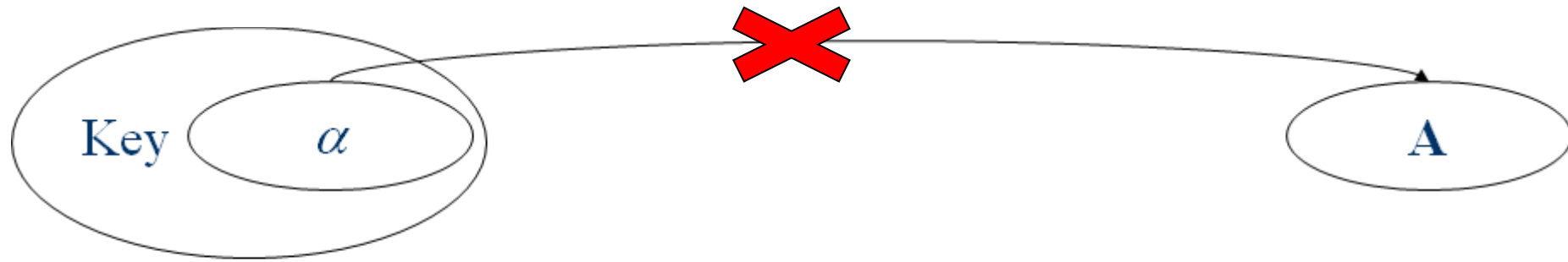


## 2NF

Spunem că avem o *dependență funcțională parțială* într-o relație atunci când un atribut *ne-cheie* este dependent funcțional de o parte a unei chei candidat a relației (dar nu de întreaga cheie).

**Definiție.** O relație se află în *A Doua Formă Normală* (**2NF**) dacă este 1NF și nu are dependențe parțiale.

# 2NF



Partial dependencies (A not in a KEY)

# BCNF

**Definiție.** O relație R ce satisface dependențele funcționale F se află în *Forma Normală Boyce-Codd* (BCNF) dacă se află în 2NF și pentru toate  $\alpha \rightarrow A$  din  $F^+$ :

- $A \in \alpha$  (DF *trivială*), sau
- $\alpha$  conține o cheie a lui R.

R este în BCNF dacă singurele dependențe funcționale satisfăcute de R sunt cele corespunzătoare constrângerilor de cheie.

# BCNF



A not in a KEY

# 3NF

**Definitie.** O relație R ce satisface dependențele funcționale F se află în *A Treia Formă Normală (3NF)* dacă se află în 2NF și pentru toate  $\alpha \rightarrow A$  din  $F^+$

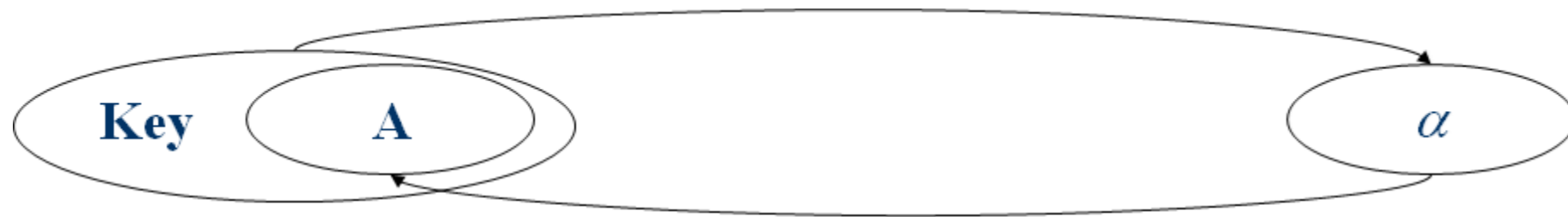
- $A \in \alpha$  (DF *trivială*), sau
- $\alpha$  conține o cheie de-a lui R, sau
- A este un atribut prim.

■ Dacă R este în BCNF, evident este și în 3NF.

■ Dacă R este în 3NF, este posibil ca să apară anumite redundanțe. Este un compromis, utilizat atunci când BCNF nu se poate atinge.

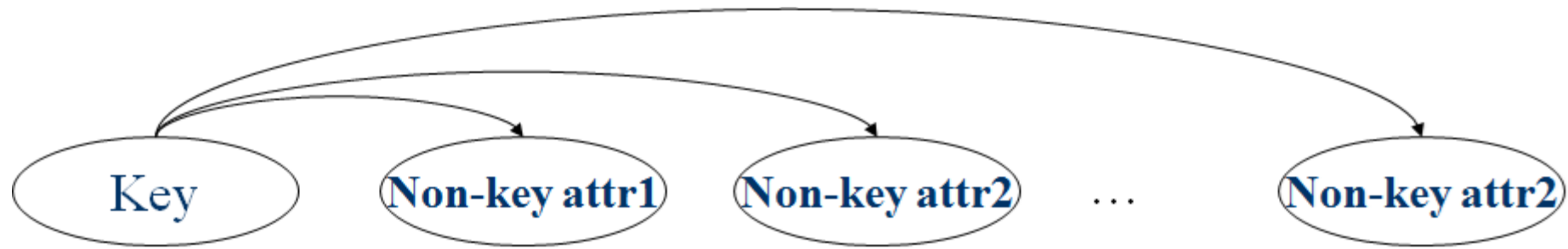
■ Descompunerea *cu joncțiune fără pierderi & cu păstrarea dependențelor* a relației R într-o mulțime de relații 3NF este întotdeauna posibilă.

# 3NF



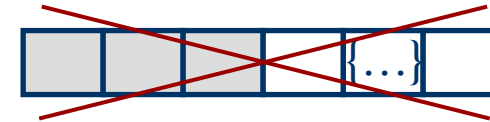
*A is in KEY*

# BCNF & 3NF

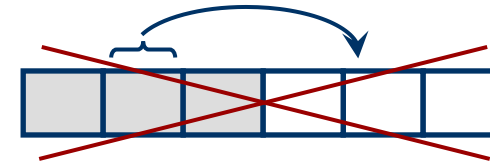
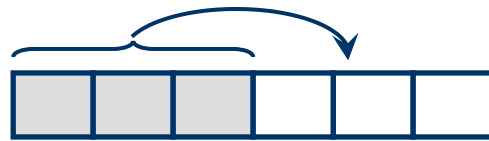


# Forme Normale bazate pe DF

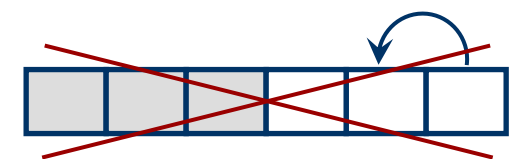
**1NF** - toate valorile atributelor sunt atomice



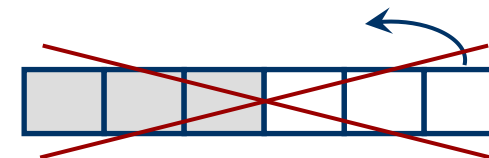
**2NF** - toate attributele non-cheie depind de **întreaga** cheie (nu sunt dependențe parțiale)



**3NF** - tabele în 2NF și toate attributele non-prime depind **doar** de cheie (nu sunt dependențe tranzitive)



**BCNF** - Toate dependențele sunt date de chei





# Normalizarea pe scurt

Fiecare atribut depinde:

de cheie, .....► definiție cheie

de întreaga cheie, .....► 2NF

și de nimic altceva

decât de cheie .....► BCNF

# Normalizarea pe scurt

Fiecare atribut <sup>neprim</sup> depinde:

- de cheie, .....► definiție cheie
- de întreaga cheie, .....► 2NF
- și de nimic altceva  
decât de cheie .....► 3NF

# Exemple de nerespectare a FN

**2NF** - toate attributele neprime trebuie să depindă de **întreaga** cheie

Exam (*Student*, *Course*, Teacher, Grade)



**3NF** - toate attributele neprime trebuie să depindă **doar** de cheie

Dissertation(*Student*, Title, Teacher, Department)



**BCNF** - toate DF sunt implicate de cheile candidat

Schedule (*Day*, *Route*, *Bus*, Driver)



# "Strategia" de normalizare

BCNF prin descompunere cu joncțiune fără pierderi și păstrarea dependențelor  
(prima alegere)

3NF prin descompunere cu joncțiune fără pierderi și păstrarea dependențelor  
(a doua alegere)

*deoarece uneori dependențele  
nu pot fi păstrate pt a obține BCNF*

# Descompunerea în BCNF

Fie relația  $R$  cu dependențele funcționale  $F$ . Dacă  $\alpha \rightarrow A$  nu respectă BCNF, descompunem  $R$  în  $R - A$  și  $\alpha A$ .

Aplicarea repetată a acestei idei va conduce la o colecție de relații care

- sunt în BCNF;
- conduc la joncțiune fără pierderi;
- garantează terminarea.

# Descompunerea în BCNF

*Exemplu:*

$R(\underline{C}, S, J, D, P, Q, V)$ , **C** cheie,

$\{JP \rightarrow C, SD \rightarrow P, J \rightarrow S\}$

Alegem  $SD \rightarrow P$ , decompunând în

$(\underline{S}, \underline{D}, P), (\underline{C}, S, J, D, Q, V)$ .

Apoi alegem  $J \rightarrow S$ , decompunând  $(\underline{C}, S, J, D, Q, V)$  în

$(\underline{J}, S)$  și  $(\underline{C}, J, D, Q, V)$

În general, mai multe dependențe pot cauza nerespectarea BCNF. Ordinea în care le ``abordăm'' poate conduce la decompuneri de relații complet diferite!

În general, descompunerea în BCNF nu păstrează dependențele.

*Exemplu.*  $R(C, S, Z)$ ,  $\{CS \rightarrow Z, Z \rightarrow C\}$

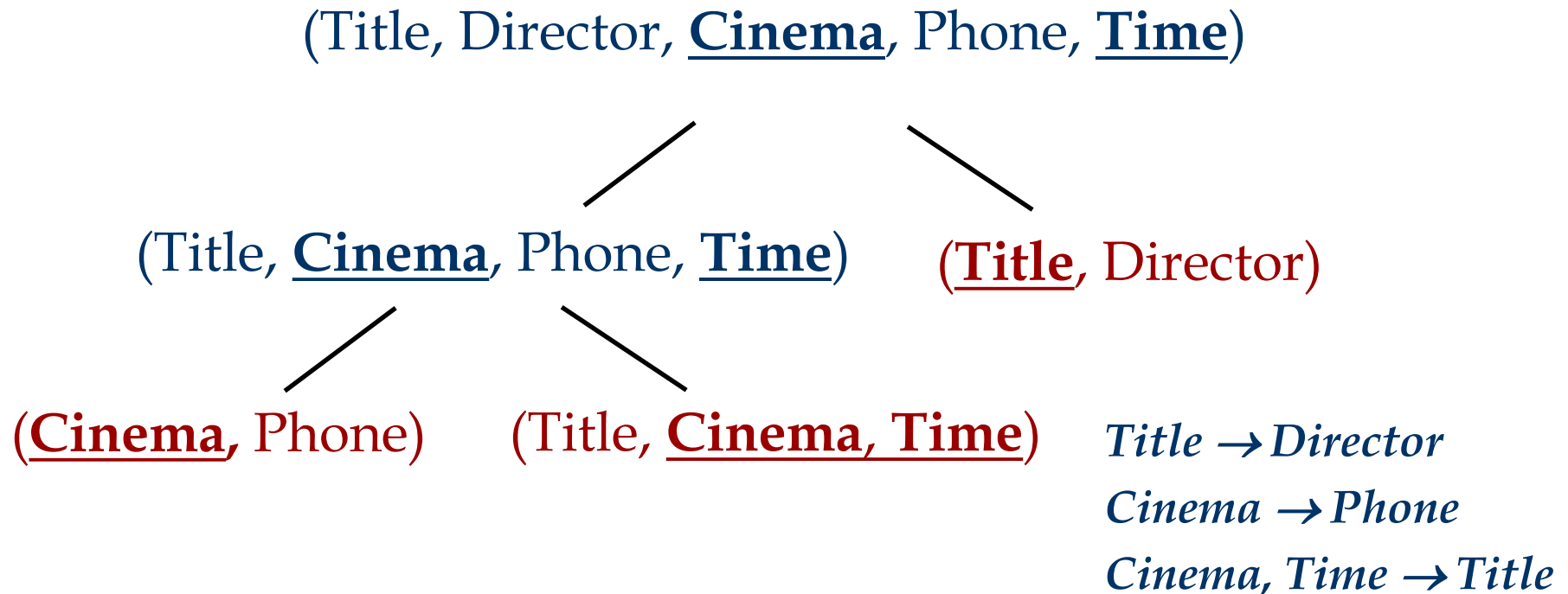
*Exemplu.*  $R(C, S, J, P, D, Q, V)$  descompus în  
(S, D, P), (J, S) și (C, J, D, Q, V)  
nu păstrează dependențele inițiale  $\{JP \rightarrow C, SD \rightarrow P, J \rightarrow S\}$ ).

! adăugând JPC la mulțimea de relații obținem *descompunere cu păstrarea dependențelor*.

⇒ BCNF & redundanță

# Exemplu

1. Fie  $\alpha \rightarrow A$  o DF din F ce nu respectă BCNF
2. Descompunem R în  $R_1 = \alpha A$  și  $R_2 = R - A$ .
3. Dacă  $R_1$  sau  $R_2$  nu sunt în BCNF, descompunerea continuă





# Descompunerea în 3NF

Evident, procedeul descompunerii din BCNF poate fi utilizat și pentru descompunerea 3NF.

- Cum asigurăm păstrarea dependențelor?
  - Dacă  $X \rightarrow Y$  nu se păstrează, adăugăm  $XY$ .
  - Problema este că  $XY$  e posibil să nu respecte 3NF! (pp. că adăugăm CJP pt `păstrarea'  $JP \rightarrow C$ . Dacă însă are loc și  $J \rightarrow C$  atunci nu e corect.)
- *Rafinare:* În loc de a utiliza mulțimea inițială  $F$ , folosim o *acoperire minimală a lui  $F$ .*

# Redundanța în DF

■ Un atribut  $A \in \alpha$  e redundant în DF  $\alpha \rightarrow B$  dacă

$$(F - \{\alpha \rightarrow B\}) \cup \{\alpha - A \rightarrow B\} \equiv F$$

■ Pentru a verifica dacă  $A \in \alpha$  e redundant în  $\alpha \rightarrow B$ , calculăm  $(\alpha - A)^+$ . Apoi  $A \in \alpha$  e redundant în  $\alpha \rightarrow B$  dacă  $B \in (\alpha - A)^+$

■ *Exercițiu:* Care sunt atributele redundante în  $AB \rightarrow C$  având:  
 $\{AB \rightarrow C, A \rightarrow B, B \rightarrow A\}$ ?

# Redundanța în DF

- O DF  $f \in F$  e **redundantă** dacă  $F - \{f\}$  e echivalent cu  $F$
- Verificăm că  $\alpha \rightarrow A$  e redundantă în  $F$ , calculând  $\alpha^+$  pe baza  $F - \{\alpha \rightarrow A\}$ . Atunci  $\alpha \rightarrow A$  e redundantă în  $F$  dacă  $A \in \alpha^+$
- *Exercițiu:* Care sunt dependențele funcționale redundante în:  
 $\{A \rightarrow C, A \rightarrow B, B \rightarrow A, B \rightarrow C, C \rightarrow A\}$ ?

# Acoperire minimală

■ O **acoperire minimală** pentru mulțimea **F** de dependente functionale este o multime **G** de dependente functionale pentru care:

1. Fiecare DF din **G** e de forma  $\alpha \rightarrow A$
2. Pt fiecare DF  $\alpha \rightarrow A$  din **G**,  $\alpha$  nu are attribute redundante
3. Nu sunt DF redundante in **G**
4. **G** și **F** sunt echivalente

Fiecare multime de DF are cel puțin o acoperire minimală!

Algoritm de calcul al acoperirii minime pt F:

1. Folosim descompunerea pentru a obține DF cu 1 atribut în partea dreaptă.
2. Se elimină attributele redundante
3. Se elimină dependențele funcționale redundante

# Calcul Acoperire Minimală

Fie  $F = \{ABCD \rightarrow E, E \rightarrow D, A \rightarrow B, AC \rightarrow D\}$

Atributele BD din  $ABCD \rightarrow E$  sunt redundante:

$\Rightarrow F = \{AC \rightarrow E, E \rightarrow D, A \rightarrow B, AC \rightarrow D\}$

$AC \rightarrow D$  este redundanta

$\Rightarrow F = \{AC \rightarrow E, E \rightarrow D, A \rightarrow B\}$

care este o acoperire minimala

*Acoperirile minimale nu sunt unice  
(depind de ordinea de alegere a DF/atr. redundante)*

# Decompunere în 3NF

**Input:** Schema  $R$  with  $F$  which is a minimal cover

**Output:** A dependency-preserving, lossless-join 3NF decomposition of  $R$

Initialize  $D = \emptyset$

Apply *union rule* to combine FDs in  $F$  with same L.H.S. into a single FD

For each FD  $\alpha \rightarrow \beta$  in  $F$  do

    Insert the relation schema  $\alpha\beta$  into  $D$

    Insert  $\delta$  into  $D$ , where  $\delta$  is some key of  $R$

Remove redundant relation schema from  $D$  as follows:

    delete  $R_i$  from  $D$  if  $R_i \subseteq R_j$ , where  $R_j \in D$

return  $D$

# Exemplu

Fie  $R(A,B,C,D,E)$  cu dependentele functionale:

$$F = \{ABCD \rightarrow E, E \rightarrow D, A \rightarrow B, AC \rightarrow D\}$$

- Acoperirea minimala a  $F$  este  $\{AC \rightarrow E, E \rightarrow D, A \rightarrow B\}$
- Unica cheie:  $AC$
- $R$  nu e in 3NF deoarece  $A \rightarrow B$  nu respecta 3NF
- descompunerea 3NF a  $R$ :
  - Relatii pentru fiecare DF:  $R_1(A, C, E)$ ,  $R_2(E,D)$ , si  $R_3(A,B)$
  - Relatie pentru cheia lui  $R$ :  $R_4(A, C)$
  - Eliminare relatie redundanta:  $R_4$  (deoarece  $R_4 \subseteq R_1$ )
  - $\Rightarrow$  descompunerea 3NF este  $\{R_1(A,C,E), R_2(E,D), R_3(A,B)\}$
- Descompunerea 3NF nu este unică. Depinde de:
  - Alegerea acoperirii minimale sau
  - Alegerea relatiei redundante care va fi eliminata



# Din nou despre... descompunere

■ Descompunerea este ultima solutie de rezolvare a problemelor generate de redundanțe & anomalii

■ Excesul poate fi nociv!

Exemplu:

$R = (\text{Teacher}, \text{Dept}, \text{Phone}, \text{Office})$

cu DF  $F = \{\text{Teacher} \rightarrow \text{Dept Phone Office}\}$

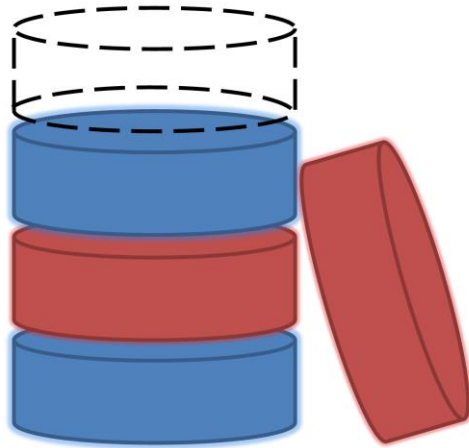
$R = (\text{Teacher}, \text{Dept}, \text{Phone}, \text{Office})$



■ Uneori, din motive de performanță se practica de-normalizarea

# Forme normale

continuare



# Dependențe multivaloare

| <b>course</b> | <b>teacher</b> | <b>book</b>  |
|---------------|----------------|--------------|
| alg101        | Green          | Alg Basics   |
| alg101        | Green          | Alg Theory   |
| alg101        | Brown          | Alg Basics   |
| alg101        | Brown          | Alg Theory   |
| logic203      | Green          | Logic B.     |
| logic203      | Green          | Logic F.     |
| logic203      | Green          | Logic intro. |

relația e în BCNF

# Dependențe multivaloare

■ Fie  $\alpha, \beta$  două submulțimi de attribute din R. Dependența multivaloare  $\alpha \twoheadrightarrow \beta$  este respectată de R dacă, pentru fiecare instanță validă r a lui R, fiecare valoare  $\alpha$  este asociată cu o mulțime de valori pentru  $\beta$  și această mulțime valori este independentă de valorile altor attribute.

■ Formal: dacă  $\alpha \twoheadrightarrow \beta$  e respectată de R și  $\gamma = R - \alpha\beta$ , următoarea afirmație e adevărată pentru orice instanță validă r a lui R:

$$t_1, t_2 \in r \text{ și } \pi_\alpha(t_1) = \pi_\alpha(t_2) \Rightarrow \\ \exists t_3 \in r \text{ a.î. } \pi_{\alpha\beta}(t_1) = \pi_{\alpha\beta}(t_3) \text{ și } \pi_\gamma(t_2) = \pi_\gamma(t_3)$$

Ca și consecință, pentru  $t_2$  și  $t_1$  se poate deduce că există și  $t_4 \in r$  a.î.  $\pi_{\alpha\beta}(t_2) = \pi_{\alpha\beta}(t_4)$  și  $\pi_\gamma(t_1) = \pi_\gamma(t_4)$

# Dependențe multivaloare

|         |          |          |          |
|---------|----------|----------|----------|
|         | <b>X</b> | <b>Y</b> | <b>Z</b> |
| $t_1$ → | a        | $b_1$    | $c_1$    |
| $t_2$ → | a        | $b_2$    | $c_2$    |
| $t_3$ → | a        | $b_1$    | $c_2$    |
| $t_4$ → | a        | $b_2$    | $c_1$    |

$\forall t_1, t_2 \in r$  și  $\pi_X(t_1) = \pi_X(t_2) \Rightarrow$   
 $\exists t_3 \in r$  astfel încât  
 $\pi_{XY}(t_1) = \pi_{XY}(t_3),$   
 $\pi_Z(t_2) = \pi_Z(t_3)$

Reguli adiționale:

**Complementare:**  $X \twoheadrightarrow Y \Rightarrow X \twoheadrightarrow R - XY$

**Augumentare:**  $X \twoheadrightarrow Y, Z \subseteq W \Rightarrow WX \twoheadrightarrow YZ$

**Tranzitivitate:**  $X \twoheadrightarrow Y, Y \twoheadrightarrow Z \Rightarrow X \twoheadrightarrow Z - Y$

**Replicare:**  $X \rightarrow Y \Rightarrow X \twoheadrightarrow Y$

**Fuzionare:**  $X \twoheadrightarrow Y, W \cap Y = \emptyset, W \rightarrow Z, Z \subseteq Y \Rightarrow X \rightarrow Z$

# A patra formă normală (4NF)

**Definiție.** Fie  $R$  o schemă relațională și  $F$  o mulțime de dependențe funcționale și multivaloare pe  $R$ .

Spunem că  $R$  este în a patra forma normală NF4 dacă este în 3NF și pentru orice dependență multivaloare  $X \twoheadrightarrow Y$ :

- $Y \subseteq X$  sau
- $XY = R$  sau
- $X$  e super-cheie

# A patra formă normală (4NF)

| course   | teacher | book         |
|----------|---------|--------------|
| alg101   | Green   | Alg Basics   |
| alg101   | Green   | Alg Theory   |
| alg101   | Brown   | Alg Basics   |
| alg101   | Brown   | Alg Theory   |
| logic203 | Green   | Logic B.     |
| logic203 | Green   | Logic F.     |
| logic203 | Green   | Logic intro. |

course $\twoheadrightarrow$ teacher

Relatia se poate descompune in:

(Course,Teacher)

si (Course,Book)

| course   | teacher |
|----------|---------|
| alg101   | Green   |
| alg101   | Brown   |
| logic203 | Green   |

| course   | book         |
|----------|--------------|
| alg101   | Alg Basics   |
| alg101   | Alg Theory   |
| logic203 | Logic B.     |
| logic203 | Logic F.     |
| logic203 | Logic intro. |

# Dependența *Join*

■ Spunem ca R satisface dependența *join*

$\otimes\{R_1, \dots, R_n\}$  dacă

$R_1, R_2, \dots, R_n$

este o descompunere cu joncțiuni fără pierderi a lui R.

O dependență multivaloare  $X \twoheadrightarrow Y$  poate fi exprimată ca o dependență join:

$\otimes\{XY, X(R-Y)\}.$

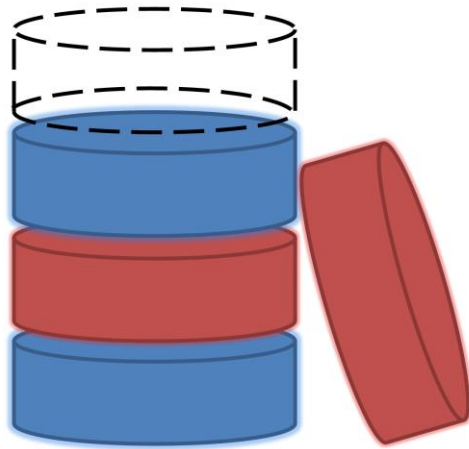


## A cincea formă normală (5NF)

O relație  $R$  este în NF5 dacă și numai dacă pentru orice dependență *join* a lui  $R$ :

- $R_i = R$  pentru un  $i$  oarecare, sau
- dependența este implicată de o mulțime de dependențe functionale din  $R$  în care partea stângă e o cheie pentru  $R$

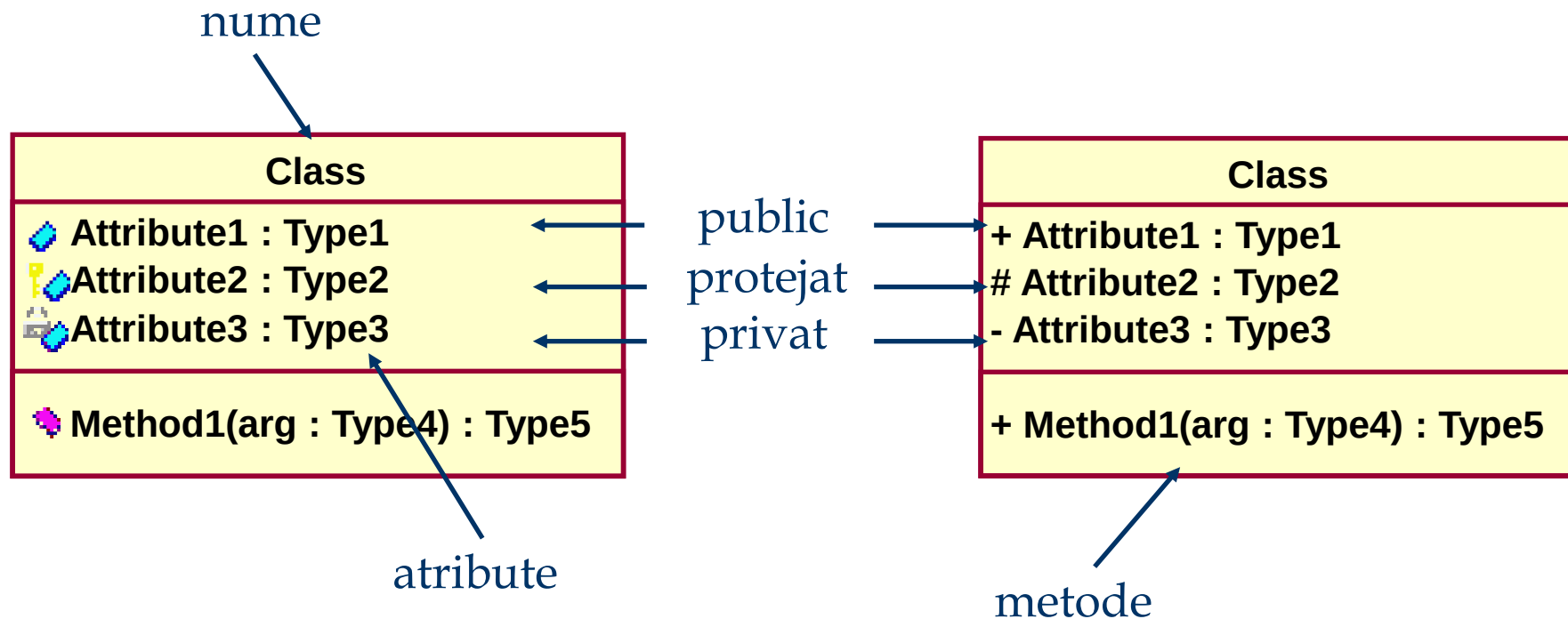
# Proiectarea bazelor de date



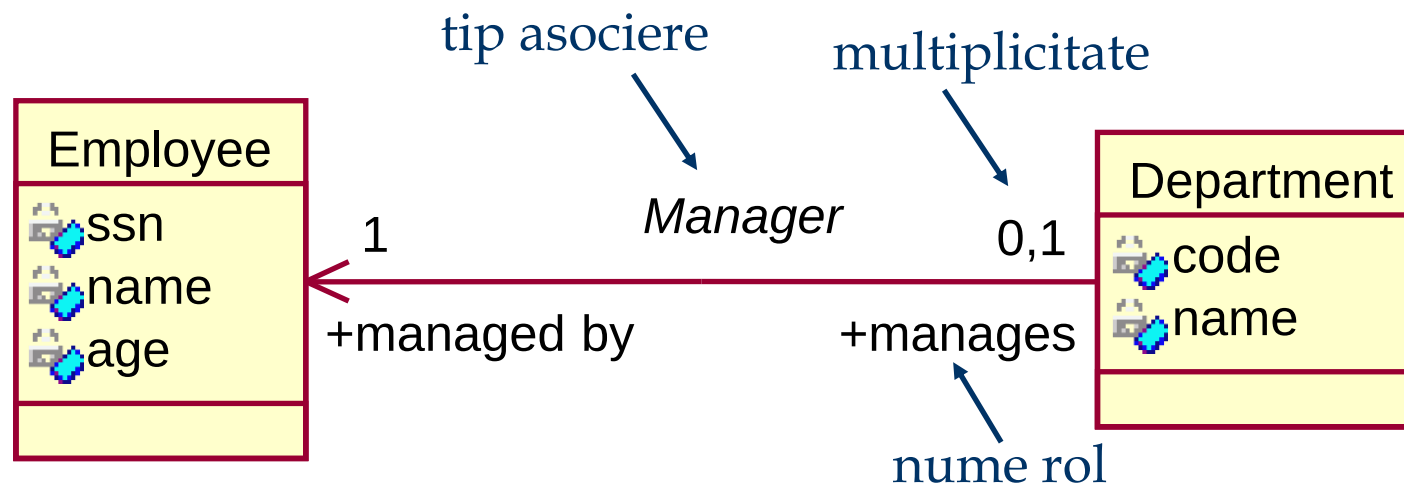
# Proiectarea bazelor de date

- **Proiectare conceptuală** (ex. diagrama de clase)
  - Identificarea entităților și a relațiilor dintre ele
- **Proiectarea logică**
  - Transformarea modelului conceptual într-o structură de baze de date (relațională sau nu)
- **Rafinarea bazei de date (normalizare)**
  - Eliminarea redundanțelor și a problemelor conexe
- **Proiectare fizică și eficientizare**
  - Indexare
  - De-normalizare!

# Diagrama de clase UML - Clase



# Diagrama de clase UML - Asocieri



## ■ Multiplicități:

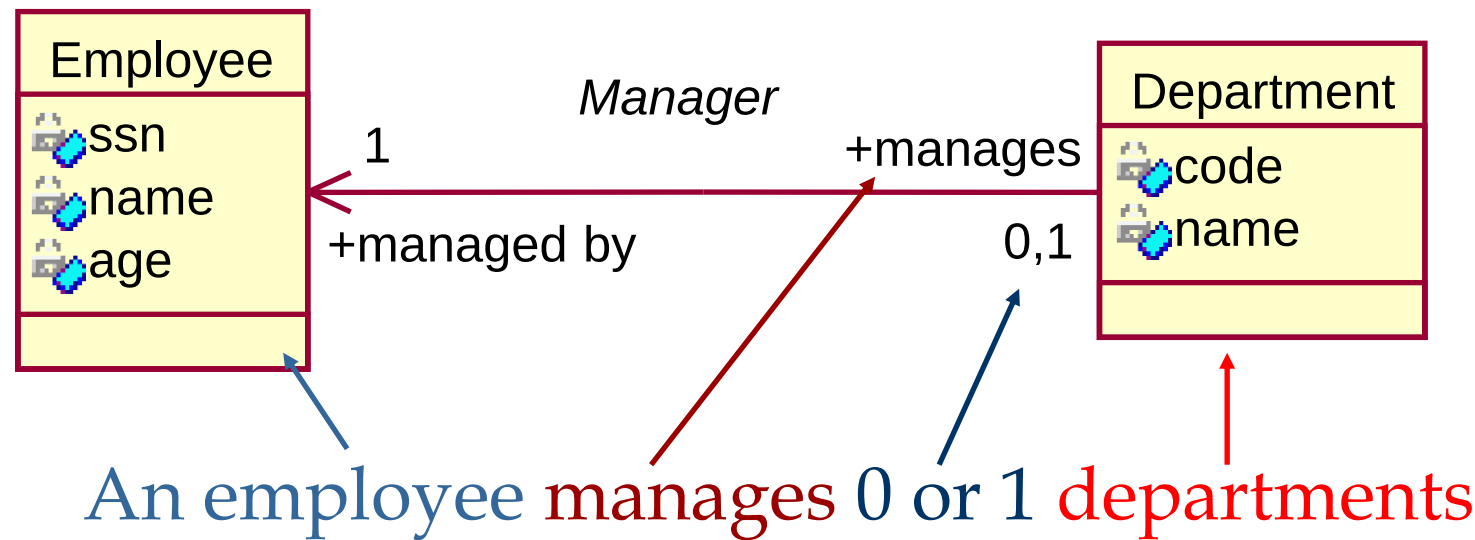
- valori: 4,5
- intervale: 1..10
- nedefinit: \*

## ■ Navigabilitatea asocierii:

- un sens
- bidirectional

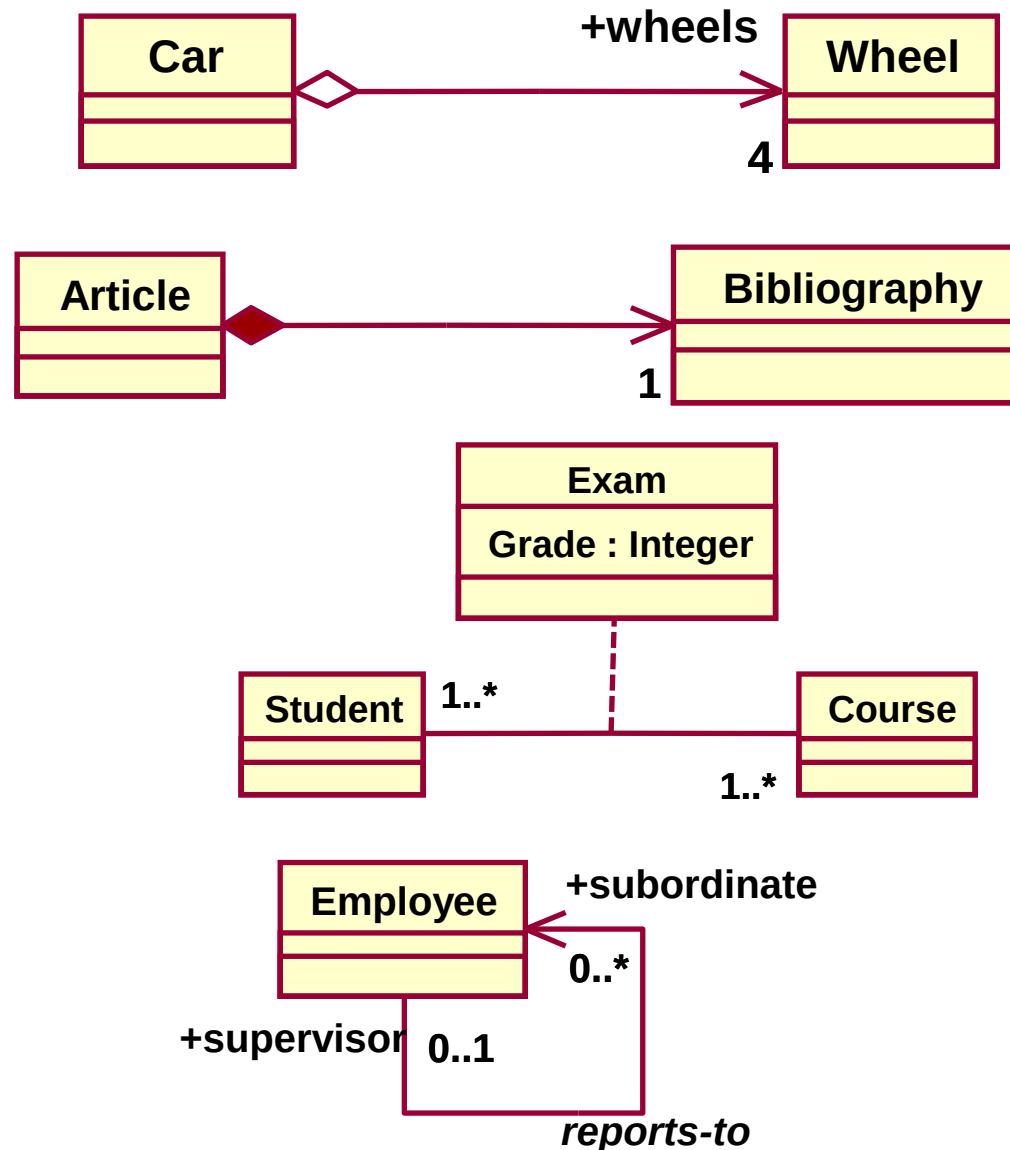
# Diagrama de clase UML - Asociieri

## ■ Citirea numelor de rol

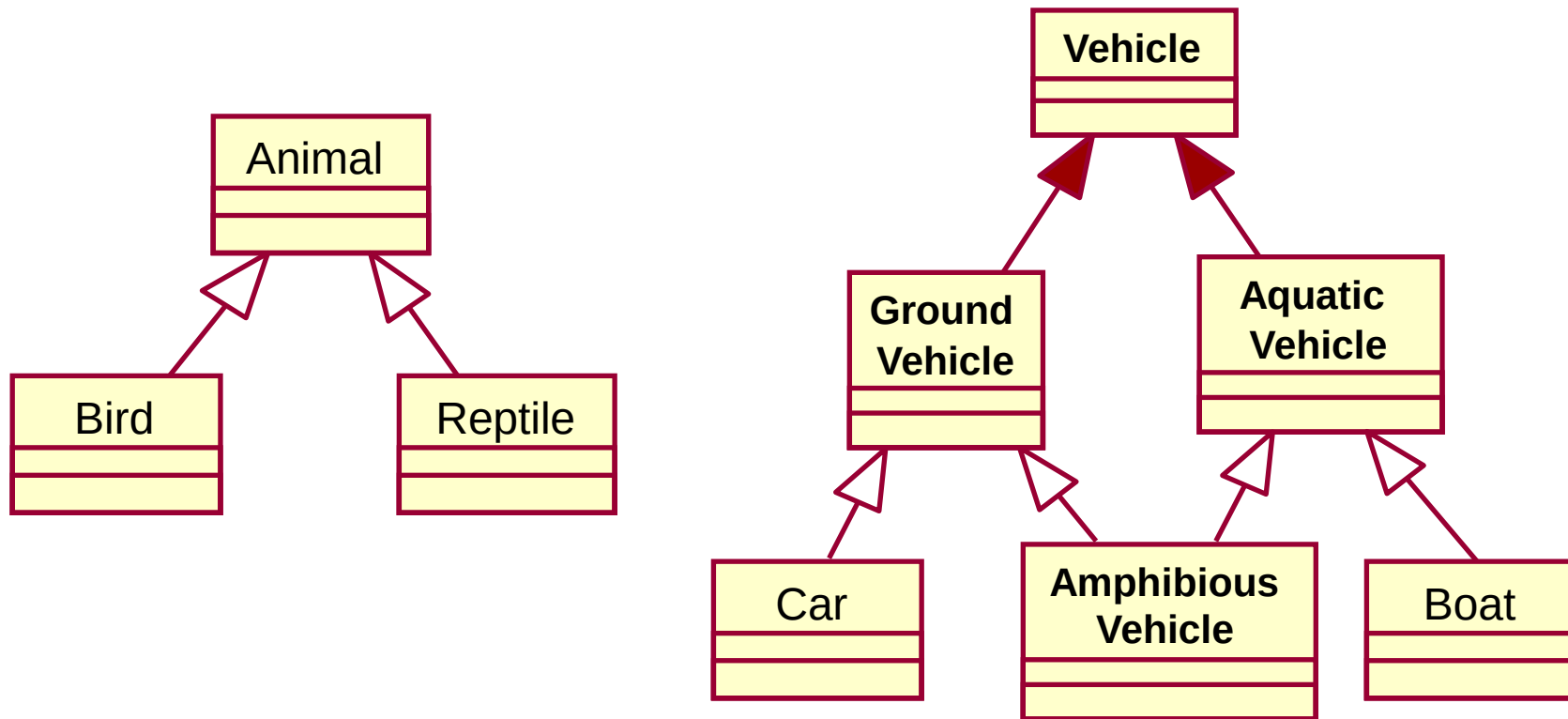


# Diagrama de clase UML - Asocieri

- Agregare
  - asociere parte-intreg
- Compunere
  - “weak entities”
- Clasa asociere
- Asociere reflexiva



# Diagrama de clase UML - Mostenire



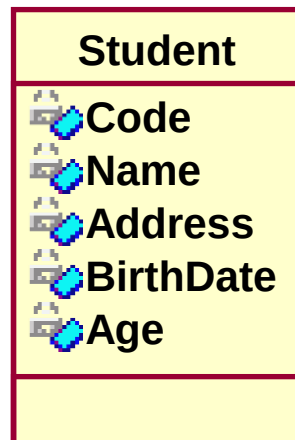


# Modelul conceptual $\Rightarrow$ bază de date relațională

- Transformare 1:1 a claselor în tabele:
  - Prea multe tabele – pot rezulta mai multe tabele decât este necesar
  - Prea multe op. join – consecință imediată a faptului că se obțin prea multe tabele
  - Tabele lipsă – asocierile m:n între clase implică utilizarea unei tabele speciale (*cross table*)
  - Tratarea necorespunzătoare a moștenirii
  - Denormalizarea datelor – anumite date se regăsesc în mai multe tabele

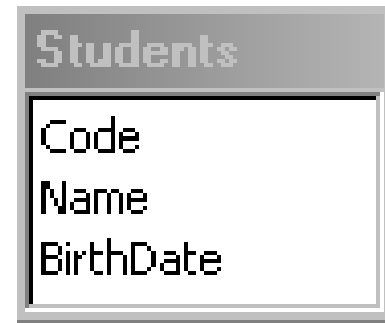
# Transformarea claselor în tabele

- Numele tabelului reprezintă pluralul numelui clasei
- Toate attributele simple sunt transformate în câmpuri
- Attributele compuse devin tabele de sine stătătoare
- Attributele derivate nu vor avea nici un corespondent în tabelă



Students (Code, Name, BirthDate)

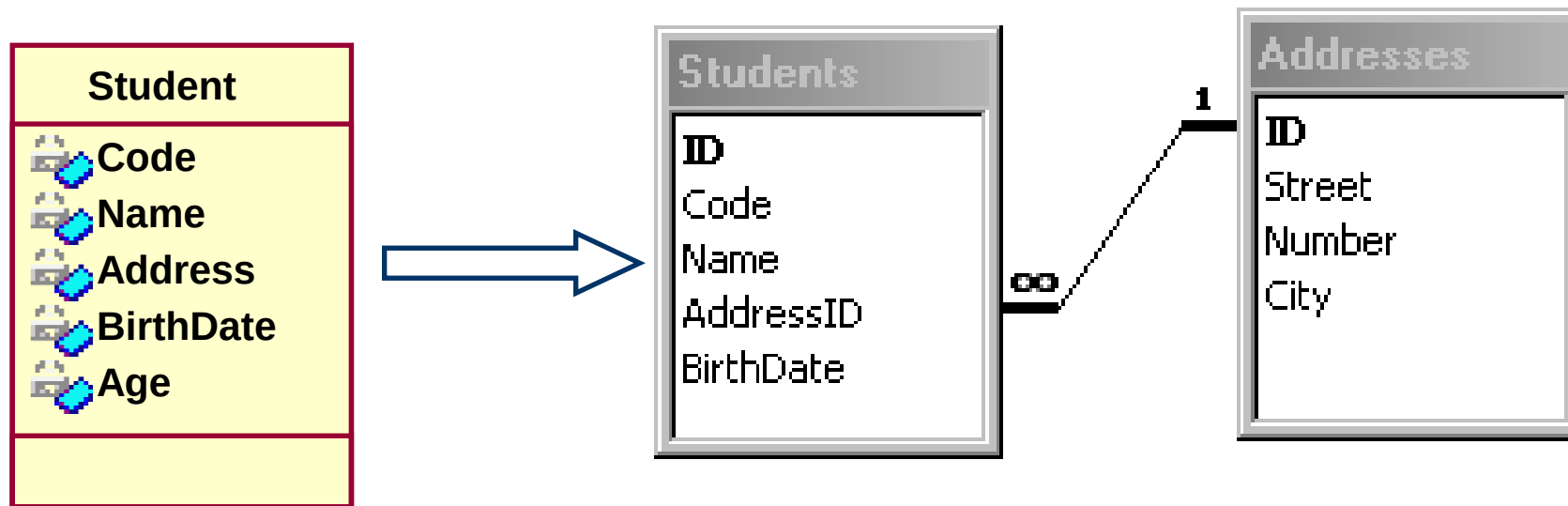
Addresses (Street, Number, City)



# Transformarea claselor în tabele

- Chei surogat – chei care nu sunt obținute din domeniul problemei modelate
- Conceptul de cheie nu este definit în cadrul claselor UML
- O *bună practică*: utilizarea (atunci când este posibil) a cheilor de tip întreg generate automat de SGBD:
  - ușor de întreținut (responsabilitatea sistemului)
  - eficient (interogări rapide)
  - simplifică definirea cheilor străine
- Disciplină de proiectare a BD:
  - toate cheile surogat vor fi numite **ID**
  - toate cheile străine se numesc **<NumeTabel>ID**

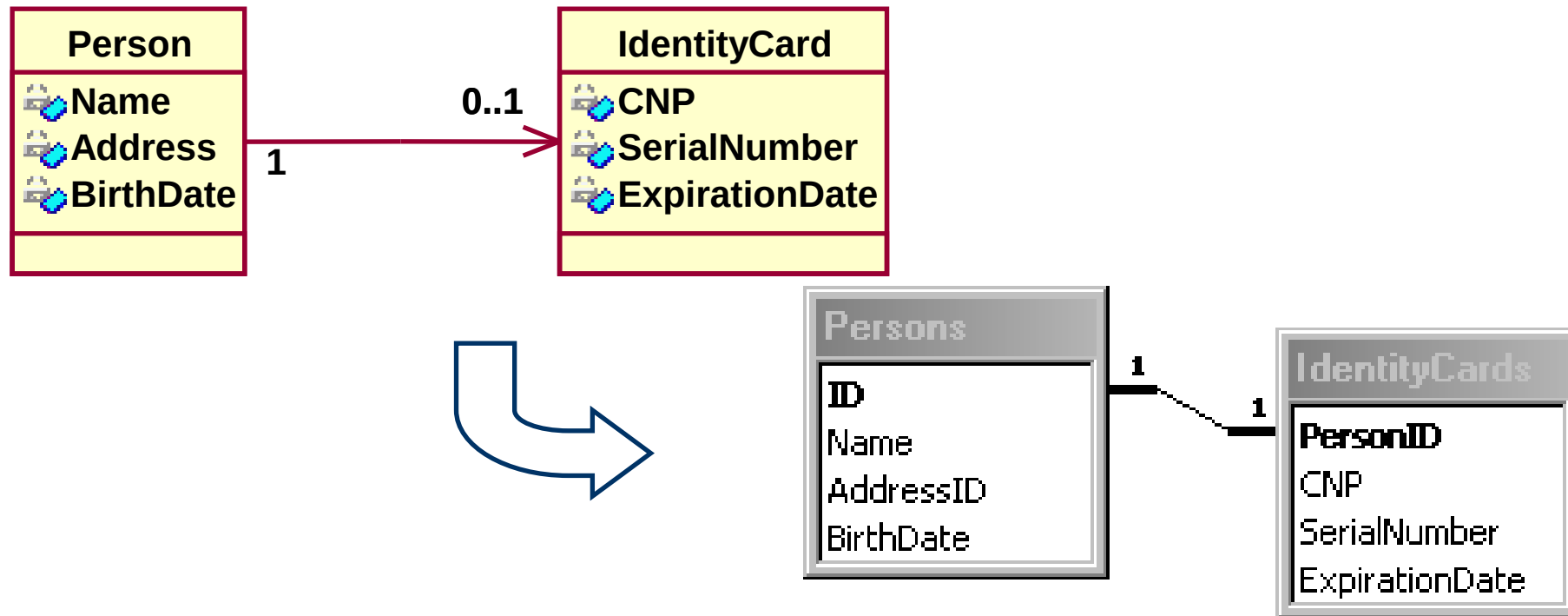
## Transformarea claselor în tabele (cont)



# Transformarea asocierilor simple

## ■ 1 : 0,1

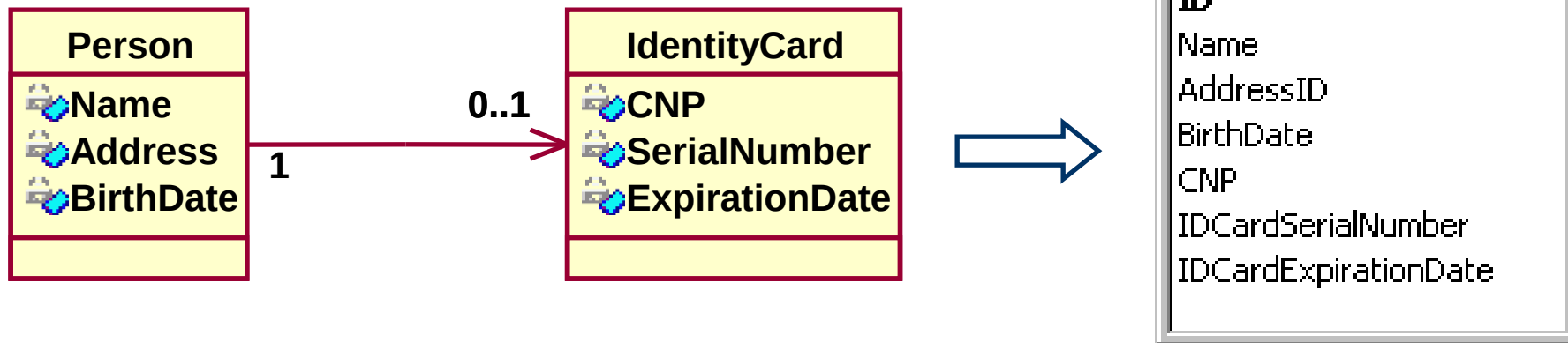
- se crează câte o tabelă corespunzătoare fiecărei clase implicate în asociere
- cheia tablei corespunzătoare multiplicității “0, 1” este cheia străină în cea de-a doua tabelă
- o singură cheie va fi generată automat (de obicei cea corespunzătoare multiplicității “1”)



# Transformarea asocierilor simple (cont)

## ■ 1 : 1

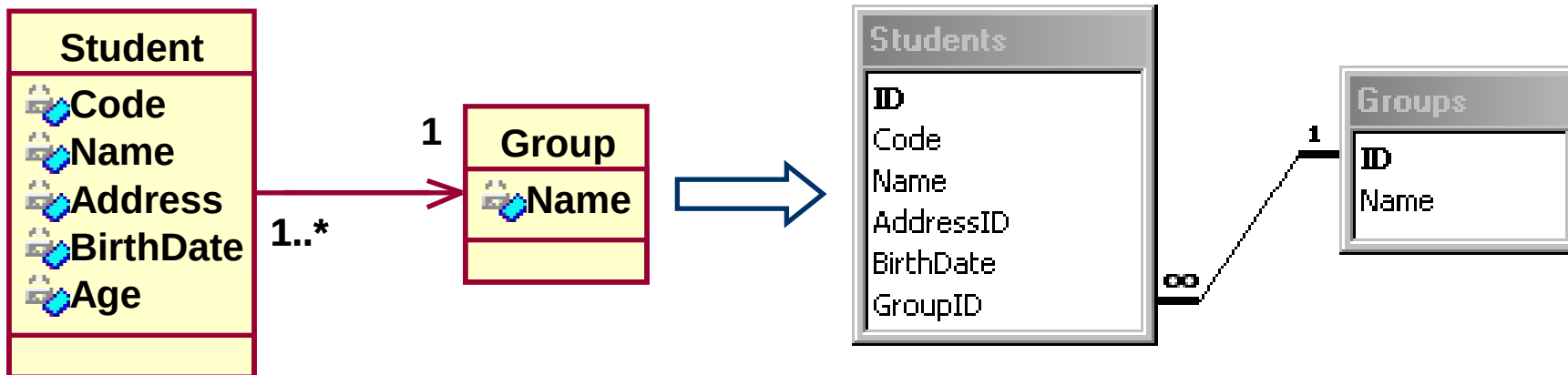
- se crează o singură tabelă ce conține attributele ambelor clase asociate
- aceasta variantă de transformare se aplică și asocierilor “1 : 0,1” atunci când este vorba de un număr relativ mic de cazuri în care obiectele primei clase nu sunt legate de obiectele celei de-a doua clase



# Transformarea asocierilor simple (cont)

## ■ 1 : 1..\*

- se crează câte o tabelă corespunzătoare fiecărei clase implicate în asociere
- cheia tabelii corespunzătoare multiplicității “ 1 ” este cheia străină în cea de-a doua tabelă, corespunzătoare multiplicității “1..\*”



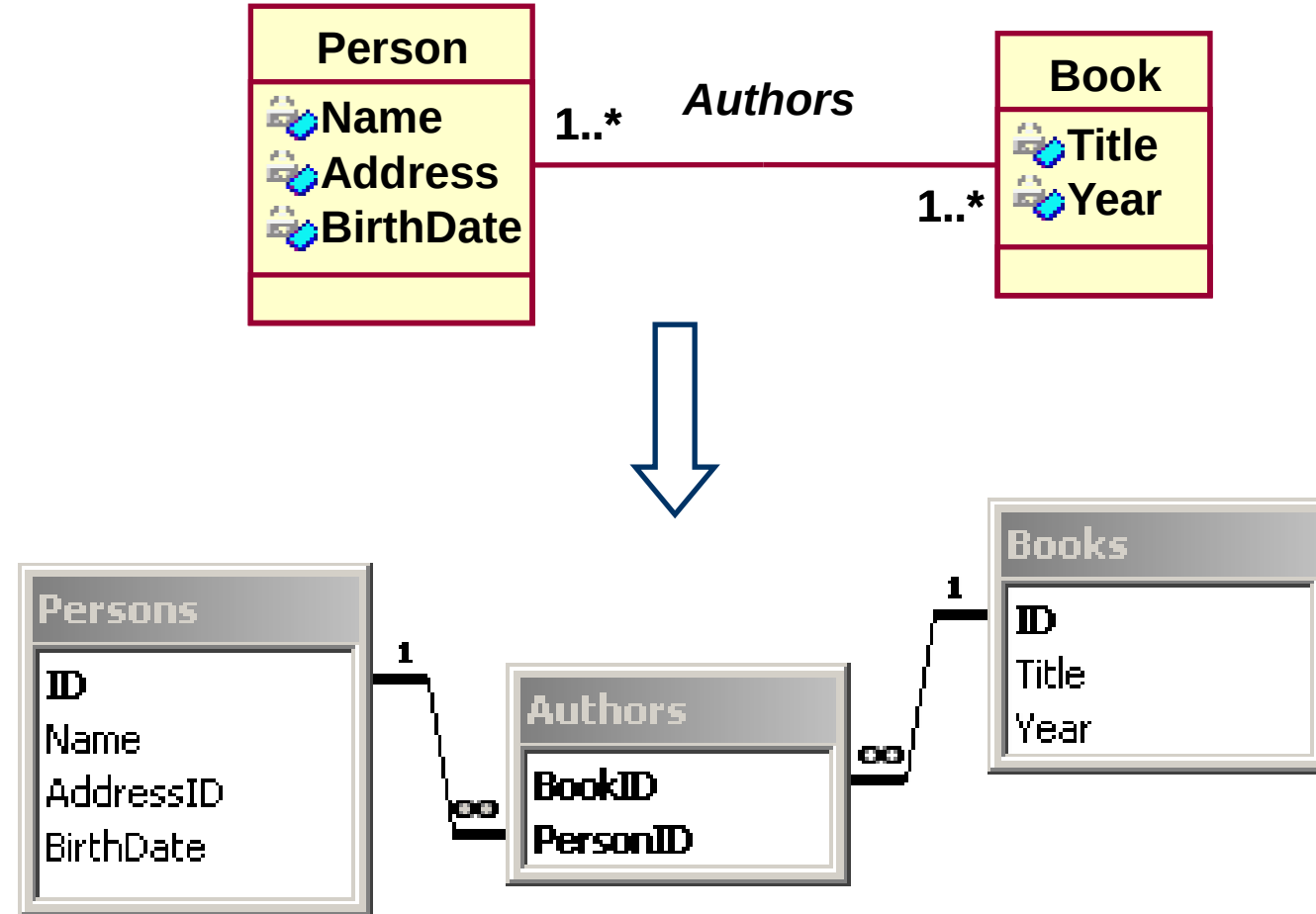
# Transformarea asocierilor simple (cont)

## ■ 1..\* : 1..\*

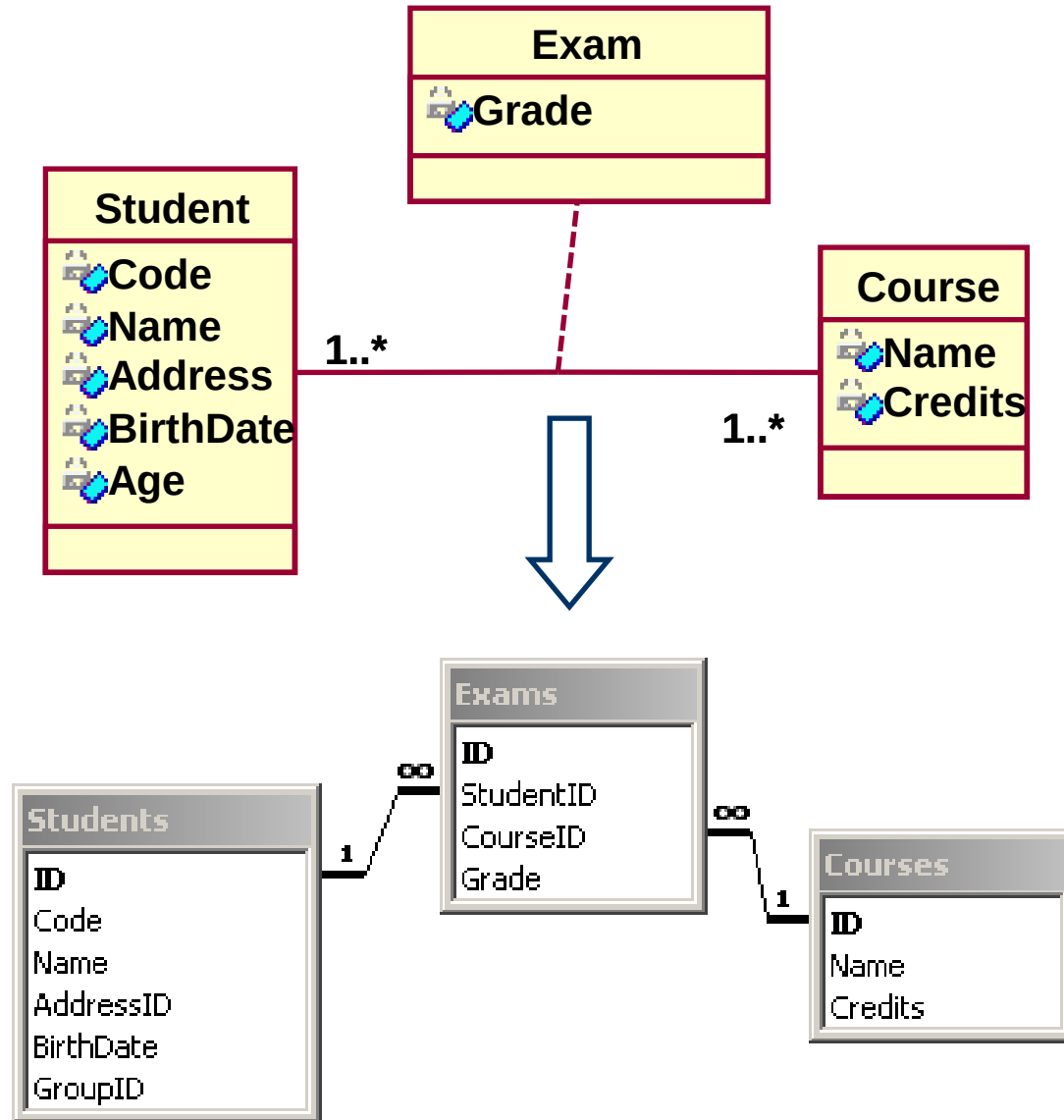
- se crează câte o tabelă corespunzătoare fiecărei clase implicate în asociere
- se crează o tabelă adițională numită tabelă de intersecție (*cross table*)
- cheile primare corespunzătoare tabelelor inițiale sunt definite ca și chei străine în tabela de intersecție
- cheia primară a tabelii de intersecție este, de obicei, compusă din cele două chei străine spre celelalte tabele. Sunt cazuri în care se utilizează și aici cheie surogat.
- dacă asocierea conține o clasă asociere, toate atributele acestei clase vor fi inserate în tabela de intersecție
- uzual, numele tabelii de intersecție este o combinație a numelor tabelilor inițiale dar acest lucru nu este necesar.



# Transformarea asocierilor simple (cont)



# Transformarea asocierilor simple (cont)



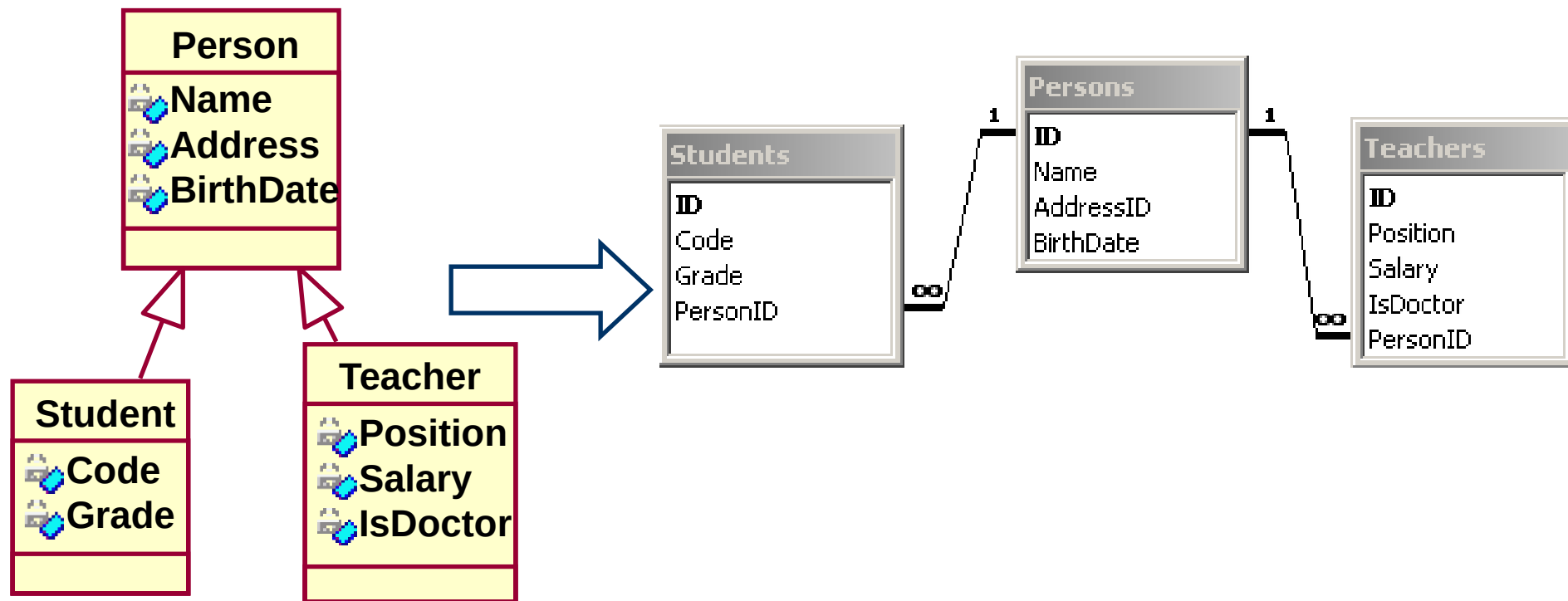
# Transformarea moștenirii

## Metoda 1

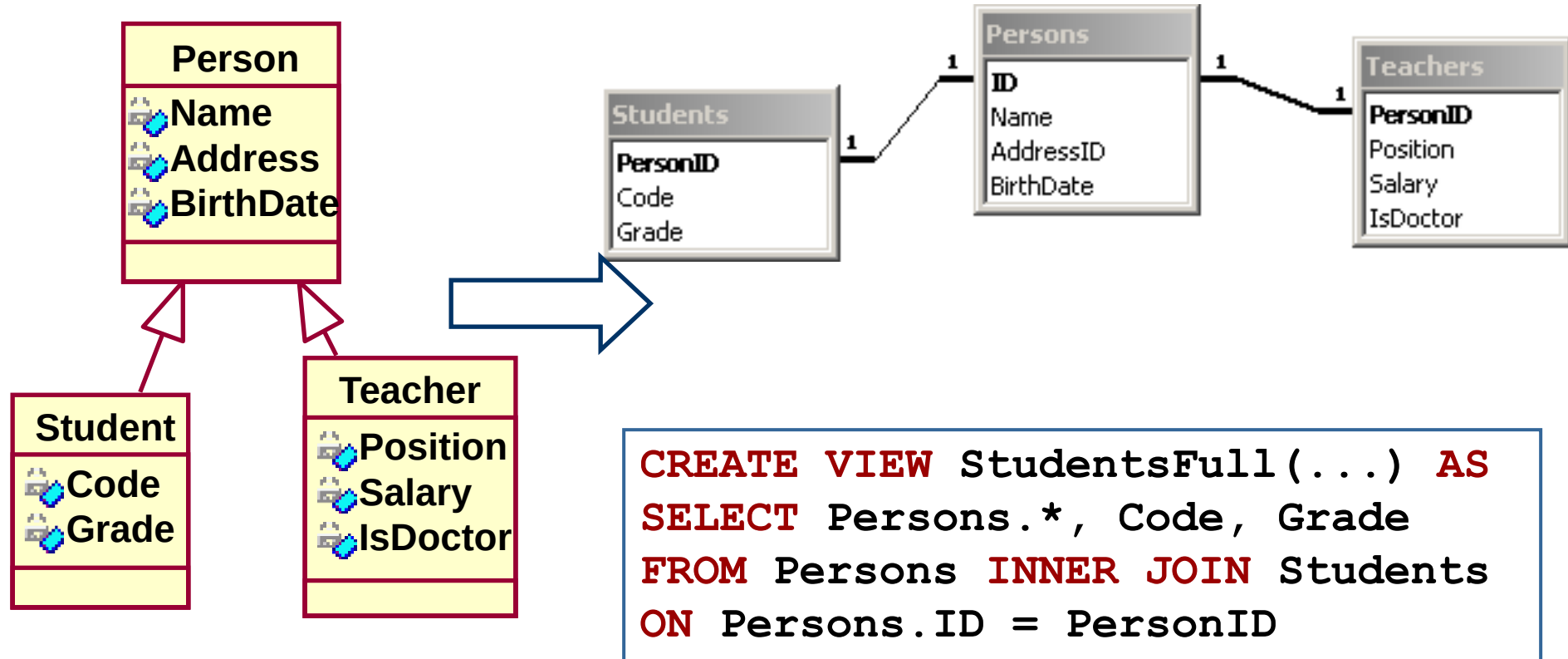
Presupune crearea câte unui tabel corespunzător fiecărei clase și a câte unui *view* pentru fiecare pereche super-clasă/subclasă

- Flexibilitate – permite adăugarea viitoarelor subclase fără impact asupra tabelelor/*view*-urilor deja existente
- Implică crearea celor mai multe tabele/*view*-uri
- Posibile probleme de performanță deoarece fiecare access va implica execuția unui *join*

# Transformarea moștenirii



# Transformarea moștenirii



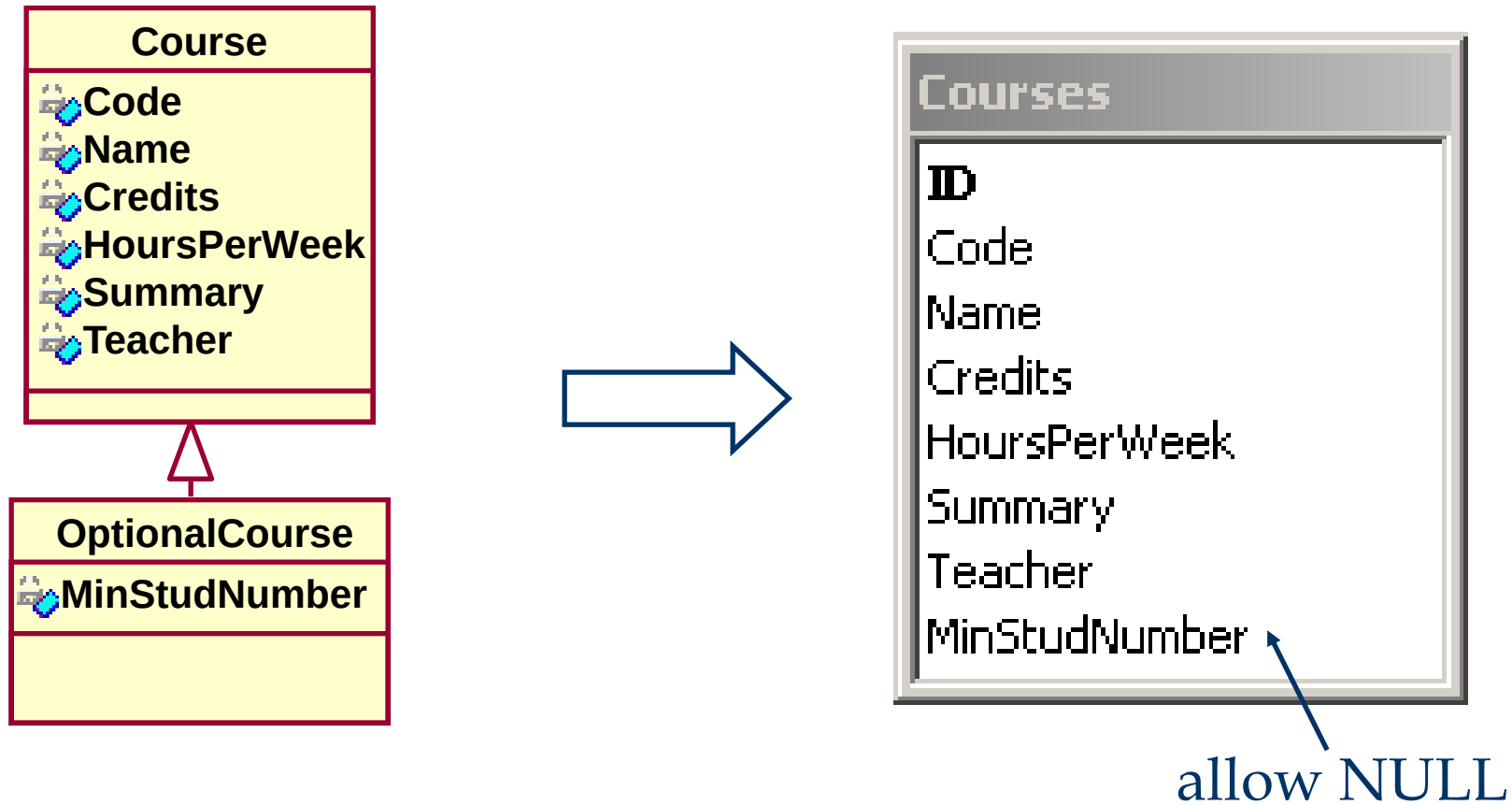
# Transformarea moștenirii

## Metoda 2

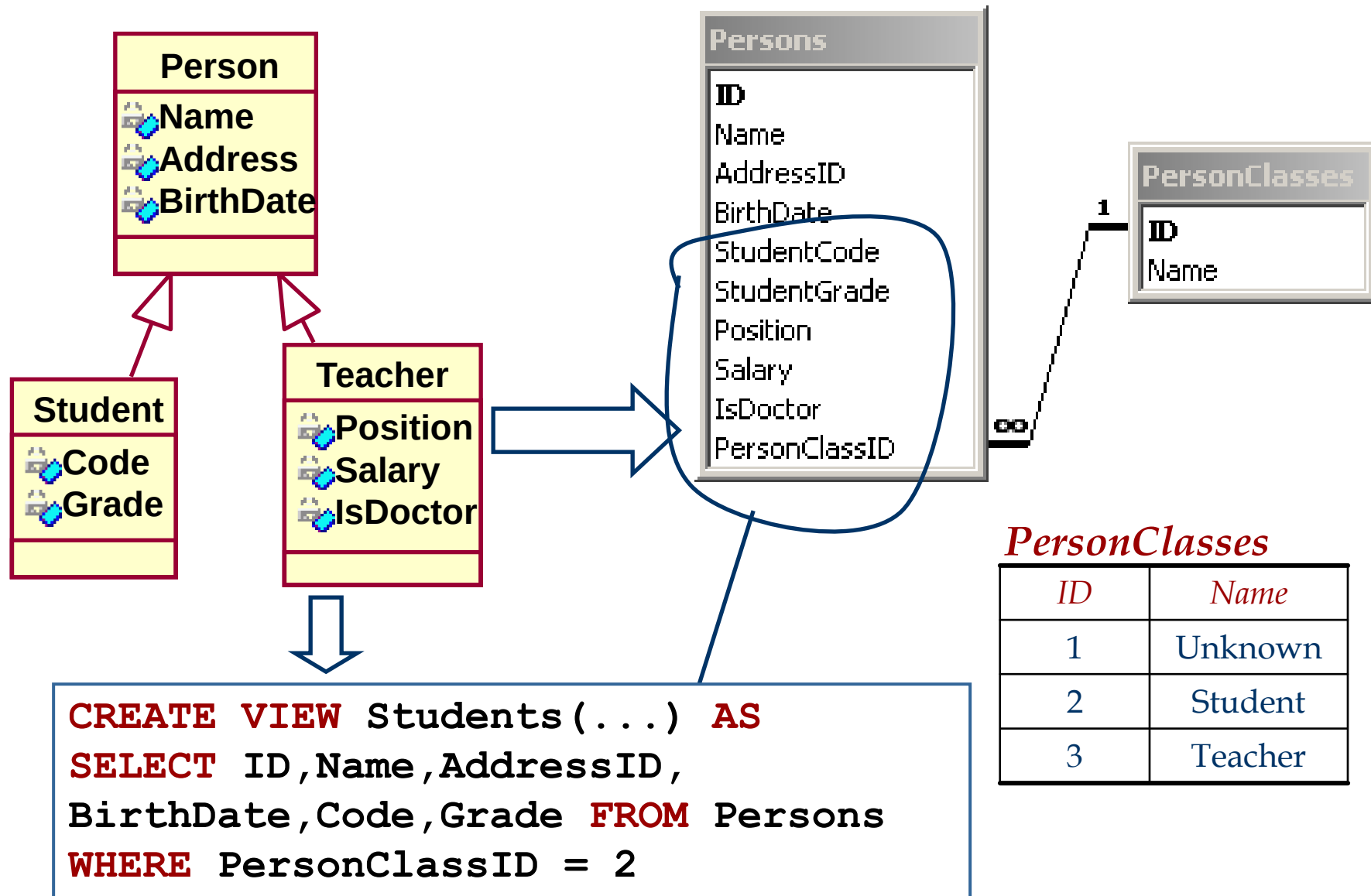
Se crează o singură tabelă (corespunzătoare superclasei) și se de-normalizează toate attributele subclaselor acesteia.

- Implică crearea celor mai puține tabele/*view*-uri - optional, se poate defini o tabelă de subclase și *view*-uri corespunzătoare fiecărei subclase.
- Se obține, de obicei, cea mai mare performanță
- Adăugarea unei noi subclase implică modificări structurale
- Creștere “artificială” a spațiului utilizat

# Transformarea moștenirii



# Transformarea moștenirii





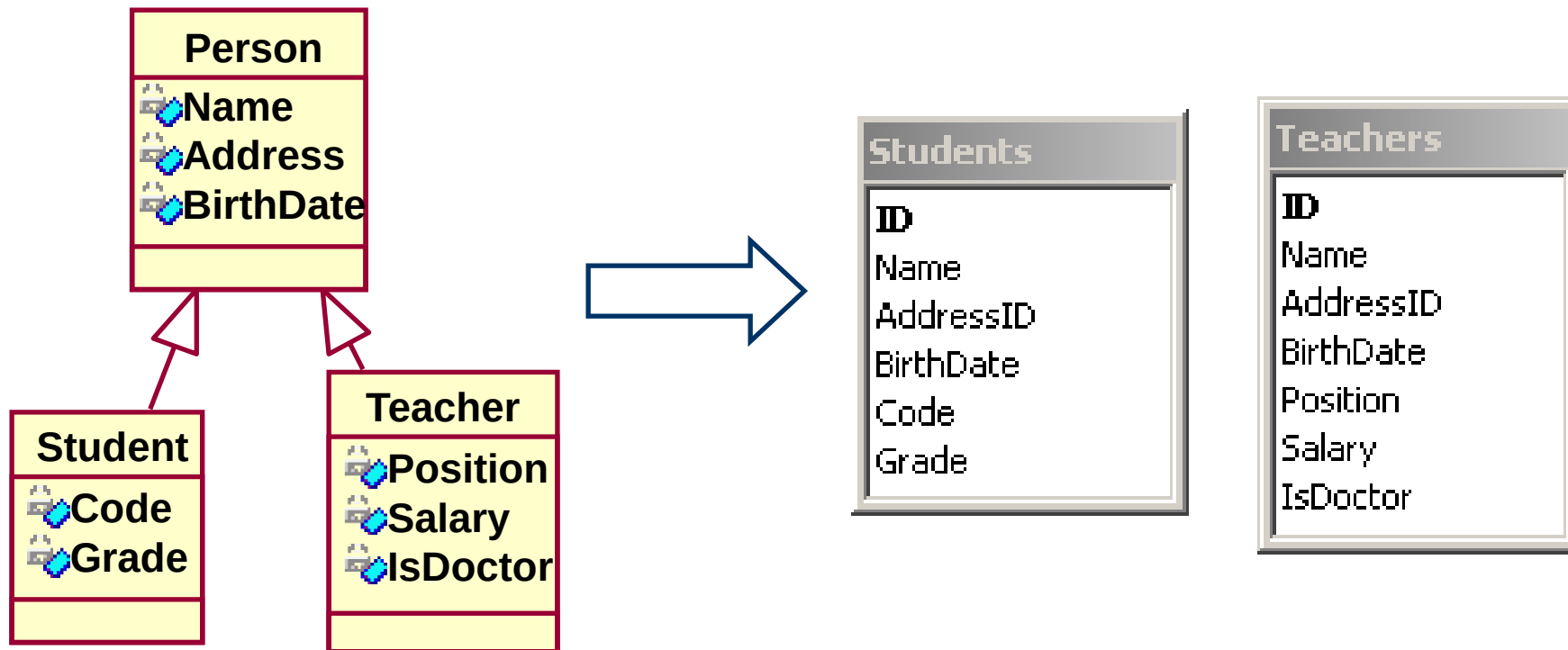
# Transformarea moștenirii

## Metoda 3

Presupune crearea câte unui tabel corespunzător fiecărei sub-clase și de-normalizarea atributelor super-clasei în fiecare dintre tabelele create

- Performanța obținută este satisfăcătoare
- Adăugarea unei noi subclase **nu** implică modificări structurale
- Posibilele modificări structurale la nivelul superclasei afectează toate tabelele definite!

# Transformarea moștenirii



# Transformarea moștenirii

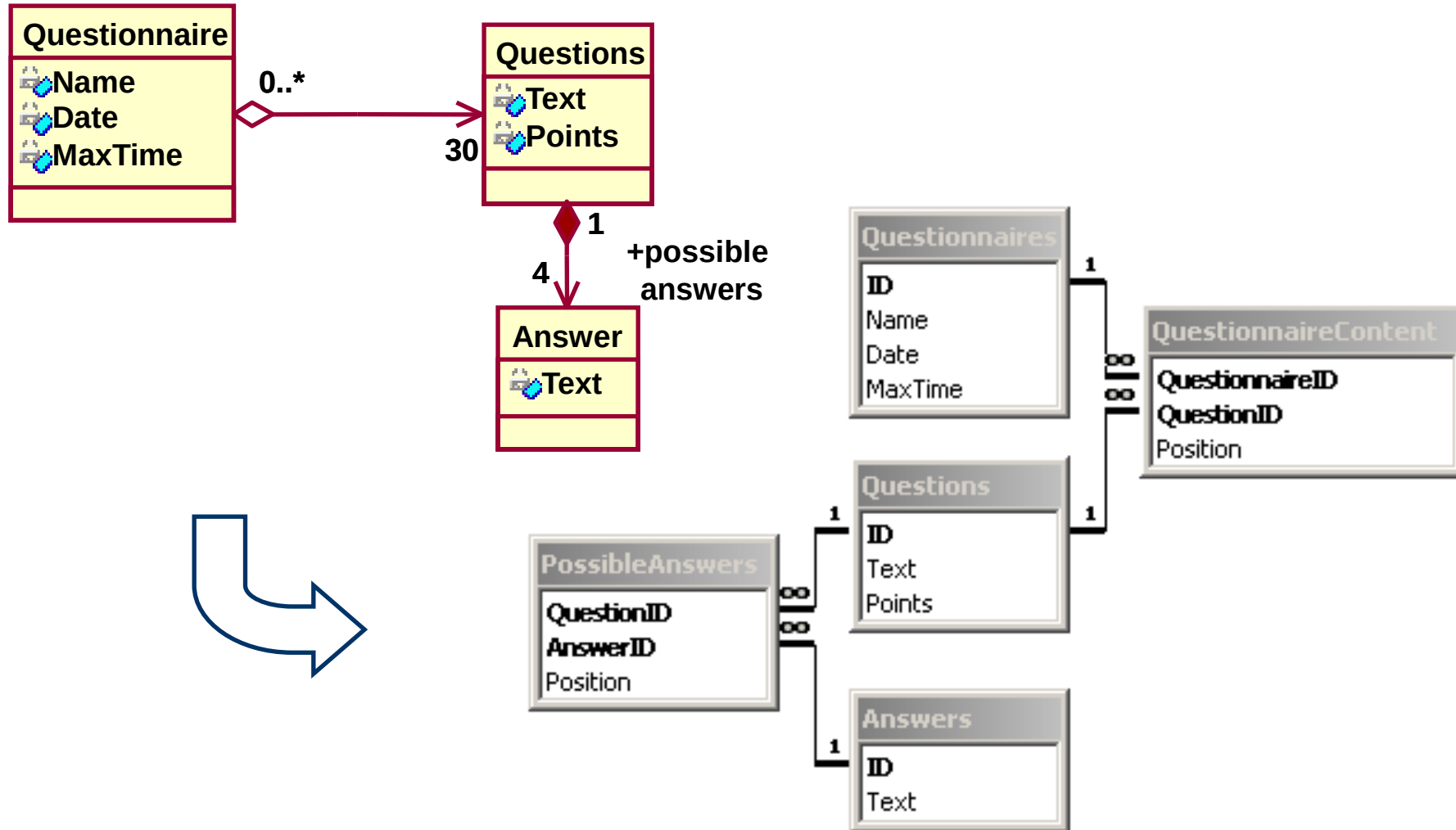
## Care este metoda potrivită?

- Dacă numărul înregistrărilor stocate în tabele este redus (deci performanța nu reprezintă o problemă), atunci poate fi selectată cea mai flexibilă metodă - **Metoda 1**
- Dacă superclasa are un număr restrâns de attribute (comparativ cu subclasele sale) atunci metoda potrivită este **Metoda 3**.
- Dacă subclasele au instanțe puține atunci cea mai bună este utilizarea **Metoda 2**.

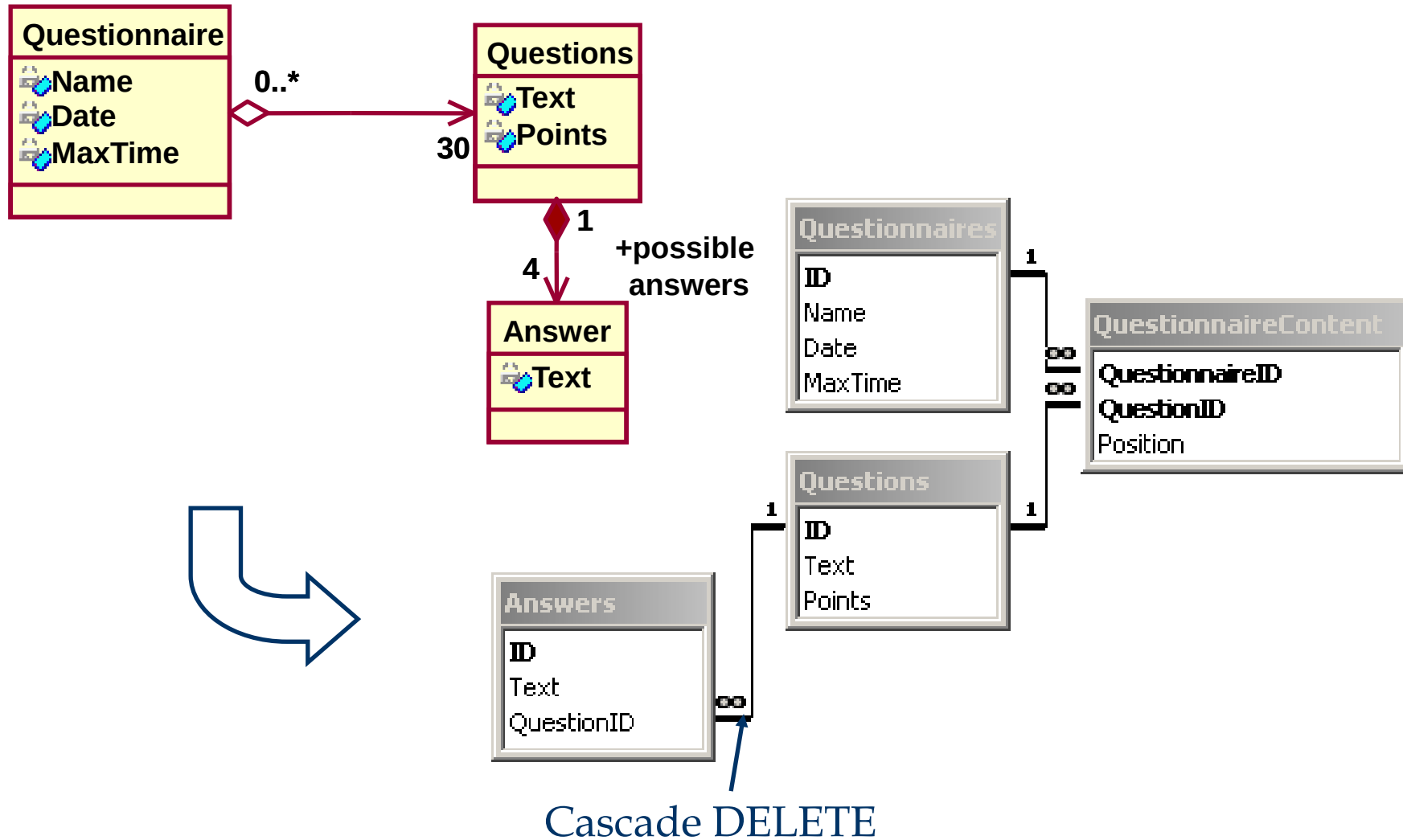
# Transformarea agregării/compunerii

- Agregarea și compunerea sunt modelate în mod asemănător modelării asocierilor
- În cazul relațiilor de compunere de obicei se utilizează o singură tabelă (*cross-tables*) - deoarece compunerea implică mai multe relații 1:1
- Numărul fix de “părți” într-un “întreg” presupune introducerea unui număr egal de chei străine în tabela “întreg”
- În cazul implementării compunerii în tabele separate este necesară setarea “ștergerii în cascadă” (în cazul agregării acest lucru nu este necesar)

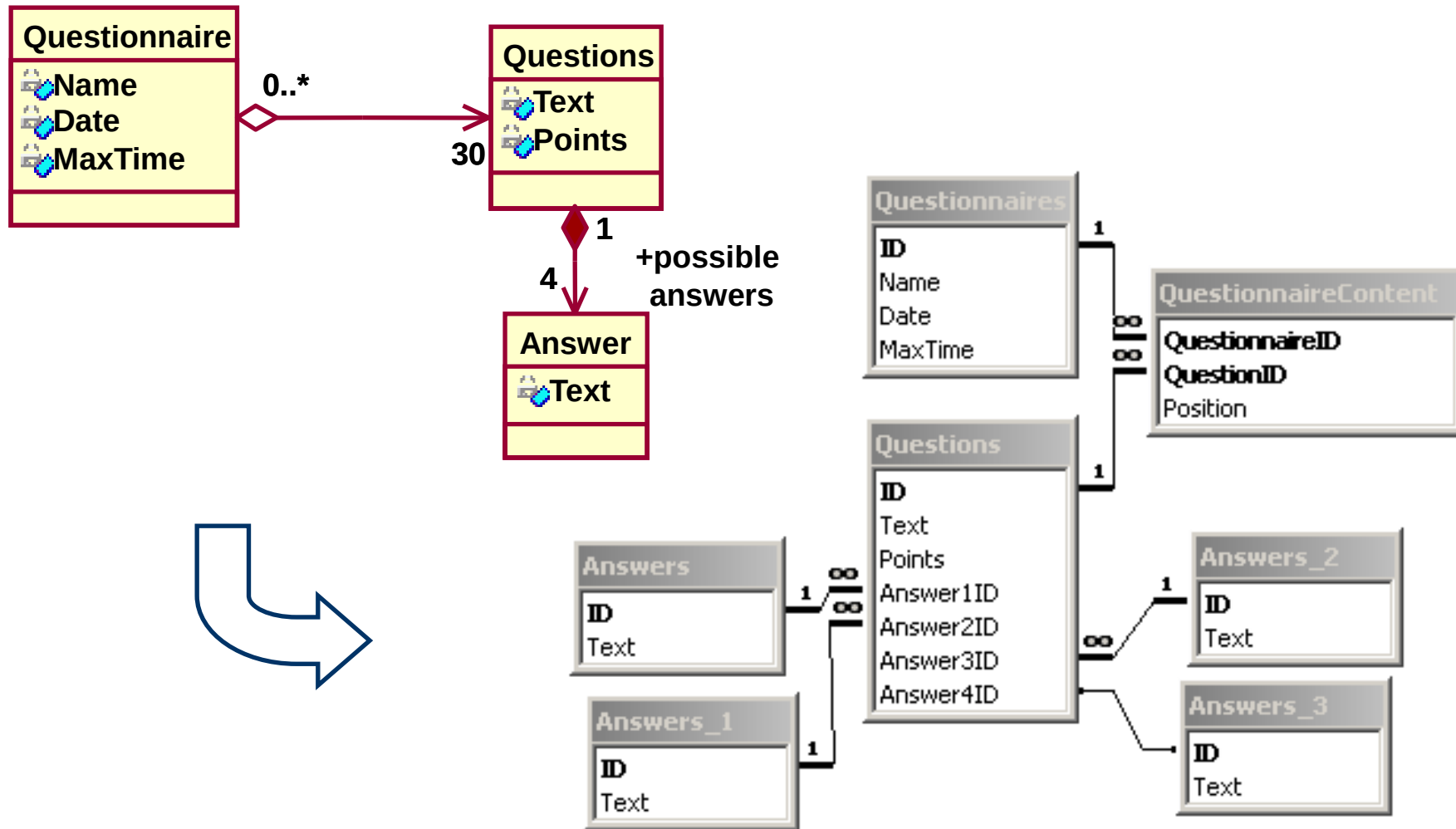
# Transformarea agregării/compunerii



# Transformarea agregării/compunerii

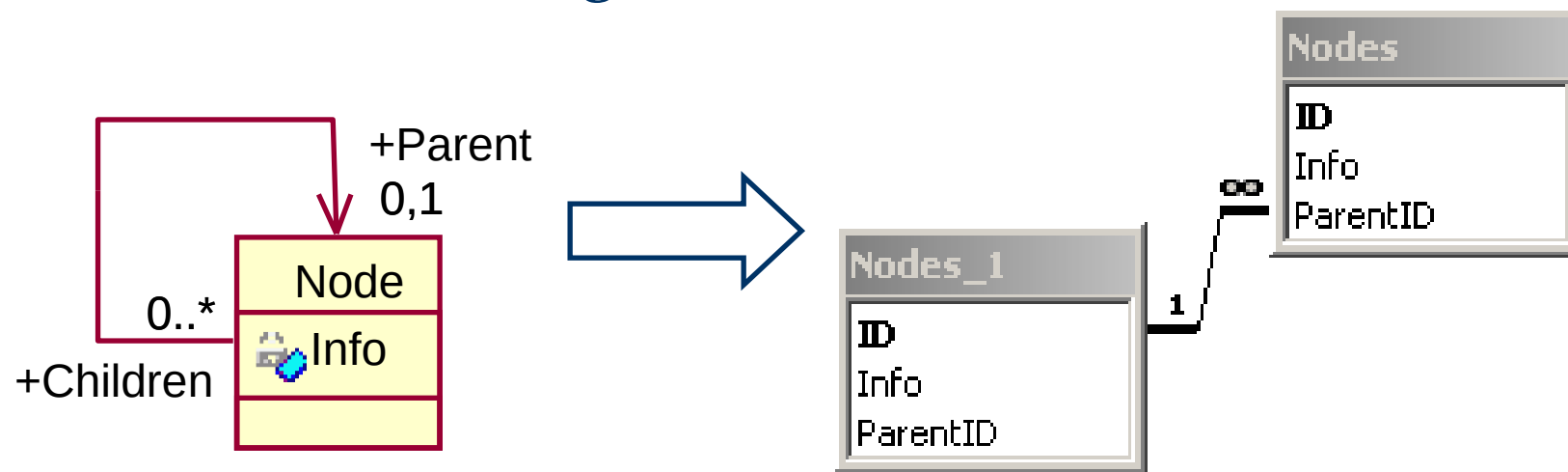


# Transformarea agregării/compunerii



# Transformarea auto-asocierilor

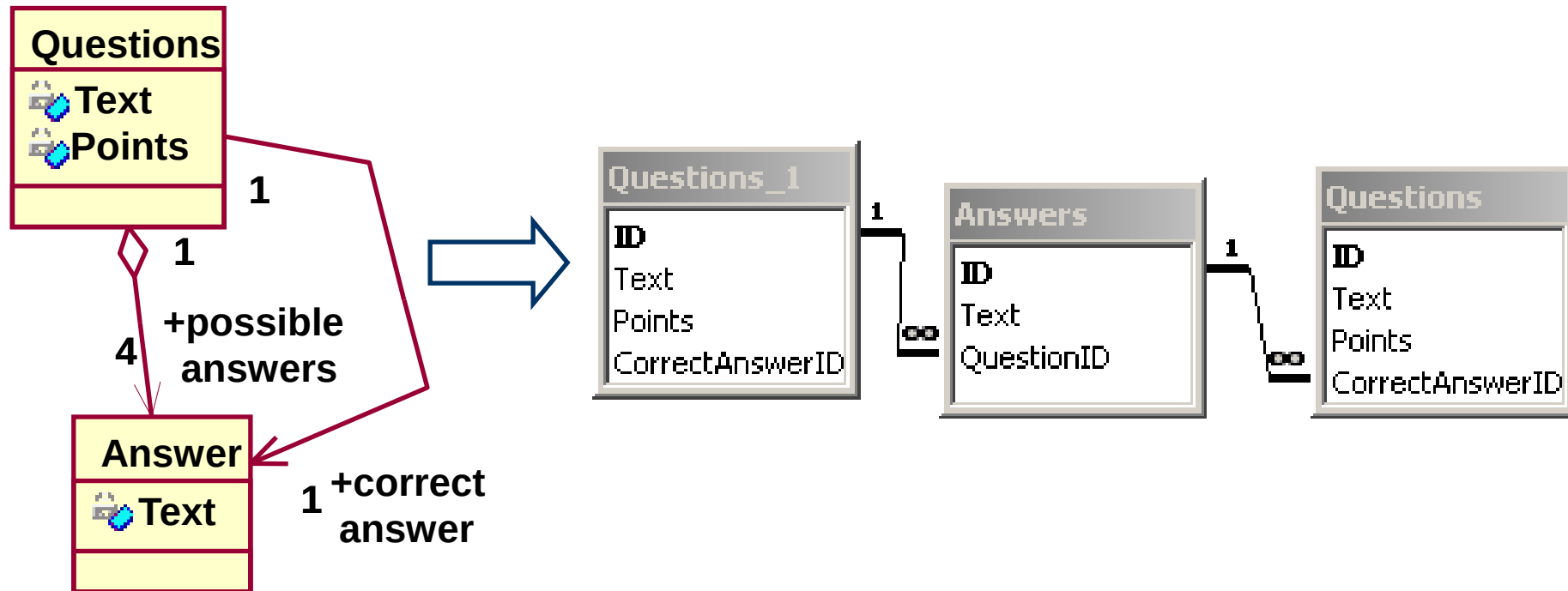
- Se introduce o cheie străină ce pointează spre aceeași (numit *relație recursivă*)
- Dacă este setată proprietatea ștergerii în cascadă există 2 înregistrări care se referă reciproc, ștergerea uneia dintre ele va genera o eroare





# Transformarea auto-asocierilor

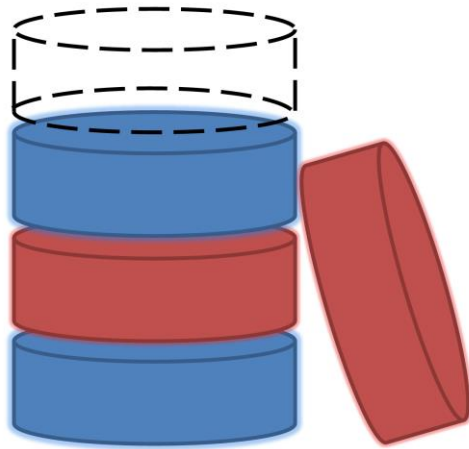
- “Ștergerea în cascadă” generează o problemă similară și în cazul a două tabele ce se referă reciproc



# Generarea automata a bazelor de date

- CASE tool: instrument de modelare vizuală
- Automatizează anumiți pași privind translatarea diagramelor de clase în tabele relaționale.
  - Este necesară și intervenția manuală
- Object-Relational Mapping (ORM)
  - biblioteci/componente ce generează comenzi SQL de creare a tabelelor si manipulare a datelor
    - Hibernate (Java),
    - Entity Framework, NHibernate (C#),
    - Django ORM, SQLAlchemy (Python)

# Algebra Relațională



# Limbaje de interogare relațională

- Limbaj de interogare: Permite manipularea și regăsirea datelor dintr-o bază de date.
- Modelul relațional oferă suport pentru limbaje de interogare simple & puternice:
  - Fundament formal, bazat pe logică.
  - Plajă largă de optimizări.
- Limbaje de interogare **!=** limbaje de programare!
  - nu sunt "Turing complete"
  - nu sunt utilizate pentru calcule complexe
  - oferă o modalitate simplă și eficientă de acces la mulțimi de date voluminoase

# Limbaje de interogare formale

- Două limbaje de interogare formează baza pentru limbajele utilizate în practică (ex. SQL):
  - Algebra Relațională: Mai **operatională**, utilă pentru reprezentarea planurilor de execuție.
  - Relational Calculus: Permite utilizatorilor să descrie **ce**, și nu **cum** să obțină ceea ce doresc. (**Non-operational**, declarativ)

# Algebra relațională

- O interogare se aplică *instanței* unei relații, și rezultatul interogării reprezintă de asemenea o instanță de relație.
  - *Structura* relațiilor ce apar într-o interogare este fixă (dar interogarea se va executa indiferent de instanța relației la un moment dat)
  - Structura *rezultatului* unei interogări este de asemenea fixă și este determinată de definițiile construcțiilor limbajului de interogare.
- Notăție pozițională sau prin nume:
  - Notăția pozițională este mai utilă în definiții formale, însă utilizarea numelor de câmpuri conduce la interogări mai ușor de citit.
  - Ambele variante sunt utilizate în SQL

# Algebra relațională

## ■ Operatori de bază:

- Proiectia ( $\pi$ ) Elimină attributele nedorite ale unei relații
- Selectie ( $\sigma$ ) Selectează o submulțime de tupluri ale unei relații.
- Prod cartezian ( $\times$ ) Permite combinarea a două relații.
- Diferenta ( $-$ ) Tuplurile ce aparțin unei relații dar nu aparțin celeilalte
- Reuniunea ( $\cup$ ) Tuplurile aparținând ambelor relații

## ■ Operatori adiționali:

- Intersecția, join, câtul, redenumirea: nu sunt esențiale dar sunt foarte folosite.

■ Deoarece fiecare operator returnează o relație, **operatorii pot fi compuși** (algebra este “închisă”).

# Proiecția

- $L = (a_1, \dots, a_n)$  este o listă de atribute (sau *o lista de coloane*) ale relației  $R$
- Returnează o relație eliminând toate atributele care nu sunt în  $L$

$$\pi_L(R) = \{ t \mid t_1 \in R \wedge \\ t.a_1 = t_1.a_1 \wedge \\ \dots \wedge \\ t.a_n = t_1.a_n \}$$



# Exemplu proiecție

$\pi_{\text{cid, grade}}(\text{Enrolled})$

$\pi_{\text{cid, grade}}($

| <i>sid</i> | <i>cid</i> | <i>grade</i> |
|------------|------------|--------------|
| 1234       | Alg1       | 9            |
| 1235       | Alg1       | 10           |
| 1234       | DB1        | 10           |
| 1234       | DB2        | 9            |
| 1236       | DB1        | 7            |
| 1237       | DB2        | 9            |
| 1237       | DB1        | 5            |
| 1237       | Alg1       | 10           |

) =

| <i>cid</i> | <i>grade</i> |
|------------|--------------|
| Alg1       | 9            |
| Alg1       | 10           |
| DB1        | 10           |
| DB2        | 9            |
| DB1        | 7            |
| DB1        | 5            |

# Proiecția

Este  $\pi_{\text{cid, grade}}(\text{Enrolled})$  echivalentă cu

```
SELECT cid, grade FROM Enrolled ?
```

**Nu!** Algebra relațională operează cu mulțimi => nu există duplicate.

```
SELECT DISTINCT cid, grade  
FROM Enrolled
```

# Selecția

- Selectează tuplurile unei relații **R** care verifică o condiție *c* (numită și *predicat de selecție*).

$$\sigma_c(R) = \{ t \mid t \in R \wedge c \}$$

$$\sigma_{\text{grade} > 8}(\text{Enrolled}) = \{ t \mid t \in \text{Enrolled} \wedge \text{grade} > 8 \}$$

| <i>sid</i> | <i>cid</i> | <i>grade</i> |
|------------|------------|--------------|
| 1234       | Alg1       | 9            |
| 1235       | Alg1       | 10           |
| 1234       | DB2        | 9            |
| 1236       | DB1        | 7            |
| 1237       | DB1        | 5            |
| 1237       | Alg1       | 6            |

$$\sigma_{\text{grade} > 8}(\text{Enrolled}) =$$

| <i>sid</i> | <i>cid</i> | <i>grade</i> |
|------------|------------|--------------|
| 1234       | Alg1       | 9            |
| 1235       | Alg1       | 10           |
| 1234       | DB2        | 9            |

# Selecția

$\sigma_{\text{grade} > 8}(\text{Enrolled})$

```
SELECT DISTINCT *  
FROM Enrolled  
WHERE grade > 8
```

# Condiția selecției

- **Term Op Term** este o condiție, unde
  - **Term** este un nume de atribut, sau
  - **Term** este o constantă
  - **Op** este un operator logic (ex.  $<$ ,  $>$ ,  $=$ ,  $\neq$  etc.)
- $(C1 \wedge C2)$ ,  $(C1 \vee C2)$ ,  $(\neg C1)$  sunt condiții formate din operatorii  $\wedge$  (și logic),  $\vee$  (sau logic) sau  $\neg$  (negație), iar  $C1$  și  $C2$  sunt la rândul lor condiții

# Compunere

Rezultatul unei interogări este o relație

$$\pi_{\text{cid, grade}}(\sigma_{\text{grade} > 8}(\text{Enrolled}))$$

$$\pi_{\text{cid, grade}}(\sigma_{\text{grade} > 8}(\text{Enrolled})) =$$

| <i>sid</i> | <i>cid</i> | <i>grade</i> |
|------------|------------|--------------|
| 1234       | Alg1       | 9            |
| 1235       | Alg1       | 10           |
| 1234       | DB1        | 10           |
| 1234       | DB2        | 9            |
| 1236       | DB1        | 7            |
| 1237       | DB2        | 9            |
| 1237       | DB1        | 5            |
| 1237       | Alg1       | 10           |

| <i>cid</i> | <i>grade</i> |
|------------|--------------|
| Alg1       | 9            |
| Alg1       | 10           |
| DB1        | 10           |
| DB2        | 9            |

Proiecție

$\pi_{attr1, attr2}(Relație)$

**SELECT DISTINCT attr<sub>1</sub>, attr<sub>2</sub>**

**FROM Relație**

**WHERE c**

Selecție  $\sigma_c(Relație)$

$\pi_{\text{cid, grade}}(\sigma_{\text{grade} > 8}(\text{Enrolled}))$

```
SELECT DISTINCT cid, grade  
FROM Enrolled  
WHERE grade > 8
```

$\sigma_{\text{grade} > 8}(\pi_{\text{cid, grade}}(\text{Enrolled}))$

Putem schimba întotdeauna ordinea operatorilor  $\sigma$  și  $\pi$ ?



# Reuniune, intersecție, diferență

- $R_1 \cup R_2 = \{ t \mid t \in R_1 \vee t \in R_2 \}$
- $R_1 \cap R_2 = \{ t \mid t \in R_1 \wedge t \in R_2 \}$
- $R_1 - R_2 = \{ t \mid t \in R_1 \wedge t \notin R_2 \}$

Relațiile  $R_1$  și  $R_2$  trebuie să fie *compatibile*:

- același număr de atribute (aceeași *aritate*)
- atributele aflate pe aceeași poziție au domenii *compatibile* și *același nume*

# Reuniune, intersecție, diferență în SQL

$$R_1 \cup R_2$$

```
SELECT DISTINCT *  
FROM R1
```

**UNION**

```
SELECT DISTINCT *  
FROM R2
```

$$R_1 \cap R_2$$

```
SELECT DISTINCT *  
FROM R1
```

**INTERSECT**

```
SELECT DISTINCT *  
FROM R2
```

$$R_1 - R_2$$

```
SELECT DISTINCT *  
FROM R1
```

**EXCEPT**

```
SELECT DISTINCT *  
FROM R2
```

# Toți sunt operatorii esențiali?

$$R_1 \cap R_2 = ( (R_1 \cup R_2) - (R_1 - R_2) ) - (R_2 - R_1)$$

Identifică tuplurile  
incluse în  $R_1$  sau  $R_2$

Elimină tuplurile  
ce aparțin doar  $R_1$

Elimină tuplurile  
ce aparțin doar  $R_2$

# Produs cartezian

- Combinarea a doua relații

$$R_1(a_1, \dots, a_n) \text{ și } R_2(b_1, \dots, b_m)$$

$$R_1 \times R_2 = \{ t \mid t_1 \in R_1 \wedge t_2 \in R_2$$

$$\wedge t.a_1 = t_1.a_1 \dots \wedge t.a_n = t_1.a_n$$

$$\wedge t.b_1 = t_2.b_1 \dots \wedge t.b_m = t_2.b_m \}$$

```
SELECT DISTINCT *  
FROM R1, R2
```

# $\theta$ -Join

- Combinarea a doua relații  $R_1$  și  $R_2$  cu respectarea condiției  $c$

$$R_1 \otimes_c R_2 = \sigma_c (R_1 \times R_2)$$

Students  $\otimes_{\text{Students.sid=Enrolled.sid}}$  Enrolled

```
SELECT DISTINCT *  
FROM Students, Enrolled  
WHERE Students.sid =  
Enrolled.sid
```

```
SELECT DISTINCT *  
FROM Students  
INNER JOIN Enrolled ON  
Students.sid=Enrolled.sid
```

# Equi-Join

- Combină două relații pe baza unei condiții compuse doar din egalități ale unor attribute aflate în prima și a doua relație și proiectează doar unul dintre attributele redundante (deoarece sunt egale)

$$R_1 \otimes_{E(c)} R_2$$

| <i>Courses</i> |              | $\otimes_{E(\text{Courses.cid} = \text{Enrolled.cid})}$ | <i>Enrolled</i> |            |              | $=$  |              |            |            |              |
|----------------|--------------|---------------------------------------------------------|-----------------|------------|--------------|------|--------------|------------|------------|--------------|
| <i>cid</i>     | <i>cname</i> |                                                         | <i>sid</i>      | <i>cid</i> | <i>grade</i> |      | <i>cname</i> | <i>sid</i> | <i>cid</i> | <i>grade</i> |
| Alg1           | Algorithms1  |                                                         | 1234            | Alg1       | 9            |      | Algorithms1  | 1234       | Alg1       | 9            |
| DB1            | Databases1   |                                                         | 1235            | Alg1       | 10           |      | Algorithms1  | 1235       | Alg1       | 10           |
| DB2            | Databases2   |                                                         | 1234            | DB1        | 10           |      | Databases1   | 1234       | DB1        | 10           |
|                |              |                                                         | 1234            | DB2        | 9            |      | Databases2   | 1234       | DB2        | 9            |
|                |              | 1236                                                    | DB1             | 7          | Databases1   | 1236 | DB1          | 7          |            |              |

# Join Natural

- Combină două relații pe baza egalității atributelor ce au *același nume* și proiectează doar unul dintre attributele redundante

$$R_1 \otimes R_2$$

| <i>Courses</i> |              | $\otimes$ | <i>Enrolled</i> |            |              | $=$ |              |            |            |              |
|----------------|--------------|-----------|-----------------|------------|--------------|-----|--------------|------------|------------|--------------|
| <i>cid</i>     | <i>cname</i> |           | <i>sid</i>      | <i>cid</i> | <i>grade</i> |     | <i>cname</i> | <i>sid</i> | <i>cid</i> | <i>grade</i> |
| Alg1           | Algorithms1  |           | 1234            | Alg1       | 9            |     | Algorithms1  | 1234       | Alg1       | 9            |
| DB1            | Databases1   |           | 1235            | Alg1       | 10           |     | Algorithms1  | 1235       | Alg1       | 10           |
| DB2            | Databases2   |           | 1234            | DB1        | 10           |     | Databases1   | 1234       | DB1        | 10           |
|                |              |           | 1234            | DB2        | 9            |     | Databases2   | 1234       | DB2        | 9            |
|                |              |           | 1236            | DB1        | 7            |     | Databases1   | 1236       | DB1        | 7            |

# Câtul

- Nu este un operator de bază, însă este util în anumite situații (simplifică mult interogarea)
- Fie  $R_1$  cu 2 attribute,  $x$  și  $y$  și  $R_2$  cu un atribut  $y$ :  
$$R_1 / R_2 = \{ \langle x \rangle \mid \exists \langle x, y \rangle \in R_1 \quad \forall \langle y \rangle \in R_2 \}$$
  
adică,  $R_1 / R_2$  conține toate tuplurile  $x$  a.î. pentru fiecare dintre tuplurile  $y$  din  $R_2$ , există câte un tuplu  $xy$  în  $R_1$ .  
*Sau:* Dacă mulțimea valorilor  $y$  asociate cu o valoare  $x$  din  $R_1$  conține toate valorile  $y$  din  $R_2$ , atunci  $x$  va fi returnat în rezultat  $R_1 / R_2$ .
- Generalizând,  $x$  și  $y$  pot reprezenta orice multime de attribute;  $y$  este mulțimea atributelor din  $R_2$ , și  $x \cup y$  reprezintă attributele lui  $R_1$ .



# Modelarea operatorului *cât* folosind operatori de bază

- Cât-ul nu e un operator esențial, ci doar o "*scurtătură*".
  - (este și cazul operatorilor *join*, dar aceștia sunt folosiți mult mai des în interogări și au implementări speciale în diferite sisteme)
- *Ideea*: Pentru  $R_1 / R_2$ , vom determina valorile  $x$  care nu sunt 'conectate' cu anumite valori  $y$  din  $R_2$ .
  - valoarea  $x$  este *deconectată* dacă atașând la ea o valoare  $y$  din  $R_2$ , obținem un tuplu  $xy$  ce nu se regăsește în  $R_1$ .

Valorile  $x$  deconectate:  $\pi_x ( (\pi_x(R_1) \times R_2) - R_1 )$

$$R_1 / R_2 = \pi_x(R_1) - (Valorile\ x\ deconectate)$$

# Redenumirea

- Dacă atributele și relațiile au aceleași nume (de exemplu la *join*-ul unei relații cu ea însăși) este necesar să putem redenumi una din ele

$$\rho(R' (N_1 \rightarrow N'_1, N_2 \rightarrow N'_2), R)$$

notație alternativă:  $\rho_{R' (N'_1, N'_2)}(R)$ ,

- Noua relație  $R'$  are aceeași instanță ca  $R$ , iar structura sa conține atributul  $N'_i$  în locul atributului  $N_i$

# Redenumirea

$\rho(\text{Courses2 } (\text{cid} \rightarrow \text{code},$   
 $\text{cname} \rightarrow \text{description}),$   
 $\text{Courses})$

*Courses*

| <i>cid</i> | <i>cname</i> | <i>credits</i> |
|------------|--------------|----------------|
| Alg1       | Algorithms1  | 7              |
| DB1        | Databases1   | 6              |
| DB2        | Databases2   | 6              |



*Courses2*

| <i>code</i> | <i>description</i> | <i>credits</i> |
|-------------|--------------------|----------------|
| Alg1        | Algorithms1        | 7              |
| DB1         | Databases1         | 6              |
| DB2         | Databases2         | 6              |

```
SELECT cid as code,  
       cname as description,  
       credits  
FROM Courses Courses2
```

# Operatorul de atribuire

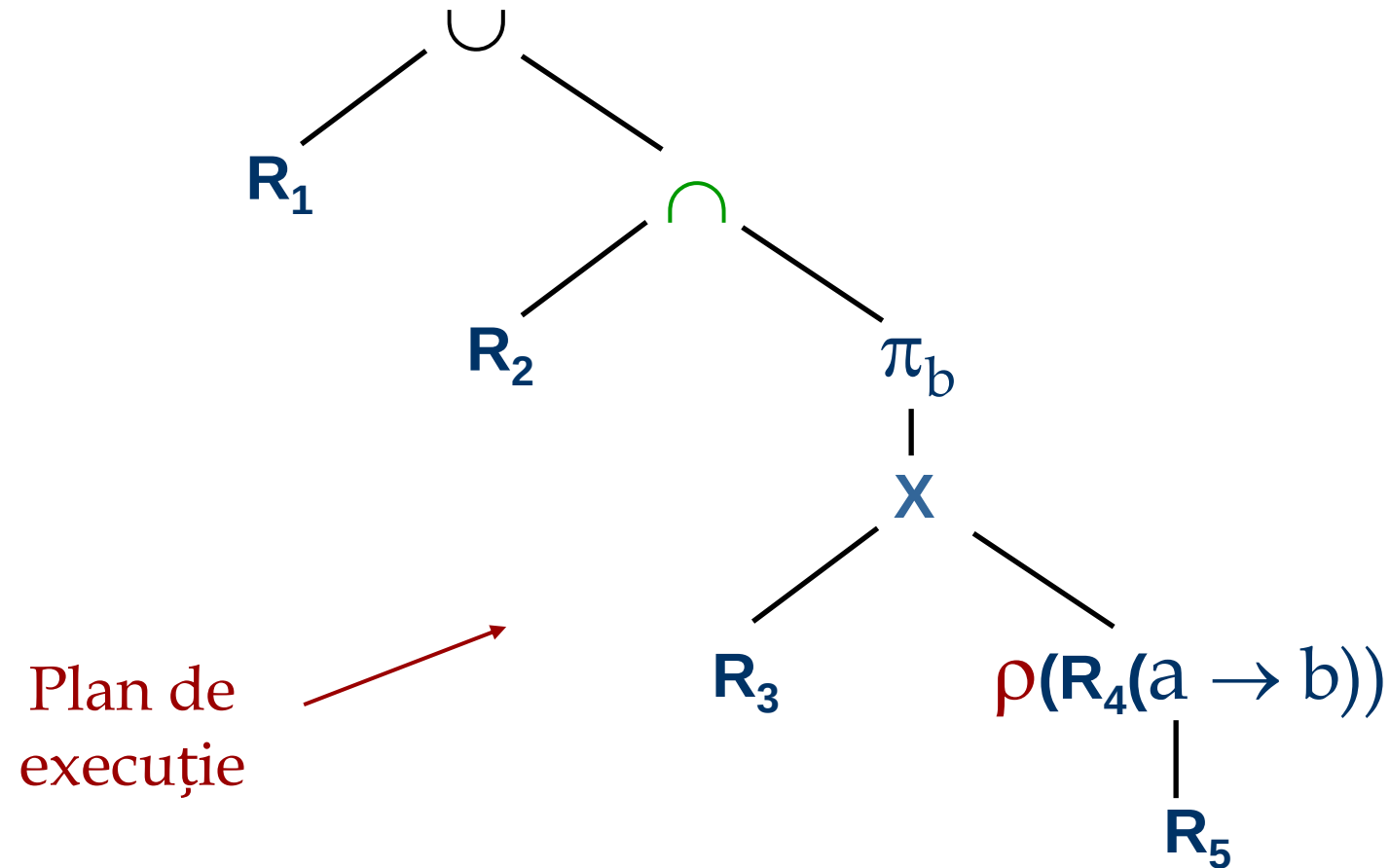
- Operatorul de atribuire (  $\leftarrow$  ) oferă un mod simplu de tratare a interogărilor complexe.
  - Atribuirile se fac întotdeauna într-o variabilă temporară

$$\text{Temp} \leftarrow \pi_x(R_1 \times R_2)$$

- Rezultatul expresiei din dreapta  $\leftarrow$  este atribuit variabilei din stânga operatorului  $\leftarrow$ .
- Variabilele pot fi utilizate apoi în alte expresii
  - $\text{result} \leftarrow \text{Temp} - R_3$

# Expresii complexe

$$R_1 \cup (R_2 \cap \pi_b (R_3 \bowtie \rho(R_4(a \rightarrow b), R_5)))$$



*Determinați numele tuturor studenților cu note la cursul 'BD1'*

Solutie 1:  $\pi_{\text{name}} ( (\sigma_{\text{cid}='BD1'}(\text{Enrolled})) \otimes \text{Students} )$

Solutie 2:  $\rho (\text{Temp}_1, \sigma_{\text{cid}='BD1'}(\text{Enrolled}))$   
 $\rho (\text{Temp}_2, \text{Temp}_1 \otimes \text{Students})$   
 $\pi_{\text{name}} ( \text{Temp}_2 )$

Solutie 3:  $\pi_{\text{name}} ( \sigma_{\text{cid}='BD1'}(\text{Enrolled} \otimes \text{Students} ) )$

*Determinați numele tuturor studenților cu note la cursuri cu 5 credite*

- Informația cu privire la credite se găsește în relația *Courses*, și prin urmare se adaugă un join natural:

$$\pi_{\text{name}} ( (\sigma_{\text{credits}=5}(\text{Courses})) \otimes \text{Enrolled} \otimes \text{Students} )$$

- O soluție mai eficientă:

$$\pi_{\text{name}} ( \pi_{\text{sid}} ( \pi_{\text{cid}} ( \sigma_{\text{credits}=5}(\text{Courses}) ) \otimes \text{Enrolled} ) \otimes \text{Students} )$$

*Modulul de optimizare a interogărilor e capabil să transforme prima soluție în a doua!*

*Determinați numele tuturor studenților cu note la cursuri cu 4 sau 5 credite*

- Se identifică toate cursurile cu 4 sau 5 credite, apoi se determină studenții cu note la unul dintre aceste cursuri:

$\rho (TempCourses, (\sigma_{credits=4 \vee credits=5}(Courses)))$

$\pi_{name} (TempCourses \otimes Enrolled \otimes Students )$

- *TempCourses* se poate defini și utilizând reuniunea!
- Ce se întâmplă dacă înlocuim  $\vee$  cu  $\wedge$  în interogare?



*Determinați numele tuturor studenților cu note la cursuri cu 4 și 5 credite*

- Abordarea anterioară nu funcționează! Trebuie identificați în paralel studenții cu note la cursuri de 4 credite și studenții cu note la cursuri de 5 credite, apoi se intersectează cele două mulțimi (*sid este cheie pentru Students*):

$\rho (Temp4, \pi_{sid}(\sigma_{credits=4} (Courses) \otimes Enrolled))$

$\rho (Temp5, \pi_{sid}(\sigma_{credits=5} (Courses) \otimes Enrolled))$

$\pi_{name} ((Temp4 \cap Temp5) \otimes Students)$

*Determinați numele tuturor studenților cu note la toate cursurile*

- Se utilizează *câtul*; trebuie pregătite structurile relațiilor înainte de a folosi operatorul *cât*:

$$\rho \text{ (TempSIDs, } \pi_{\text{sid, cid}}(\text{Enrolled}) / \pi_{\text{cid}}(\text{Courses}))$$

$$\pi_{\text{name}}(\text{TempSIDs} \otimes \text{Students})$$

# Extensii ale operatorilor algebrici relaționali

- Generalizarea proiecției
- Funcții de agregare
- Outer Join
- Modificarea bazei de date

# Generalizarea proiecției

- Operatorul *proiecție* este extins prin permiterea utilizării funcțiilor aritmetice în lista de definire a proiecției.

$$\pi_{F_1, F_2, \dots, F_n}(R)$$

- $R$  poate fi orice expresie din algebra relatională
- Fiecare dintre  $F_1, F_2, \dots, F_n$  sunt expresii aritmetice ce implică attribute din  $R$  și constante.

# Funcții de agregare

- **Funcția de agregare** returneaza ca rezultat o valoare pe baza unei colecții de valori primite ca input

**avg:** valoarea medie

**min:** valoarea minimă

**max:** valoarea maximă

**sum:** suma

**count:** numărul înregistrărilor

- **Operator de agregare** în algebra relațională

$$\mathcal{G}_{G_1, G_2, \dots, G_n}^{F_1(A_1), F_2(A_2), \dots, F_n(A_n)}(R)$$

- $R$  poate fi orice expresie din algebra relațională
  - $G_1, G_2, \dots, G_n$  e o listă de attribute pe baza cărora se grupează datele (poate fi goală)
  - Fiecare  $F_i$  este o funcție de agregare
  - Fiecare  $A_i$  este un nume de atribut

# Agregarea – Example

Relație  $R$ :

| $A$      | $B$      | $C$ |
|----------|----------|-----|
| $\alpha$ | $\alpha$ | 7   |
| $\alpha$ | $\beta$  | 7   |
| $\beta$  | $\beta$  | 3   |
| $\beta$  | $\beta$  | 10  |

$\mathcal{G}_{\text{sum}(C)}(R)$

| <b>sum(C )</b> |
|----------------|
| 27             |

- Rezultatul agregarii nu are un nume
  - se pot folosi operatorii de redenumire
  - se poate permite redenumirea ca parte a unui operator de agregare

# Outer Join

■ Extensii ale operatorului join natural care împiedică pierderea informației:

- Left Outer Join 
- Right Outer Join 
- Full Outer Join 

■ Realizează joncțiunea și apoi adaugă la rezultat tuplurile dintr-una din relatii (din stânga, dreapta sau ambele părți ale operatorului) care nu sunt conectate cu tupluri din celaltă relație.

■ Utilizează valoarea *null*:

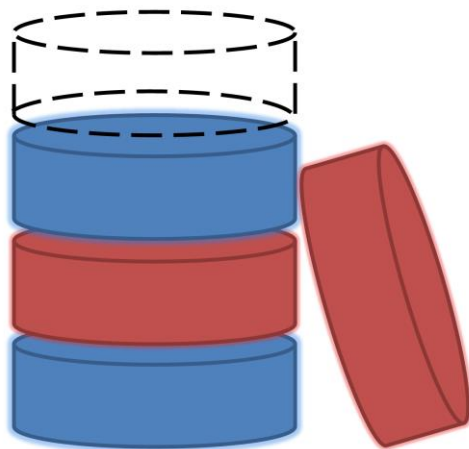
- *null* semnifică faptul că valoarea e necunoscută sau nu există
- Toate comparațiile ce implică *null* sunt (*simplu spus*) **false** prin definiție.

# Modificarea bazei de date

- Conținutul bazei de date poate fi modificat folosind următorii operatori:
  - Ștergere  $R \leftarrow R - E$
  - Inserare  $R \leftarrow R \cup E$
  - Modificare  $R \leftarrow \pi_{F_1, F_2, \dots, F_n}(R)$
- Toți acești operatori sunt exprimați prin utilizarea operatorului de atribuire.

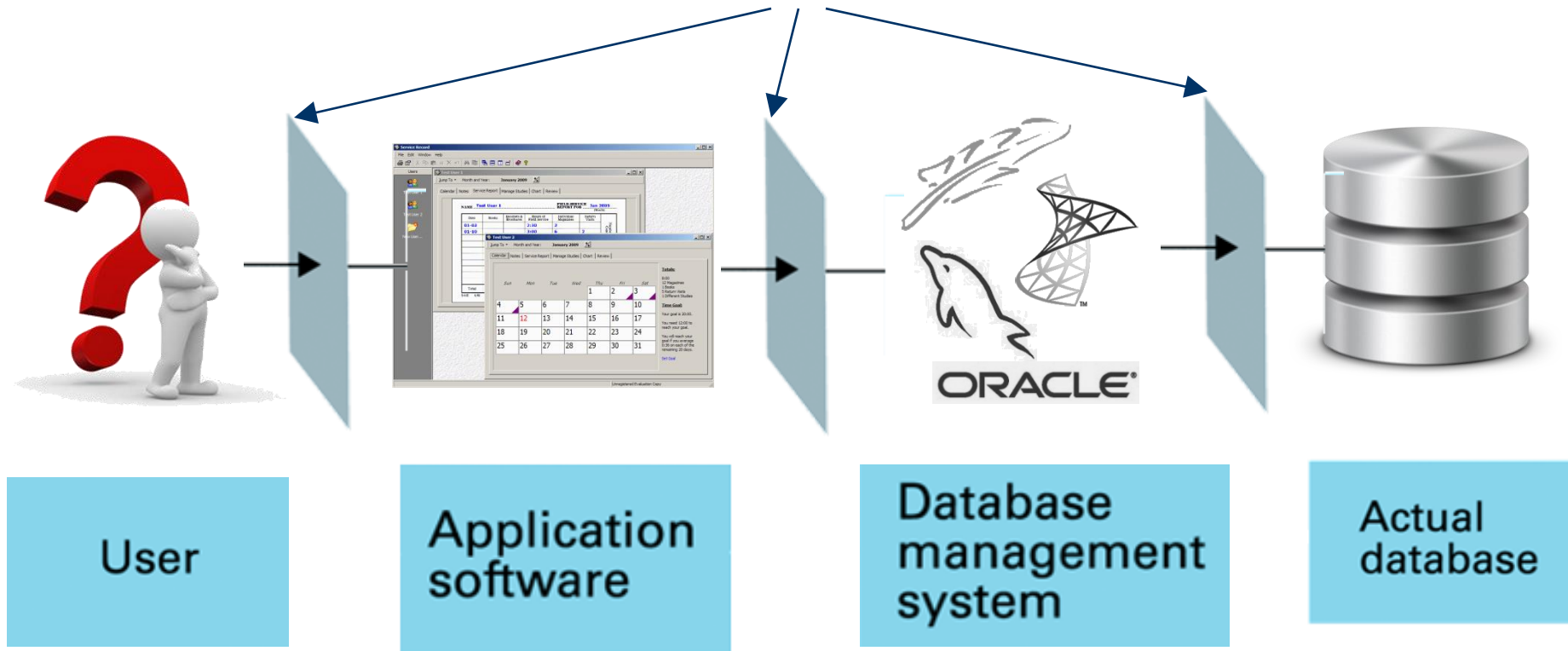


# Structura fizică a bazelor de date



# Nivelele de abstractizare

Nivele diferite  
de abstractizare



**STUDENT**

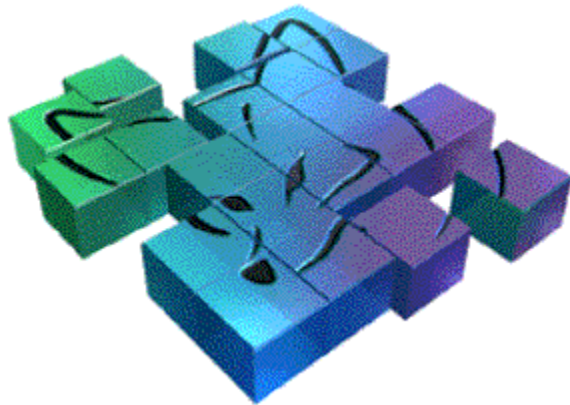
Name

Date of birth

Sex

CNP

Group



# Structura fizică

Faculty.dbc

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 42 | 53 | 54 | 42 | 20 | 2e | 30 | 37 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 03 | 50 | 81 | 01 | f0 | 06 | 4f | 52 |
| 4c | 45 | 2d | 49 | 44 | 01 | 14 | 3c |
| 54 | 2d | 4e | 41 | 4d | 45 | 01 | 13 |
| 45 | 54 | 2d | 4e | 55 | 4d | 42 | 45 |

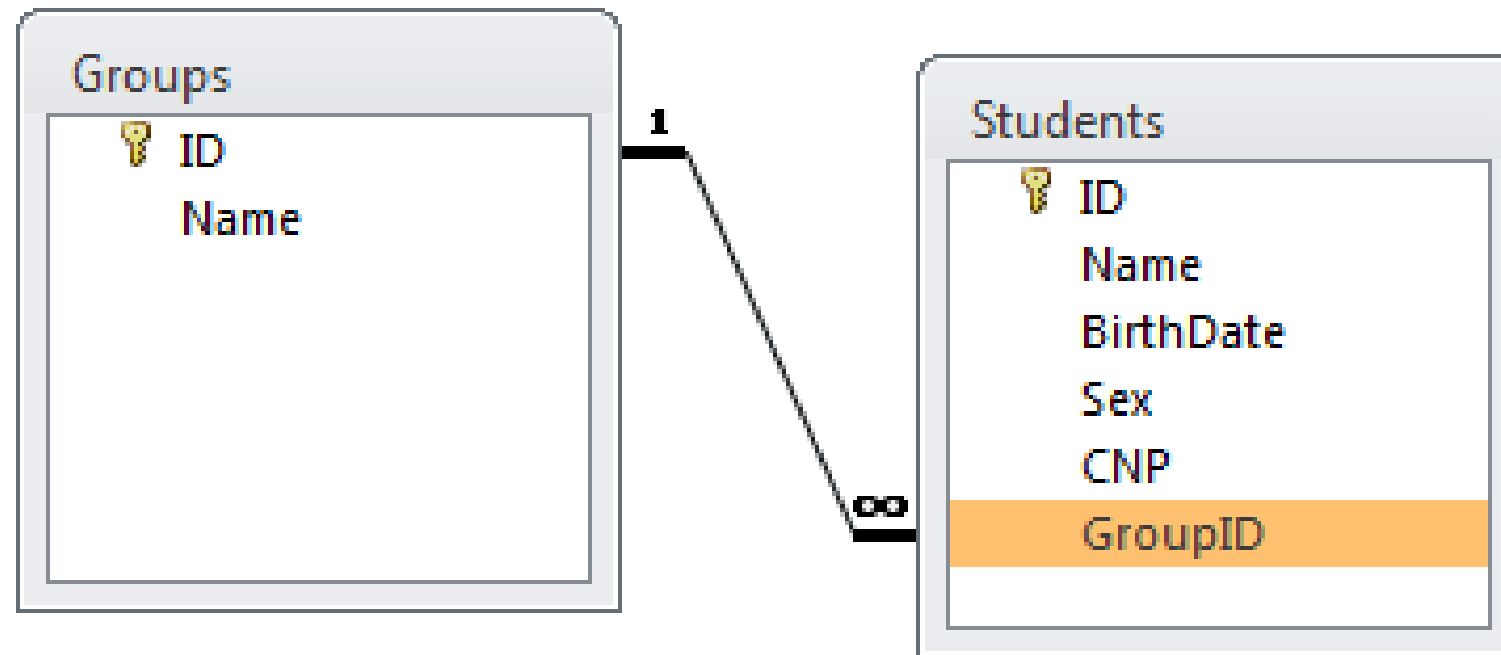
Students.dbf

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 20 | 20 | 20 | 31 | 56 | 31 | 2e | 30 |
| 38 | 31 | 39 | 32 | 44 | 65 | 66 | 61 |
| 61 | 67 | 65 | 20 | 53 | 65 | 74 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 00 | ff | 01 | 00 | 7c | 80 | 00 |
| 45 | 41 | 44 | 45 | 52 | 34 | 0f | 53 |
| 4e | 55 | 4d | 42 | 45 | 52 | 14 | 34 |
| 34 | 21 | 0a | 20 | 20 | 20 | 20 | 20 |
| 42 | 53 | 54 | 42 | 20 | 2e | 30 | 37 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 03 | 50 | 81 | 01 | f0 | 06 | 4f | 52 |
| 4c | 45 | 2d | 49 | 44 | 01 | 14 | 3c |
| 54 | 2d | 4e | 41 | 4d | 45 | 01 | 13 |
| 45 | 54 | 2d | 4e | 55 | 4d | 42 | 45 |

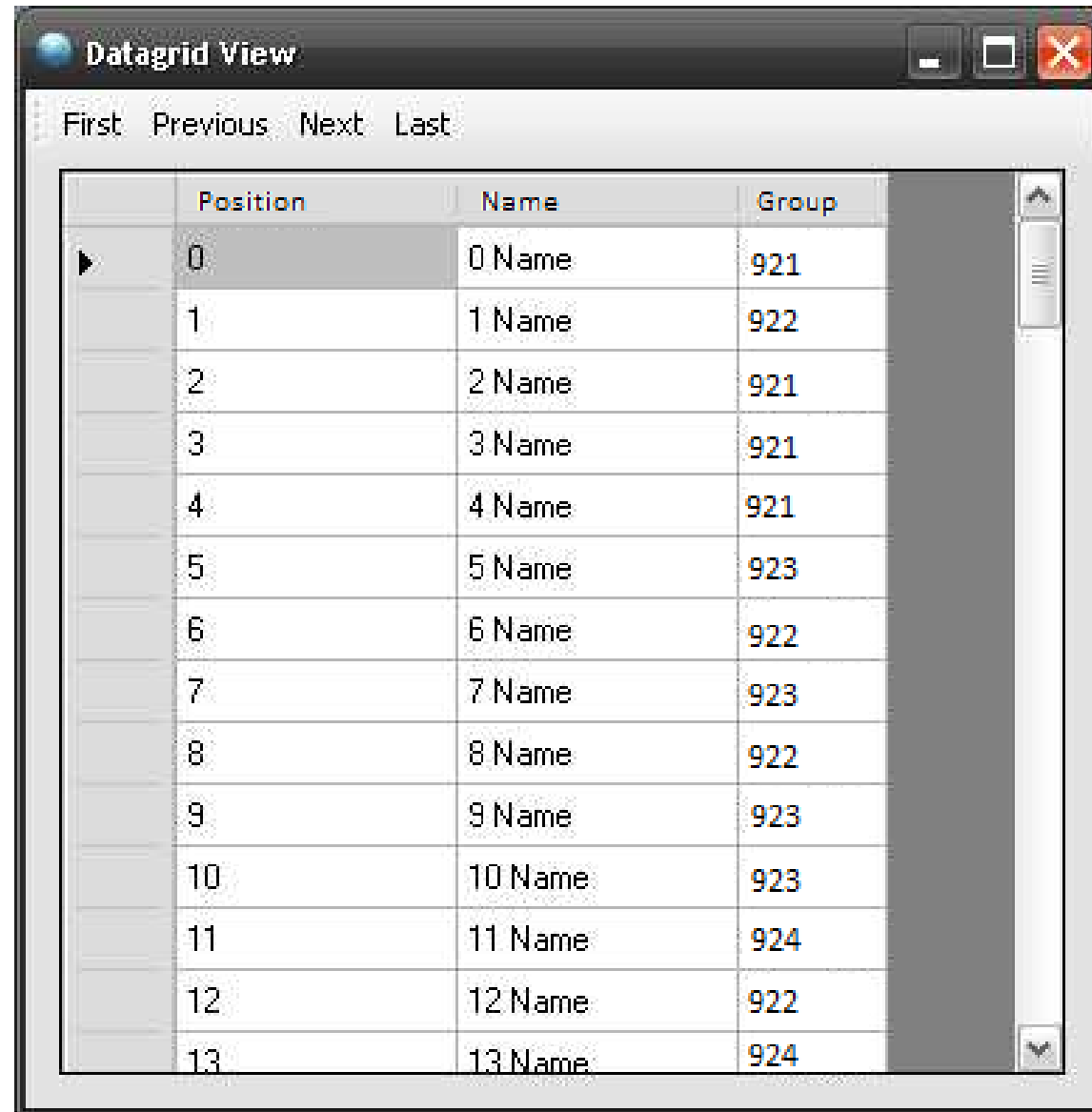
Groups.dbf

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 20 | 20 | 20 | 31 | 56 | 31 | 2e | 30 |
| 38 | 31 | 39 | 32 | 44 | 65 | 66 | 61 |
| 61 | 67 | 65 | 20 | 53 | 65 | 74 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 00 | ff | 01 | 00 | 7c | 80 | 00 |
| 45 | 41 | 44 | 45 | 52 | 34 | 0f | 53 |
| 4e | 55 | 4d | 42 | 45 | 52 | 14 | 34 |
| 34 | 21 | 0a | 20 | 20 | 20 | 20 | 20 |
| 42 | 53 | 54 | 42 | 20 | 2e | 30 | 37 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 03 | 50 | 81 | 01 | f0 | 06 | 4f | 52 |
| 4c | 45 | 2d | 49 | 44 | 01 | 14 | 3c |
| 54 | 2d | 4e | 41 | 4d | 45 | 01 | 13 |
| 45 | 54 | 2d | 4e | 55 | 4d | 42 | 45 |

# Structura conceptuală



# Vizualizare pentru utilizator



Datagrid View

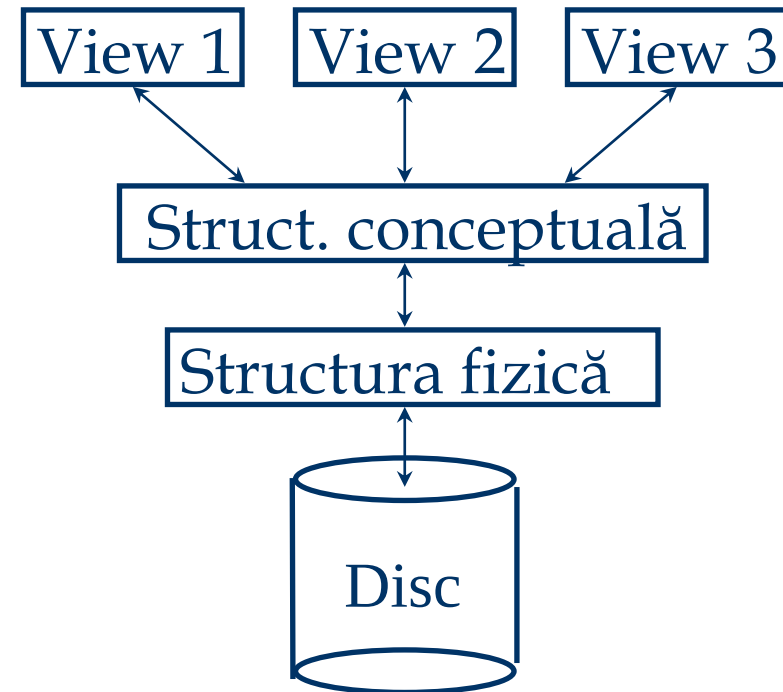
First Previous Next Last

|   | Position | Name    | Group |
|---|----------|---------|-------|
| ▶ | 0        | 0 Name  | 921   |
|   | 1        | 1 Name  | 922   |
|   | 2        | 2 Name  | 921   |
|   | 3        | 3 Name  | 921   |
|   | 4        | 4 Name  | 921   |
|   | 5        | 5 Name  | 923   |
|   | 6        | 6 Name  | 922   |
|   | 7        | 7 Name  | 923   |
|   | 8        | 8 Name  | 922   |
|   | 9        | 9 Name  | 923   |
|   | 10       | 10 Name | 923   |
|   | 11       | 11 Name | 924   |
|   | 12       | 12 Name | 922   |
|   | 13       | 13 Name | 924   |

# Nivelele de abstractizare

■ Mai multe structuri externe (views), câte o singură structură conceptuală (logică) și o structură fizică (internă).

- *Views* – cum văd utilizatorii datele.
- *Conceptual* - modelul logic compus din relații, attribute, etc
- *Fizic* - fișierele de date și indecși

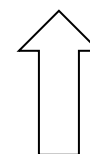
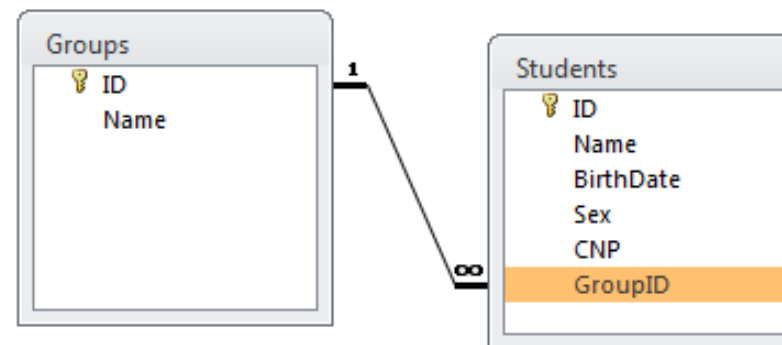
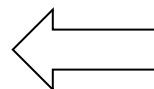


# Independența fizică a datelor

Datagrid View

First Previous Next Last

|   | Position | Name    | Group |
|---|----------|---------|-------|
| ▶ | 0        | 0 Name  | 921   |
|   | 1        | 1 Name  | 922   |
|   | 2        | 2 Name  | 921   |
|   | 3        | 3 Name  | 921   |
|   | 4        | 4 Name  | 921   |
|   | 5        | 5 Name  | 923   |
|   | 6        | 6 Name  | 922   |
|   | 7        | 7 Name  | 923   |
|   | 8        | 8 Name  | 922   |
|   | 9        | 9 Name  | 923   |
|   | 10       | 10 Name | 923   |
|   | 11       | 11 Name | 924   |
|   | 12       | 12 Name | 922   |
|   | 13       | 13 Name | 924   |



Faculty.dbf

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 42 | 53 | 54 | 42 | 20 | 2e | 30 | 37 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 03 | 50 | 81 | 01 | f0 | 06 | 4f | 52 |
| 4c | 45 | 2d | 49 | 44 | 01 | 14 | 3c |
| 54 | 2d | 4e | 41 | 4d | 45 | 01 | 13 |
| 45 | 54 | 2d | 4e | 55 | 4d | 42 | 45 |

Students.dbf

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 20 | 20 | 20 | 31 | 56 | 31 | 2e | 30 |
| 38 | 31 | 39 | 32 | 44 | 65 | 66 | 61 |
| 61 | 67 | 65 | 20 | 53 | 65 | 74 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 00 | ff | 01 | 00 | 7c | 80 | 00 |
| 45 | 41 | 44 | 45 | 52 | 34 | 0f | 53 |
| 4e | 55 | 4d | 42 | 45 | 52 | 14 | 34 |
| 34 | 21 | 0a | 20 | 20 | 20 | 20 | 20 |
| 42 | 53 | 54 | 42 | 20 | 2e | 30 | 37 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 03 | 50 | 81 | 01 | f0 | 06 | 4f | 52 |
| 4c | 45 | 2d | 49 | 44 | 01 | 14 | 3c |
| 54 | 2d | 4e | 41 | 4d | 45 | 01 | 13 |
| 45 | 54 | 2d | 4e | 55 | 4d | 42 | 45 |

Groups.dbf

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 20 | 20 | 20 | 31 | 56 | 31 | 2e | 30 |
| 38 | 31 | 39 | 32 | 44 | 65 | 66 | 61 |
| 61 | 67 | 65 | 20 | 53 | 65 | 74 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 00 | ff | 01 | 00 | 7c | 80 | 00 |
| 45 | 41 | 44 | 45 | 52 | 34 | 0f | 53 |
| 4e | 55 | 4d | 42 | 45 | 52 | 14 | 34 |
| 34 | 21 | 0a | 20 | 20 | 20 | 20 | 20 |
| 42 | 53 | 54 | 42 | 20 | 2e | 30 | 37 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 03 | 50 | 81 | 01 | f0 | 06 | 4f | 52 |
| 4c | 45 | 2d | 49 | 44 | 01 | 14 | 3c |
| 54 | 2d | 4e | 41 | 4d | 45 | 01 | 13 |
| 45 | 54 | 2d | 4e | 55 | 4d | 42 | 45 |

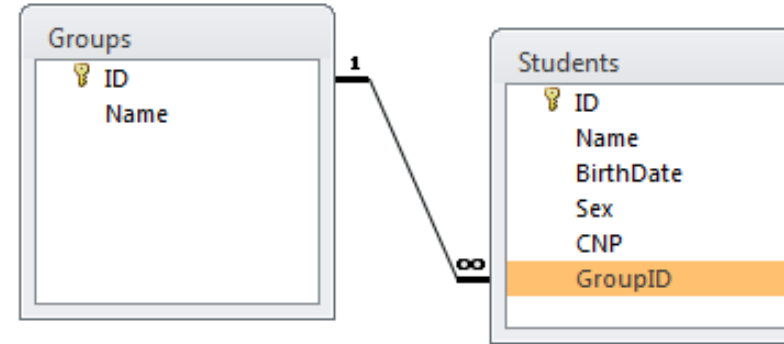


# Independența fizică a datelor

Datagrid View

First Previous Next Last

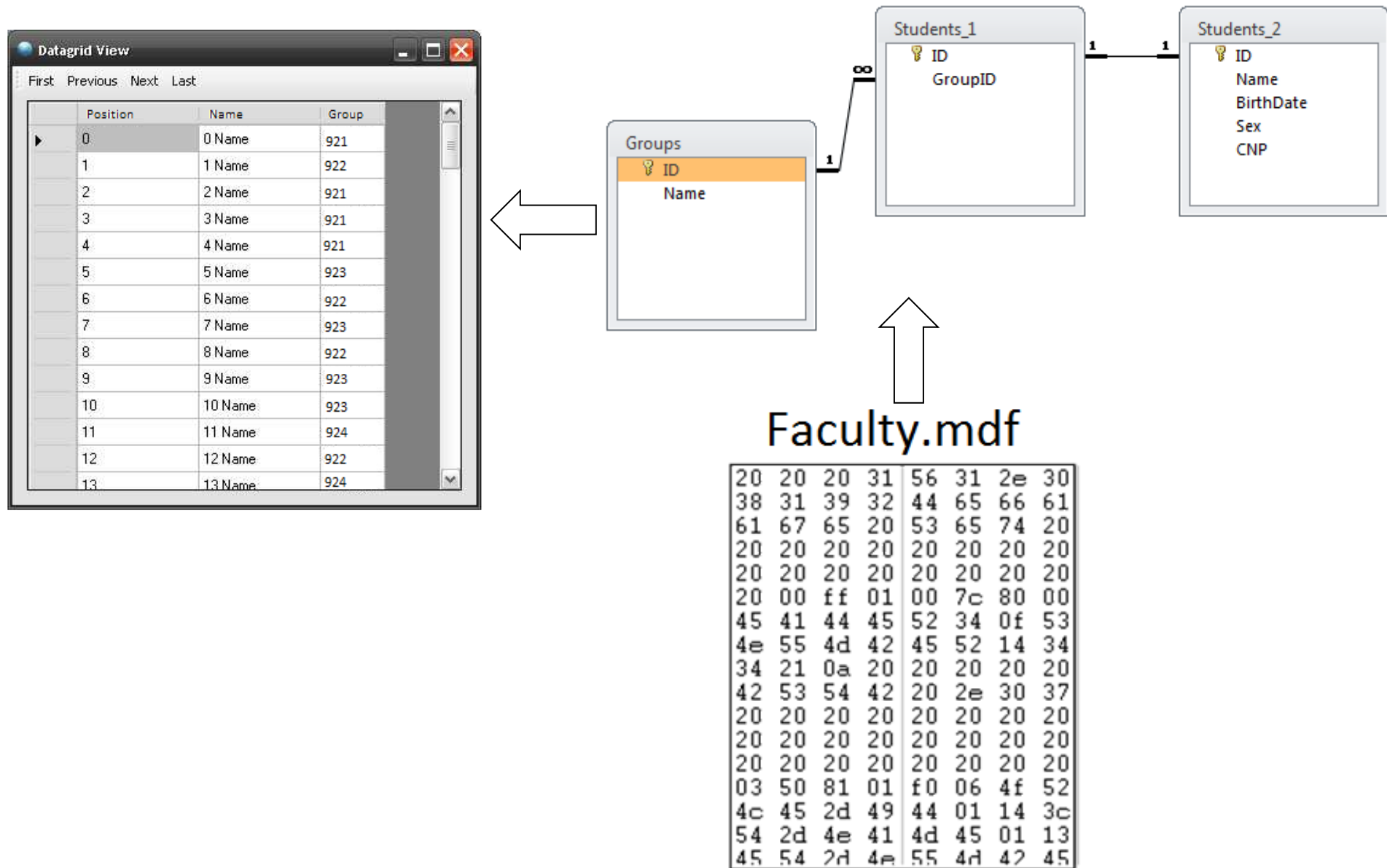
|   | Position | Name    | Group |
|---|----------|---------|-------|
| ▶ | 0        | 0 Name  | 921   |
|   | 1        | 1 Name  | 922   |
|   | 2        | 2 Name  | 921   |
|   | 3        | 3 Name  | 921   |
|   | 4        | 4 Name  | 921   |
|   | 5        | 5 Name  | 923   |
|   | 6        | 6 Name  | 922   |
|   | 7        | 7 Name  | 923   |
|   | 8        | 8 Name  | 922   |
|   | 9        | 9 Name  | 923   |
|   | 10       | 10 Name | 923   |
|   | 11       | 11 Name | 924   |
|   | 12       | 12 Name | 922   |
|   | 13       | 13 Name | 924   |



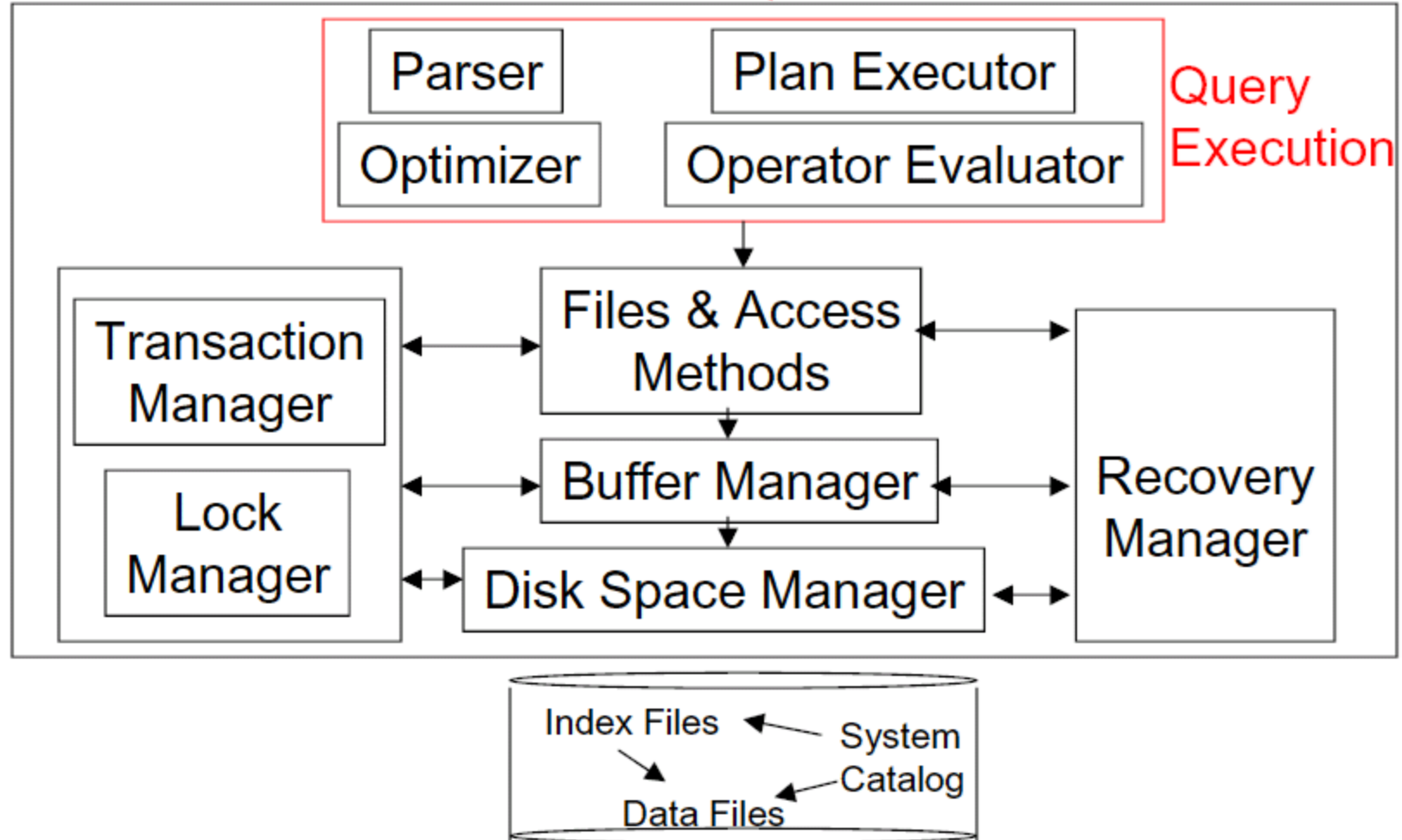
Faculty.mdf

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 20 | 20 | 20 | 31 | 56 | 31 | 2e | 30 |
| 38 | 31 | 39 | 32 | 44 | 65 | 66 | 61 |
| 61 | 67 | 65 | 20 | 53 | 65 | 74 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 00 | ff | 01 | 00 | 7c | 80 | 00 |
| 45 | 41 | 44 | 45 | 52 | 34 | 0f | 53 |
| 4e | 55 | 4d | 42 | 45 | 52 | 14 | 34 |
| 34 | 21 | 0a | 20 | 20 | 20 | 20 | 20 |
| 42 | 53 | 54 | 42 | 20 | 2e | 30 | 37 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 20 | 20 | 20 | 20 | 20 | 20 | 20 | 20 |
| 03 | 50 | 81 | 01 | f0 | 06 | 4f | 52 |
| 4c | 45 | 2d | 49 | 44 | 01 | 14 | 3c |
| 54 | 2d | 4e | 41 | 4d | 45 | 01 | 13 |
| 45 | 54 | 2d | 4e | 55 | 4d | 42 | 45 |

# Independența logică a datelor



# Structura unui SGBD



# Structura fizică a fișierelor BD

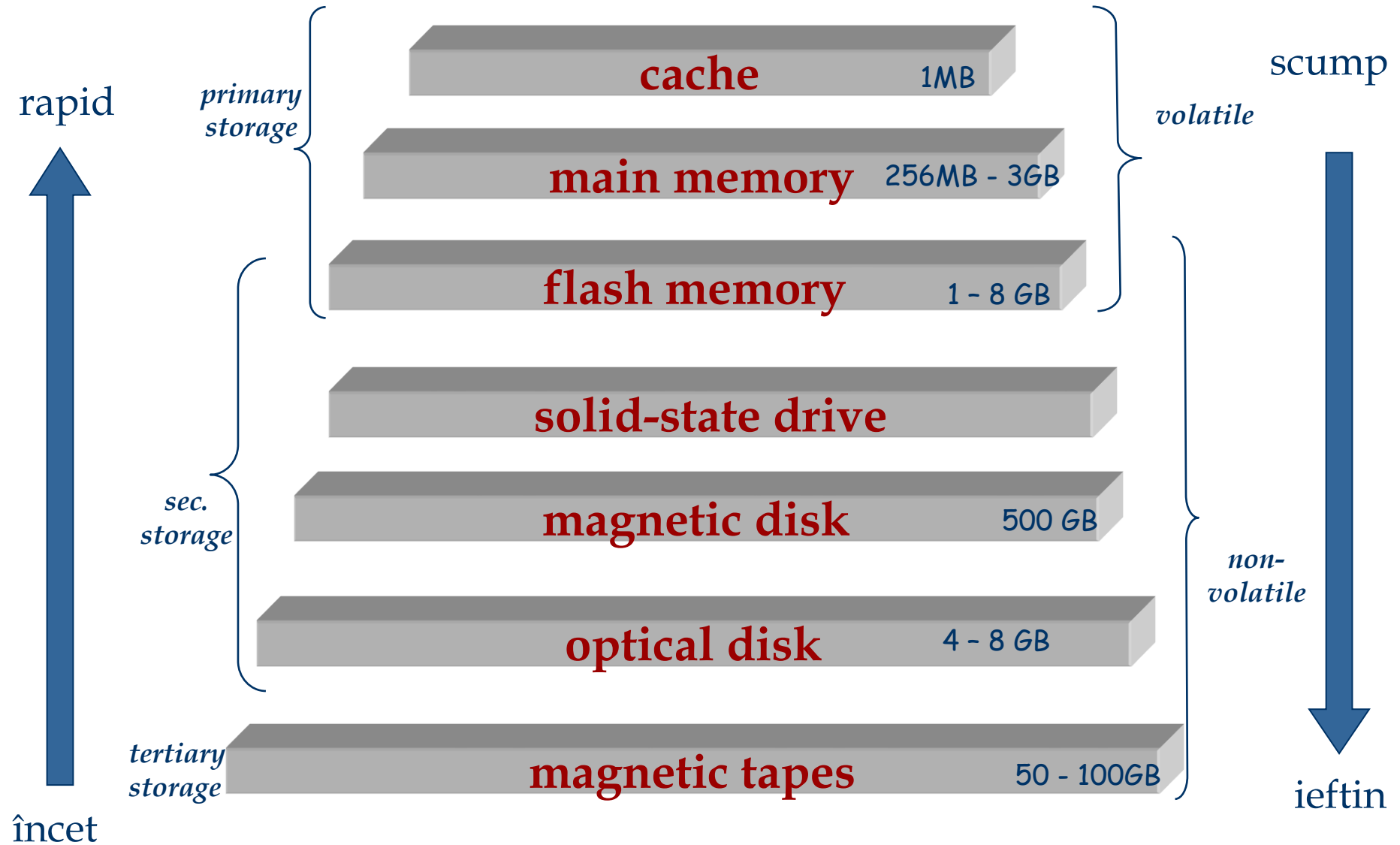
- SGBD-urile stochează informația pe disc magnetic
- Acest lucru are implicații majore în proiectarea unui SGBD!
  - **READ**: transfer date de pe disc în memoria internă
  - **WRITE**: transfer date din memoria internă pe disc

Ambele operații sunt costisitoare, comparativ cu operațiile *in-memory*, deci trebuie planificate corespunzător!

# De ce nu stocăm totul în memoria internă?

- Răspuns (tipic):
  - Costă prea mult
  - Memoria internă este volatilă (datele trebuie să fie persistente)
- Procedură tipică(“ierarhie de stocare”)
  - RAM – pentru datele utiliz. curent (*primary storage*)
  - *Hard-disks* – pentru baza de date (*secondary storage*)
  - Bandă – pentru arhivarea versiunilor anterioare ale datelor (*tertiary storage*)

# Ierarhia mediilor de stocare

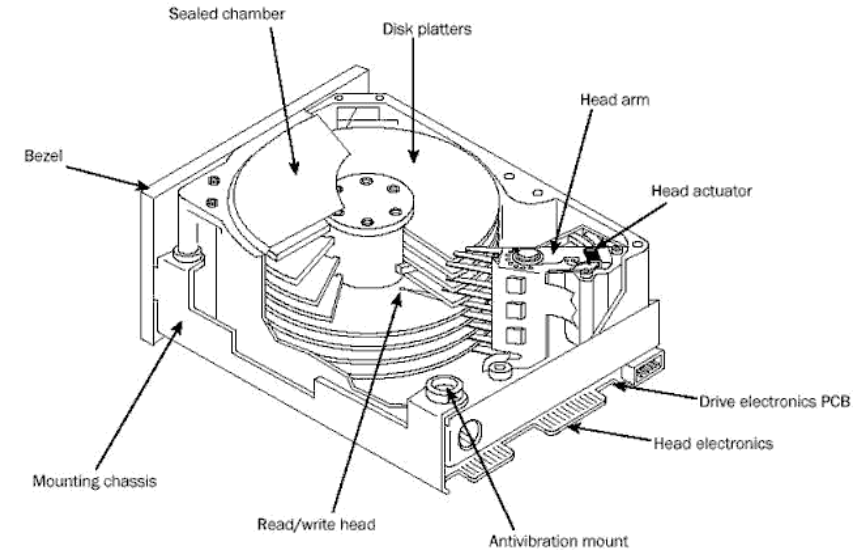


# Legea lui Moore

- Gordon Moore: *“Integrated circuits are improving in many ways, following an exponential curve that doubles every 18 months”*
    - Viteza procesoarelor
    - Numărul de biți de pe un chip
    - Numărul de octeți (*bytes*) pe un hard disk
  - Parametrii ce NU urmează legea lui Moore:
    - Viteza de accesare a datelor în memoria internă
    - Viteza de rotație a discului
- ⇒ Latența devine progresiv mai mare
- Timpul de transfer între nivelele ierarhiei mediilor de stocare este tot mai mare în comparație cu timpul de calcul

# Discuri magnetice

- Utilizat ca mediu de stocare secundar
- Avantaj major asupra benzilor: *acces direct*
- Datele sunt stocate și citite în unități numite **blocuri** sau **pagini**.
- Spre deosebire de memoria internă, timpul de transfer blocurilor/paginilor variază în funcție de poziția acestora pe disc.

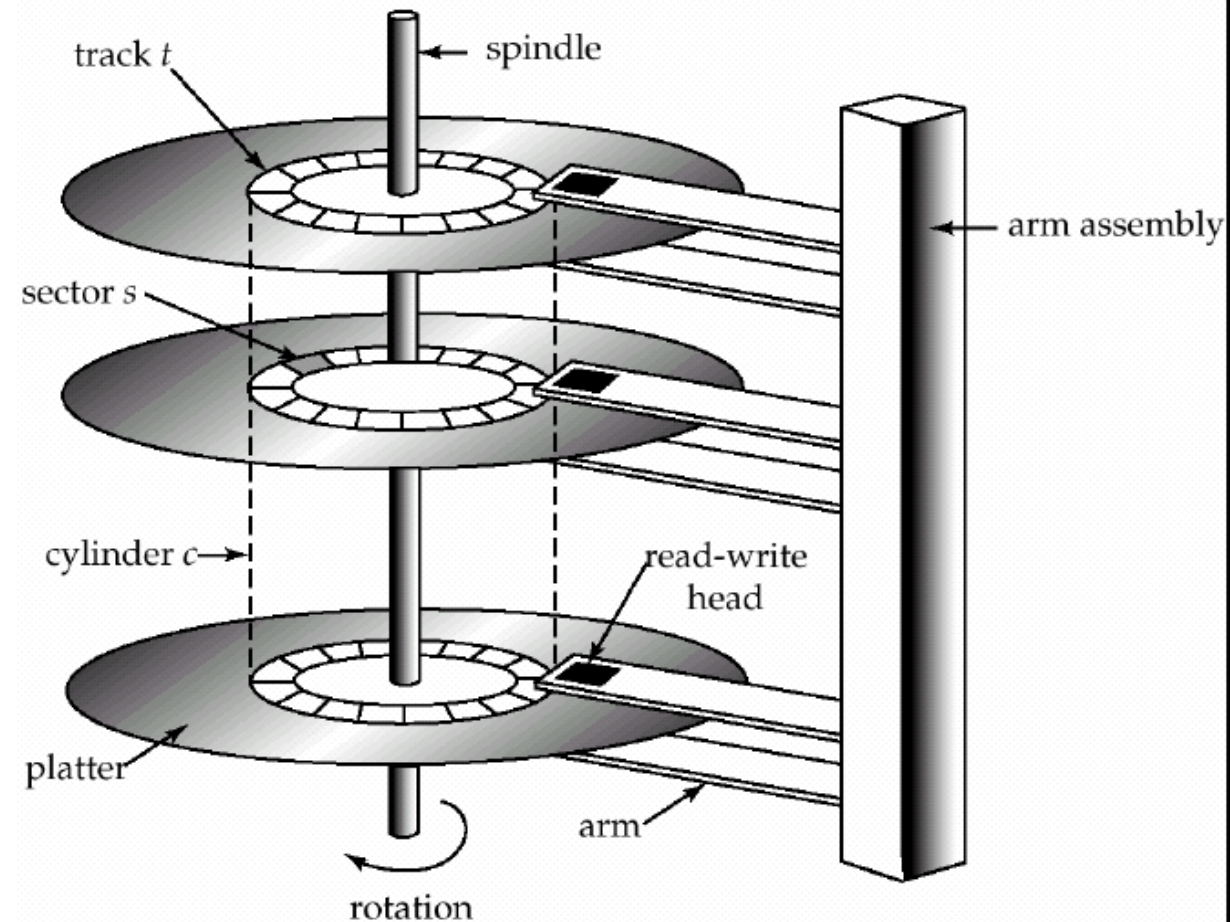


! Poziția relativă a paginilor pe disc  
are un impact major asupra performanței unui SGBD!



# Componentele unui disc dur (*hard disk*)

- Rotația platanelor (90rps)
- Ansamblu de brațe ce se deplasează pentru poziționarea capului magnetic pe pista dorită. Pistele aflate la aceeași distanță de centrul platanelor formează un **cilindru** (imaginar!).
- Un singur cap citește/ scrie la un moment dat.
- Un **bloc** e un multiplu de **sectoare** (care e fix).



# Accesarea unei pagini (bloc)

- Timp de acces (citire/scriere) a unui bloc:
  - seek time* (mutare braț pentru poziționarea capului de citire/scriere pe pistă)
  - rotational delay* (timp poziționare bloc sub cap)
  - transfer time* (transfer date de pe/pe disc)
- *Seek time* și *rotational delay* domină.
  - *Seek time* variază între 1 și 20msec
  - *Rotational delay* variază între 0 și 10msec
  - *Transfer rate* e de aproximativ 1msec pe 4KB (pagină)
- Reducerea costului I/O: *reducere seek/rotational delays!*
- Soluții *hardware* sau *software*?

# Aranjarea paginilor/blocurilor pe disc

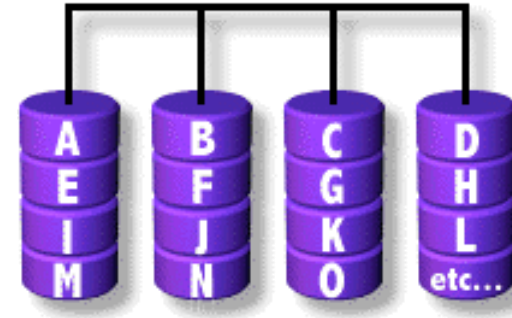
- Conceptul de *next block*:
  - blocuri pe aceeași pistă, urmate de
  - blocuri pe același cilindru, urmate de
  - blocuri pe cilindri adiacenți
- Blocurile dintr-un fișier trebuie dispuse secvențial pe disc (*'next'*), pentru a minimiza *seek delay* și *rotational delay*.
- În cazul unei *scanări secvențiale*, citirea de pagini în avans (*pre-fetching*) este esențială!

# RAID

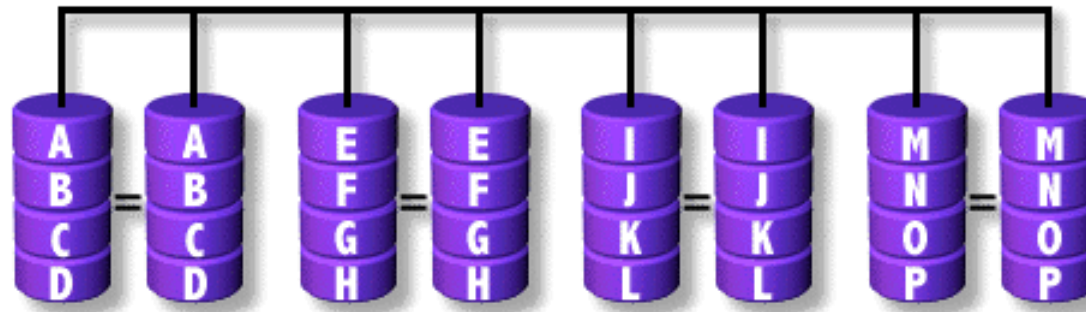
- *Disk Array*: configurație de discuri magnetice ce abstractizează un singur disc.
  - mult mai puțin costisitor; se utilizează mai multe discuri de capacitate mică și ieftine în locul unui disc de capacitate ridicată
- Scop: Creșterea performanței și fiabilității.
- Tehnici:
  - Data striping*: distribuirea datelor pe mai multe discuri (în partiții prestabilite - *striping unit*)
  - Mirroring*: stocarea automată a unei copii a datelor pe alte discuri → redundanță. Permite reconstruirea datelor în cazul unor defecte ale discurilor.

# Nivele RAID

- Nivel 0: Fără redundanță

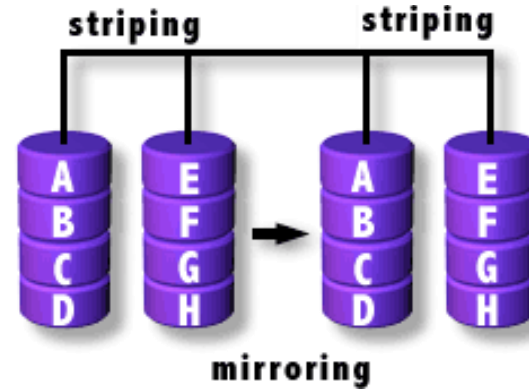


- Nivel 1: Discuri oglindite (*mirrored*)
  - Fiecare disc are o “oglină” (*check disk*)
  - Citiri paralele, o scriere implică două discuri.
  - Rata maximă de transfer = rata de transfer a unui disc

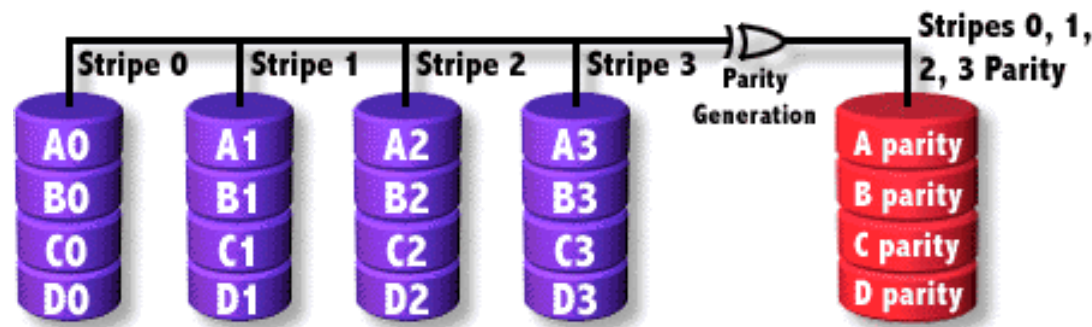


# Nivele RAID

- Nivel 0+1:  
Întreșesut și oglindit

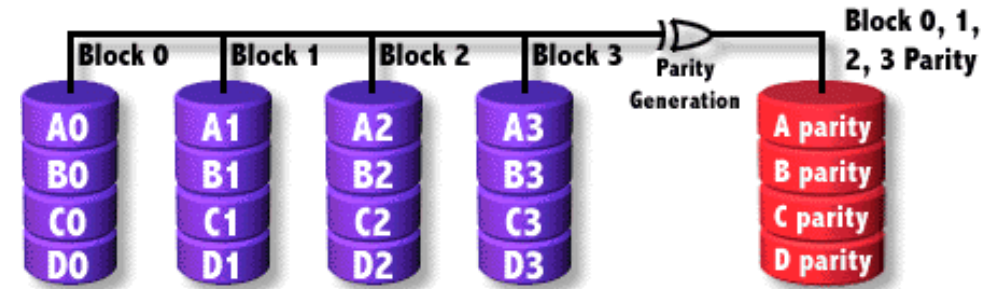


- Nivel 3: Bit de paritate intercalat
  - *Striping Unit*: un bit. un singur disc de verificare
  - Fiecare citire și scriere implică toate discurile

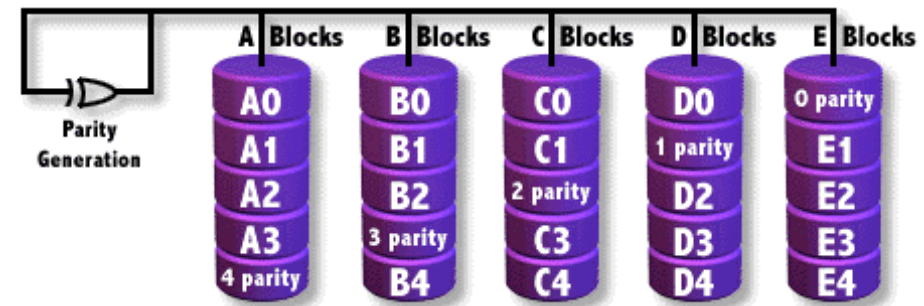


# Nivele RAID

- Nivel 4: Block de paritate
  - *Striping unit*: un bloc. un singur disc de verificare.
  - Citiri în paralel pentru cereri de dimensiune mică
  - Scrierile implică blocul modificat și discul de verificare



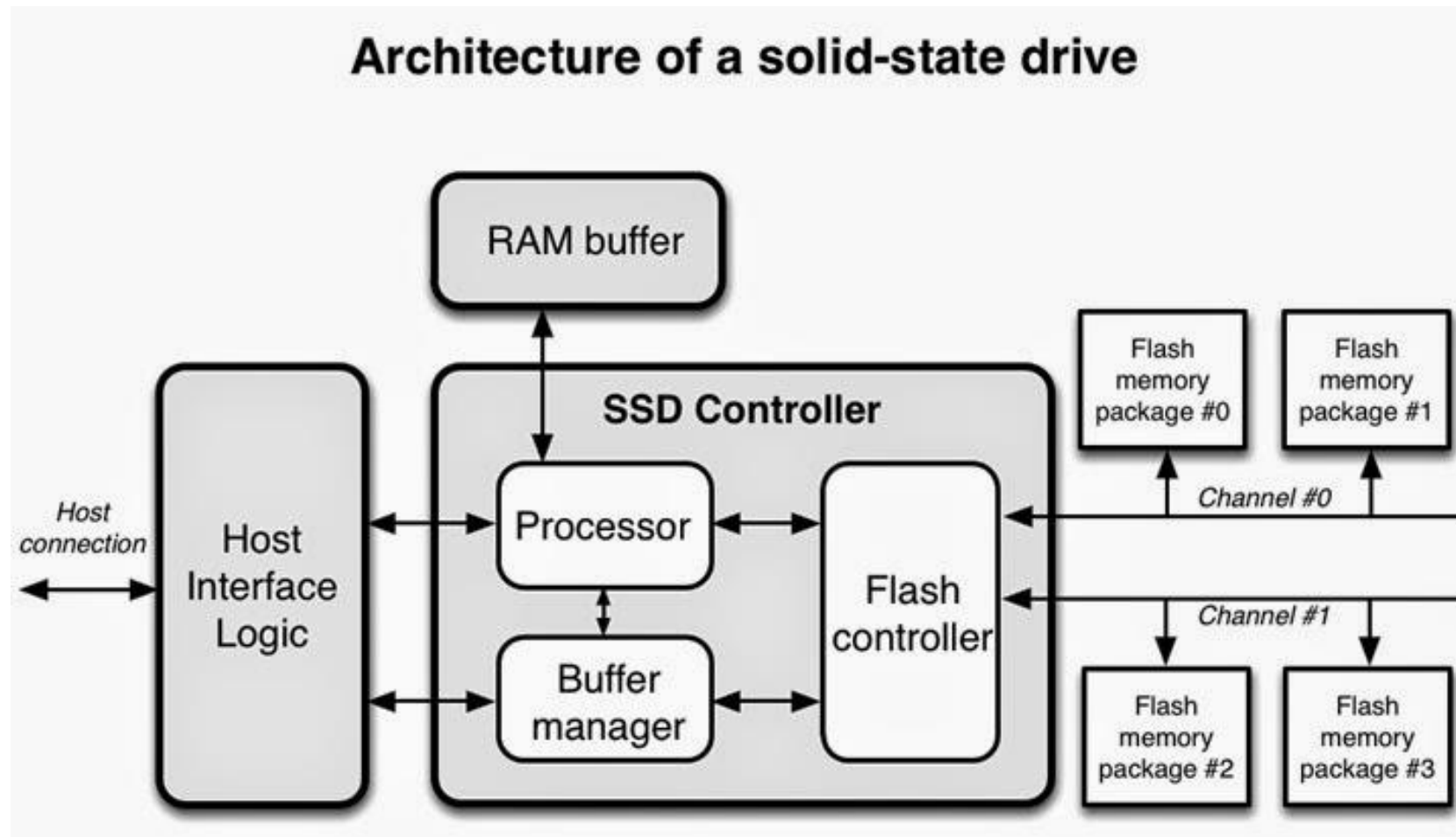
- Nivel 5: Bloc de paritate distribuit
  - Similar cu RAID 4, dar blocurile de paritate sunt distribuite pe toate discurile





# Solid State Drive

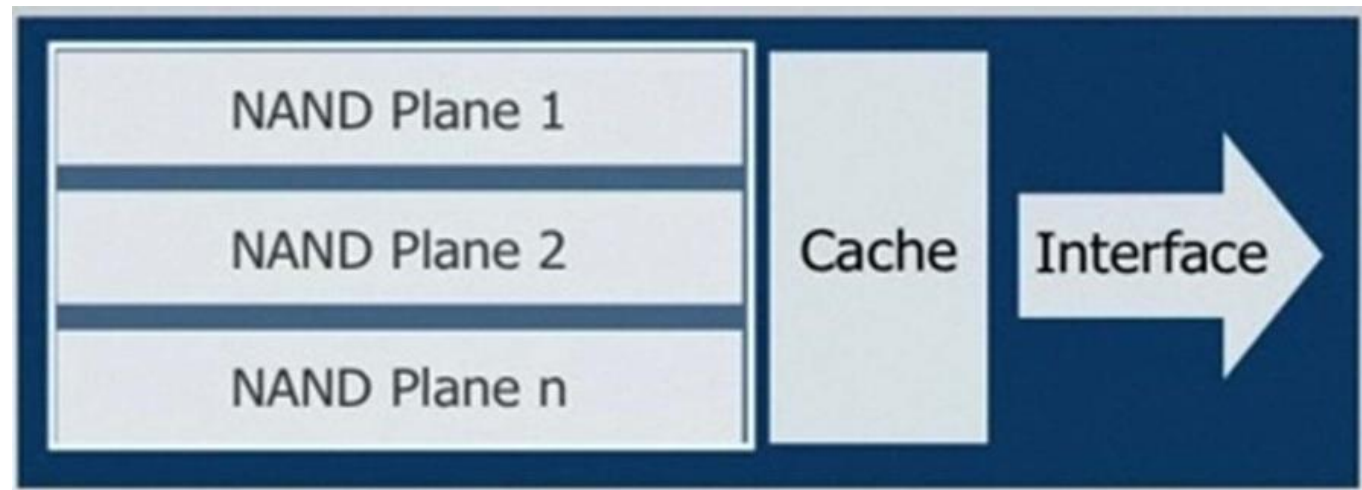
- SSD-urile conțin mai multe componente NAND flash (16/32 GB )





# Solid State Drive

- NAND Flash Media e format din mai multe celule NAND aranjate pe planuri multiple:
  - aceste planuri permit accesul paralel la NAND
  - permit, de asemenea, întrețeserea datelor
- Datele sunt transferate printr-un *cache*



# Solid State Drive - Avantaje

- Latență foarte mică
  - *seek time* este zero
- Viteze mari de citire și scriere
- Mai robuste fizic
  - Rezistente la șocuri
  - Zero părți mobile
    - Silențioase
    - Consumă puțin
- Excelează la citiri/scrieri de dimensiuni reduse
- “Imun” la fragmentarea datelor

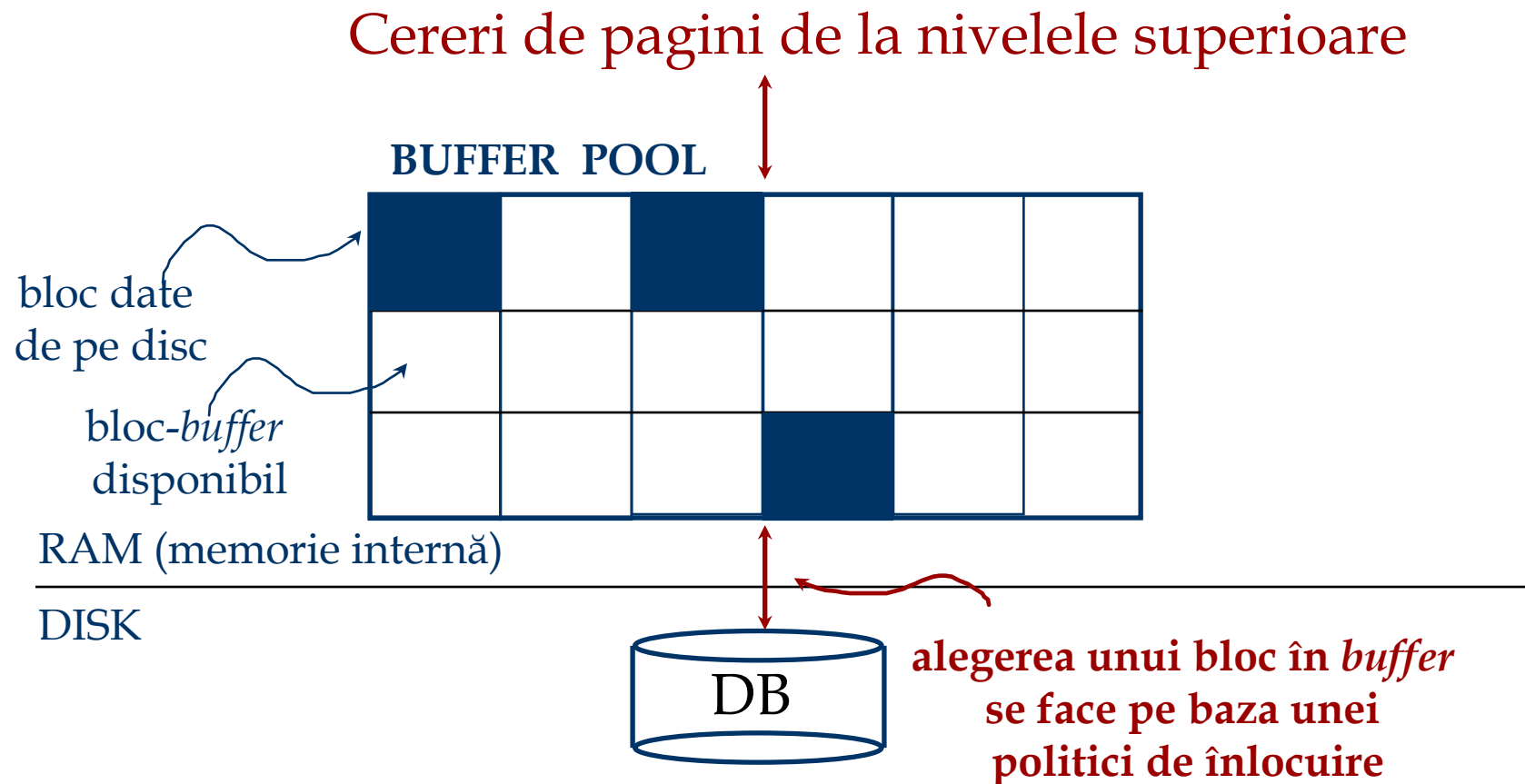
# Solid State Drive – Dezavantaje

- Cost per GB mult mai mare comparativ cu discurile magnetice
- Dimensiuni
  - HDD 3.5'' cu 4TB sunt relativ comune
  - SSD 3.5'' cu 2TB sunt disponibile (rare și scumpe)
- Cicluri de citire/scriere limitate
  - 1 - 2 milioane cicluri de scriere  $\Rightarrow$  uzura MLC (multi-level cell)
  - Sub 5 milioane cicluri de scriere  $\Rightarrow$  uzura SLC

# Gestionare *buffer* (zona de lucru) de către SGBD

- *Buffer* – partiție a memoriei interne utilizată pentru stocarea de copii ale blocurilor de date.
- *Buffer manager* – modul SGBD responsabil cu alocarea spațiului de *buffer* în memoria internă.
- *Buffer manager*-ul este apelat când este necesară accesarea unui bloc de pe disc
  - SGBD-ul operează asupra datelor din memoria internă

# Gestionare *buffer* de către SGBD



- Se actualizează o tabelă cu perechi  $\langle \text{nr\_bloc\_buffer}, \text{id\_bloc\_date} \rangle$

# La cererea unui bloc de date...

- Dacă blocul nu se regăsește în *buffer*:
  - Se alege un bloc disponibil pt. **înlocuire**
  - Dacă blocul conține modificări acesta este transferat pe disc
  - Se citește blocul dorit în locul vechiului bloc
- Blocul e *fixat* și se returnează adresa sa

*! Dacă cererile sunt predictibile (ex. scanări secvențiale)  
pot fi citite în avans mai multe blocuri la un moment dat*

# Gestionare *buffer* de către SGBD

- Programul care a cerut blocul de date trebuie să îl elibereze și să indice dacă blocul a fost modificat:
  - folosește un **dirty bit**.
- Același bloc de date din *buffer* poate fi folosit de mai multe programe:
  - folosește un **pin count**. Un bloc-*buffer* e un candidat pentru a fi înlocuit ddacă **pin count** = 0.
- Modulele *Concurrency Control & Crash Recovery* pot implica acțiuni I/O adiționale la înlocuirea unui bloc-*buffer*.

# Politici de înlocuire a blocurilor în *buffer*

- *Least Recently Used* (LRU): utilizează șablonul de utilizare a blocurilor ca predictor al utilizării viitoare. Interogările au șabloane de acces bine definite (ex, scanările secvențiale), iar un SGBD poate utiliza informațiile din interogare pentru a prezice accesările ulterioare ale blocurilor.
- *Toss-immediate*: eliberează spațiul ocupat de un bloc atunci când a fost procesat ultima înregistrare stocată în blocul respectiv
- *Most recently used* (MRU): după procesarea ultimei înregistrări dintr-un bloc, blocul este eliberat (*pin count* e decrementat) și devine blocul utilizat cel mai recent.



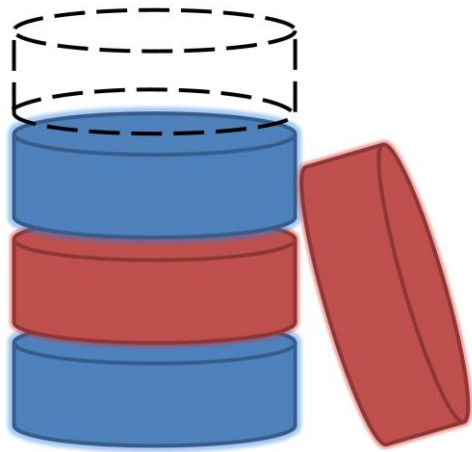
# Politici de înlocuire a blocurilor în *buffer*

- *Buffer Manager* poate utiliza **informații statistice** cu privire la probabilitatea ca o anumită cerere să refere un anumit bloc sau chiar o anumită relație
- Politicile de înlocuite pot avea un impact determinant în ceea ce privește numărul de I/Os – dependent de **șablonul de acces**.
- *Sequential flooding*: problemă generată de LRU + scanări secvențiale repetate.
  - **Nr blocuri-buffer < Nr blocuri în tabelă** → fiecare cerere de pagină determină un I/O. MRU e preferabil într-o astfel de situație.

# SGBD vs. Sistemul de fișiere al SO

- SO gestionează spațiul pe disc și *buffer*-ul.  
De ce o face și un SGBD?
- Diferențe de suport oferit de SO: probleme de portabilitate
- Existența unor limitări (ex. fișierele nu pot fi salvate pe mai multe discuri)
- Gestionarea *buffer*-ului de SGBD presupune abilitatea de a:
  - fixa/*elibera* blocuri, *forța salvarea unui bloc* pe disc (important pentru *concurrency control & crash recovery*),
  - ajustarea *politicii de înlocuire* și citirea de blocuri în avans pe baza șablonului de acces ale operațiilor tipice BD.

# Indexarea bazelor de date



# Regăsirea datelor

- Interogare folosind egalități:
  - *“Find student name whose Age = 20”*
- Interogare folosind intervale:
  - *“Find all students with Grade > 8.50”*
- Parcurgerea secvențială a fișierului este costisitoare
- Dacă datele sunt sortate în fișier:
  - Căutare binară pentru găsirea primei înregistrări (*cost ridicat*)
  - Parcurgerea fișierului pentru găsirea celorlalte înregistrări.

# Indecși

- Indecși sunt fișiere/structuri de date speciale utilizate pentru accelerarea execuției interogărilor.
- Un index se crează pe baza unei *chei de căutare*
  - *Cheia de căutare* e un atribut/set de attribute folosit(e) pentru a căuta înregistrările dintr-o tabelă/fișier.
  - *Cheia de căutare* este diferită de cheia primară / cheia candidat / supercheia
- Un index conține o colecție de înregistrări speciale și permite regăsirea eficientă a “*tuturor înregistrărilor tabeli indexate pentru care cheia de căutare are valoarea k*”.

# Caracteristicile indecșilor

- *Propagarea modificărilor*

- Adăugarea/ștergerea de înregistrări în tabela indexată implică actualizarea tuturor indecșilor definiți pentru acea tabelă.

- Orice modificare a valorilor câmpurilor ce aparțin cheii de căutare presupune actualizarea indecșilor bazați pe acea cheie de căutare

# Caracteristicile indecșilor

## ■ *Dimensiunea indecșilor*

- Deoarece utilizarea unui index presupune citirea acestuia în memoria internă → dimensiune indexului trebuie să fie redusă.
- Ce se întâmplă dacă indexul e (totuși) prea mare?
  - Utilizare structură de indexare parțială
  - Se indexează fișierul index  
(stratificat sau pe o structură arborescentă)

- Teme importante:

- Care este structura unei înregistrări de index?  
*(conținutul indexului)*
- Cum sunt organizate aceste înregistrări?  
*(tehnica de indexare)*



# Variante de structurare a conținutului unui index

- (1) înregistrările originale ce includ cheia de căutare
- (2)  $\langle k, rid \rangle$ ,  $rid$  – referință spre înreg. cu valoarea  $k$  a cheii de căutare
- (3)  $\langle k, \text{listă de } rid\text{-uri} \rangle$

- Alegerea variantei de stocare a intrărilor (înregistrărilor) din index e ortogonală cu tehnica de indexare.
  - Exemple de tehnici de indexare: arbori B+, acces direct
  - Indexul mai conține și informații auxiliare de direcționare a căutărilor spre înregistrările căutate

# Variante de structurare a conținutului unui index

## ■ Varianta (1):

- Indexul și înregistrările originale sunt stocate împreună
- Pentru o colecție de înregistrări se poate crea cel mult un index structurat conform variantei (1) (în caz contrar avem înregistrări duplicate → redundanță → potențiale inconsistențe)
- Dacă înregistrările sunt voluminoase, numărul de blocuri în care se stochează înregistrările este mare → dimensiunea informației auxiliare din index este la rândul său mare.

# Variante de structurare a conținutului unui index

## ■ Variantele (2) și (3):

### ■ Intrările din index referă înregistrările originale

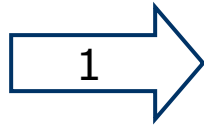
- Intrările din index sunt în general mai mici decât înregistrările (deci sunt variante mai eficiente decât varianta (1), mai ales dacă cheia de căutare este mică).
- Informația auxiliară din index folosită pentru a direcționa căutarea, este la rândul său mai redusă decât în cazul variantei (1).

■ Varianta (3) este mult mai compactă decât varianta (2), dar implică dimensiune variabilă a spațiului de stocare a intrărilor, chiar dacă cheia de căutare este de dimensiune fixă.

# Crearea unui index\*

|         | <i>Name</i> | <i>Age</i> | <i>Grade</i> |
|---------|-------------|------------|--------------|
| $rid_1$ | John        | 22         | 8.50         |
| $rid_2$ | Jack        | 21         | 9.00         |
|         |             | ...        |              |
| $rid_n$ | Peter       | 22         | 10.00        |

cheie de  
căutare

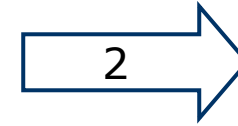


$rid_1 : 22$

$rid_2 : 21$

...

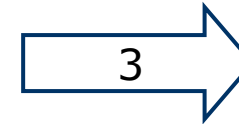
$rid_n : 22$



$22 : rid_1, rid_n \dots$

$21 : rid_2 \dots$

...



intrare



fișier index  
(inversat)

\*Exemplul presupune utilizarea variantei (3) de structura

# Clasificarea indecșilor

- Indecși *primari* vs *secundari*:
  - Un index *primar* se formează pe baza unei chei de căutare ce include cheia primară.
  - Un index e *unic* atunci când cheia de căutare conține o cheie candidat
    - Nu vor fi intrări duplicate
  - În general, indecșii *secundari* conțin duplicate

# Clasificarea indecșilor

- Indecși *clustered* (*grupat*) vs. *un-clustered* (*negrupat*):
  - Un index este *clustered* (grupat) dacă ordinea înregistrărilor este aceeași (sau foarte apropiată) cu ordinea intrărilor din index.
  - Varianta (1) implică grupare; de asemenea în practică, gruparea implică utilizarea primei variante de structurare a indecșilor.
  - O tabelă poate fi indexată grupat (*clustered*) pentru cel mult o cheie de căutare.
  - Costul regăsirii datelor prin intermediul unui index e influențat *decisiv* de grupare!

# Index Clustered vs. Un-clustered

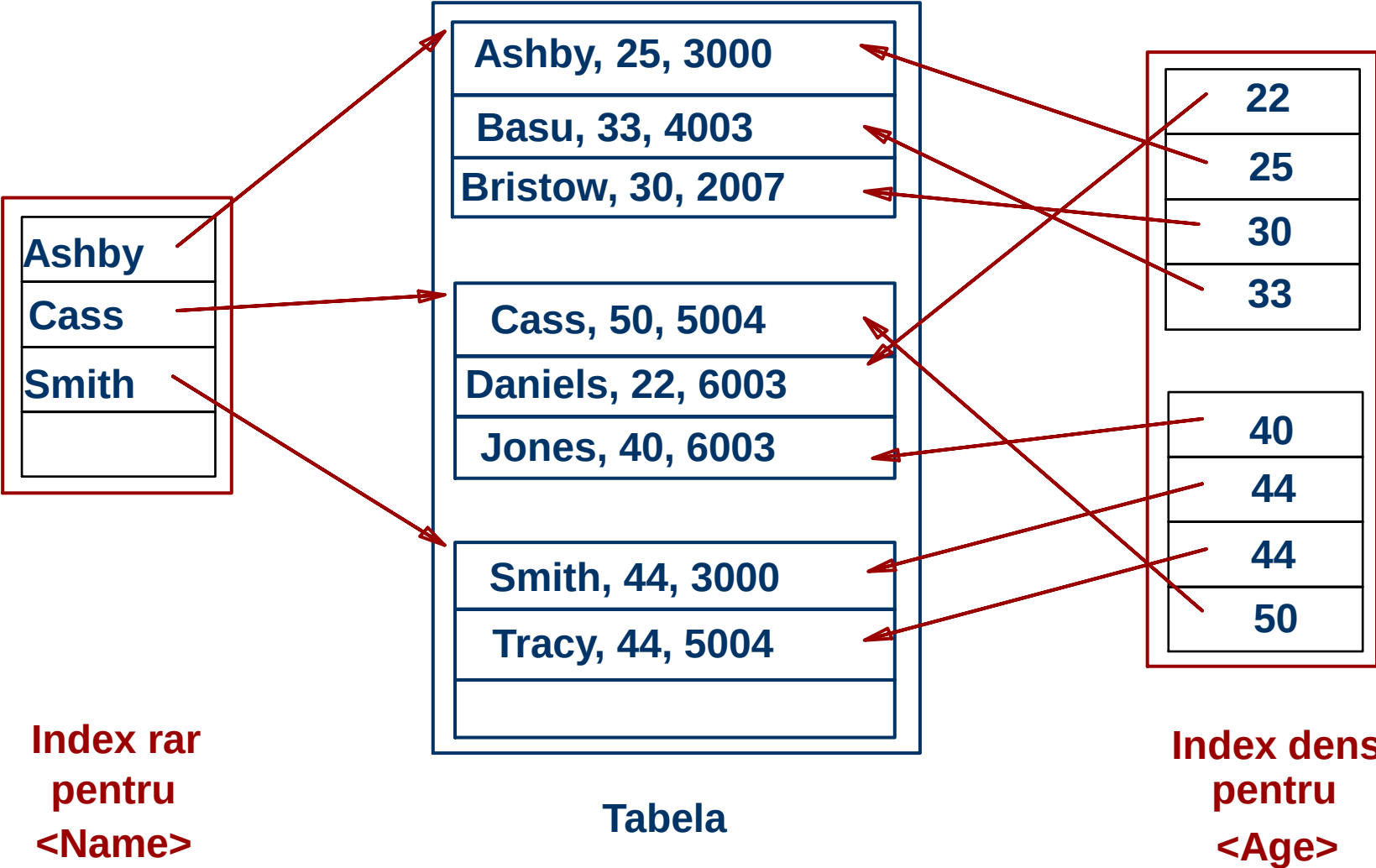
- Pentru a construi indecși grupați:
  - Se ordonează înregistrările în cadrul fișierului (*heap file*)
  - Se rezervă spațiu în fiecare pagină de memorie, pentru a se absorbi viitoarele inserări
    - dacă spațiul suplimentar e consumat, se vor forma liste înlănțuite de pagini suplimentare
    - E necesară o reorganizare periodică pentru a se asigura o performanță ridicată
- Întreținerea indecșilor grupați e foarte costisitoare

# Clasificarea indecșilor

- Indecși *denși* vs. *rari*:
- Un index este *dens* dacă are cel puțin o intrare pentru fiecare valoare a cheii de căutare (ce se regăsește în înregistrări)
  - Mai multe intrări pot avea aceeași valoare pentru cheia de căutare în cazul utilizării variantei (2) de stocare
  - Varianta (1) presupune implicit ca indexul e dens.
- Un index e *rar* dacă memorează o intrare pentru fiecare pagină de înregistrări
  - Toți indecșii rari sunt grupați (*clustered*)!
  - Indecșii rari sunt mai compacti.



# Clasificarea indecșilor

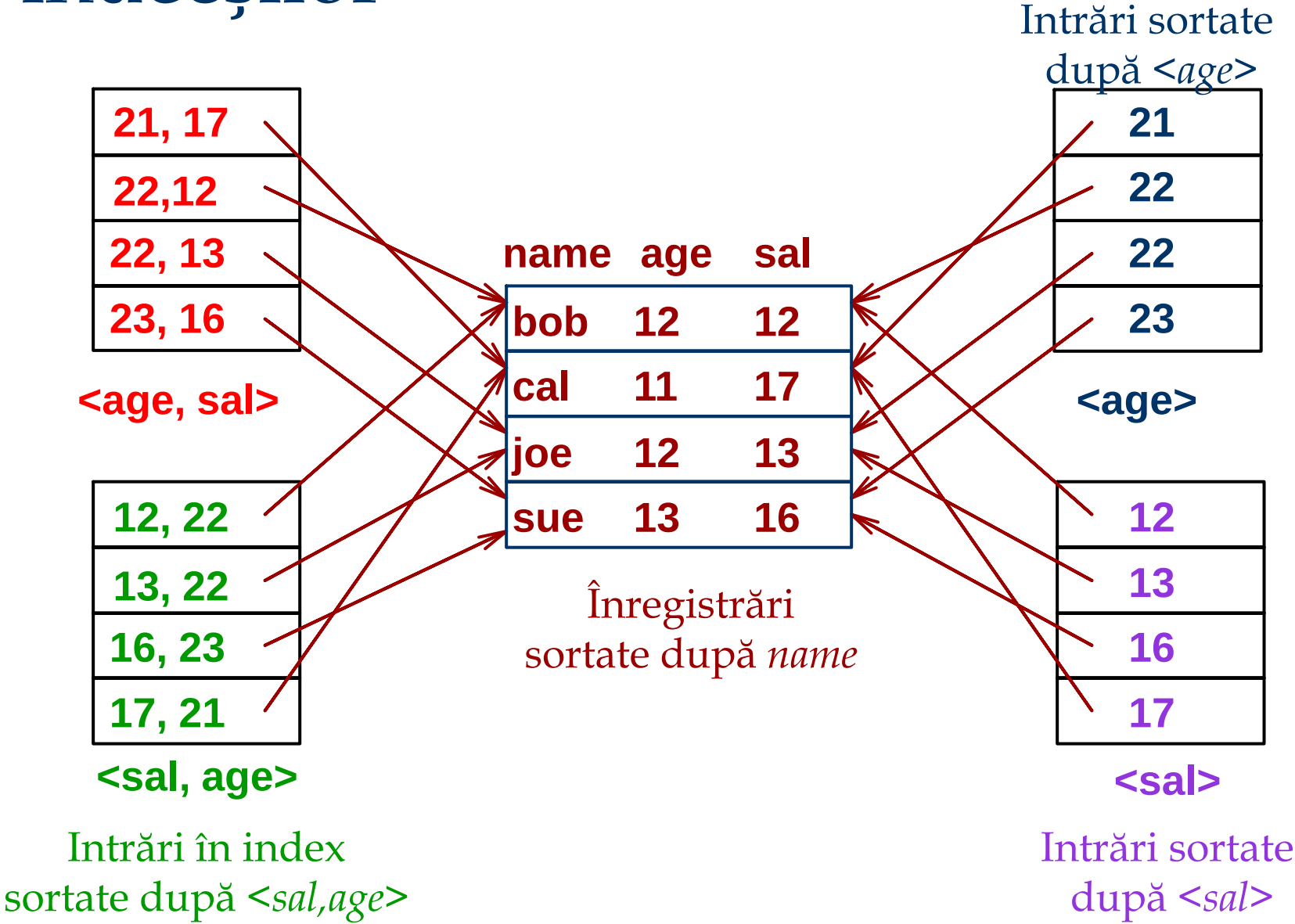


# Clasificarea indecșilor

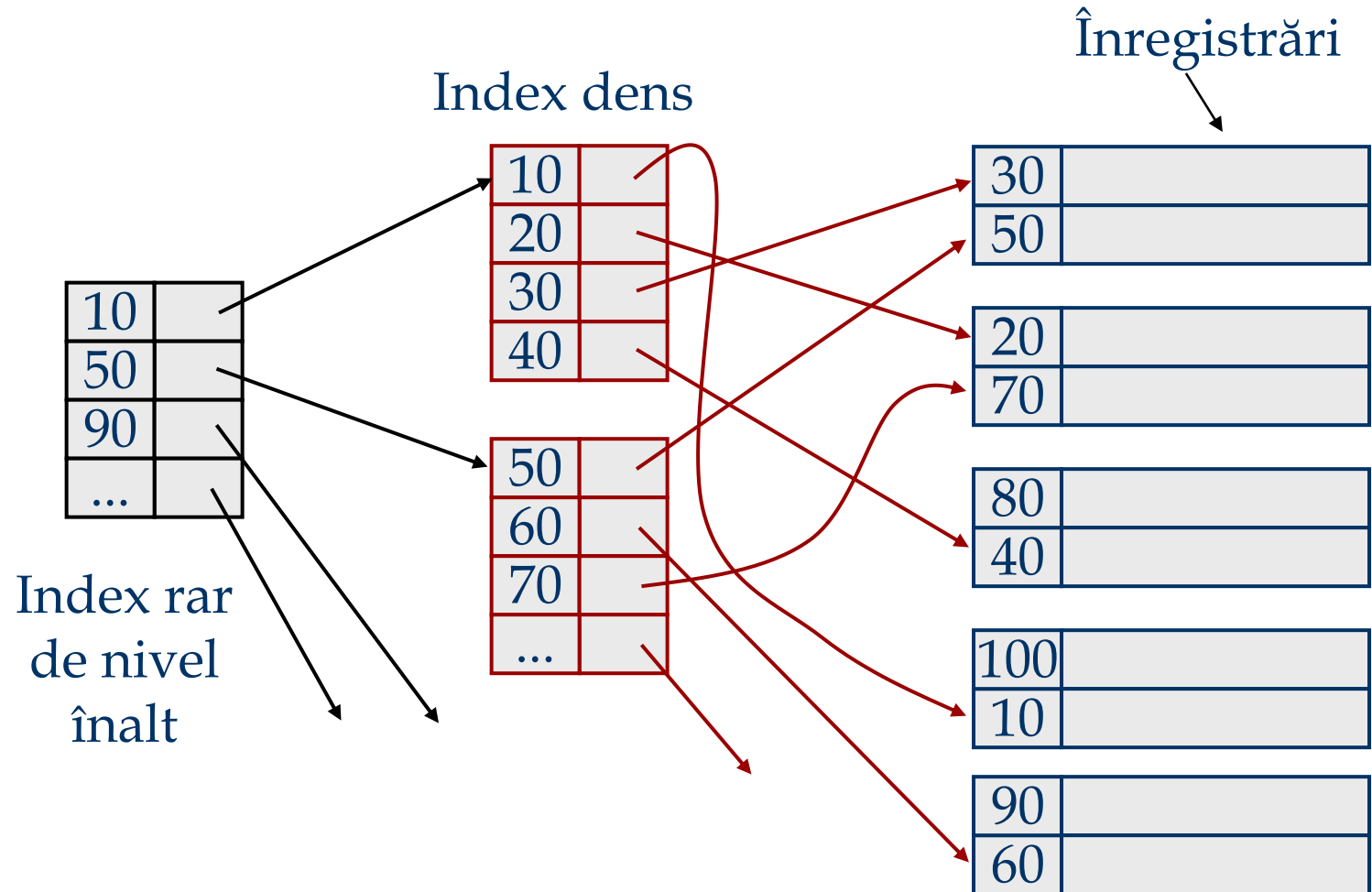
## Chei de căutare compuse:

- Utile atunci când căutarea se realizează după o combinație de câmpuri
  - Interogări cu egalitate (valoarea fiecărui câmp e egală cu o valoare constantă):  
*age=20 și sal =25*
  - Interogări cu interval (valoarea unui câmp nu e o constantă):  
*age=20 și sal > 30*
- Intrările din index sunt sortate după cheia de căutare pentru a putea fi utile interogărilor cu interval.
  - Ordine lexicografică, sau
  - Ordine spațială.

# Clasificarea indecșilor



# Indecși multinivel (indexare multiplă)



# Exemplu

- Înregistrările tablei Students sunt stocate într-un fișier sortate după câmpul *Age*. Fiecare pagină poate stoca un număr maxim de 3 înregistrări

| <i>ID</i> | <i>Name</i> | <i>Age</i> | <i>Grade</i> | <i>Course</i> |
|-----------|-------------|------------|--------------|---------------|
| 53831     | Melanie     | 11         | 7.8          | Music         |
| 53832     | George      | 12         | 8.0          | Music         |
| 53666     | John        | 18         | 9.4          | ComputerS     |
| 53688     | Sam         | 19         | 9.2          | Music         |
| 53650     | Sam         | 19         | 9.8          | ComputerS     |

- Determinați intrările în fiecare dintre următorii indecși (folosiți  $\langle page\_id, slot\ no \rangle$  pentru identificarea unui tuplu).

# Exemplu

| <i>ID</i> | <i>Name</i> | <i>Age</i> | <i>Grade</i> | <i>Course</i> |
|-----------|-------------|------------|--------------|---------------|
| 53831     | Melanie     | 11         | 7.8          | Music         |
| 53832     | George      | 12         | 8.0          | Music         |
| 53666     | John        | 18         | 9.4          | ComputerS     |
| 53688     | Sam         | 19         | 9.2          | Music         |
| 53650     | Sam         | 19         | 9.8          | ComputerS     |

1. *Age* – Dens, varianta (1)

- întreaga tabelă

2. *Age* – Dens, varianta (2)

$(11, \langle 1, 1 \rangle) (12, \langle 1, 2 \rangle) (18, \langle 1, 3 \rangle) (19, \langle 2, 1 \rangle) (19, \langle 2, 2 \rangle)$

3. *Age* – Dens, varianta (3)

$(11, \langle 1, 1 \rangle) (12, \langle 1, 2 \rangle) (18, \langle 1, 3 \rangle) (19, \langle 2, 1 \rangle, \langle 2, 2 \rangle)$

4. *Age* – Rar, varianta (1)

- un astfel de index nu poate fi construit (definiție)

# Exemplu

| <i>ID</i> | <i>Name</i> | <i>Age</i> | <i>Grade</i> | <i>Course</i> |
|-----------|-------------|------------|--------------|---------------|
| 53831     | Melanie     | 11         | 7.8          | Music         |
| 53832     | George      | 12         | 8.0          | Music         |
| 53666     | John        | 18         | 9.4          | ComputerS     |
| 53688     | Sam         | 19         | 9.2          | Music         |
| 53650     | Sam         | 19         | 9.8          | ComputerS     |

5. *Age* – rar, varianta (2)

(11,<1,1>) (19,<2,1>) - ordinea intrărilor e semnificativă

6. *Age* – rar, varianta (3)

(11,<1,1>) (19,<2,1>,<2,2>) - ordinea intrărilor e semnificativă

7. *Grade* – Dens, varianta (1)

7.8, 8.0, 9.2, 9.4, 9.8

8. *Grade* – Dense, varianta (2)

(7.8,<1,1>)(8.0,<1,2>)(9.2,<2,1>)(9.4,<1,3>)(9.8,<2,2>)

# Exemplu

| <i>ID</i> | <i>Name</i> | <i>Age</i> | <i>Grade</i> | <i>Course</i> |
|-----------|-------------|------------|--------------|---------------|
| 53831     | Melanie     | 11         | 7.8          | Music         |
| 53832     | George      | 12         | 8.0          | Music         |
| 53666     | John        | 18         | 9.4          | ComputerS     |
| 53688     | Sam         | 19         | 9.2          | Music         |
| 53650     | Sam         | 19         | 9.8          | ComputerS     |

9. *Grade* – Dense, varianta (3)

$(7.8, \langle 1, 1 \rangle)(8.0, \langle 1, 2 \rangle)(9.2, \langle 2, 1 \rangle)(9.4, \langle 1, 3 \rangle)(9.8, \langle 2, 2 \rangle)$

10. *Grade* – Rar, varianta (1)

- un astfel de index nu poate fi construit (definiție)

11. *Grade* – Rar, varianta (2)

- nu e posibil (valorile cheii de căutare nu sunt ordonate)

12. *Grade* – Rar, varianta (3)

- nu e posibil (valorile cheii de căutare nu sunt ordonate)



# Indecși convenționali

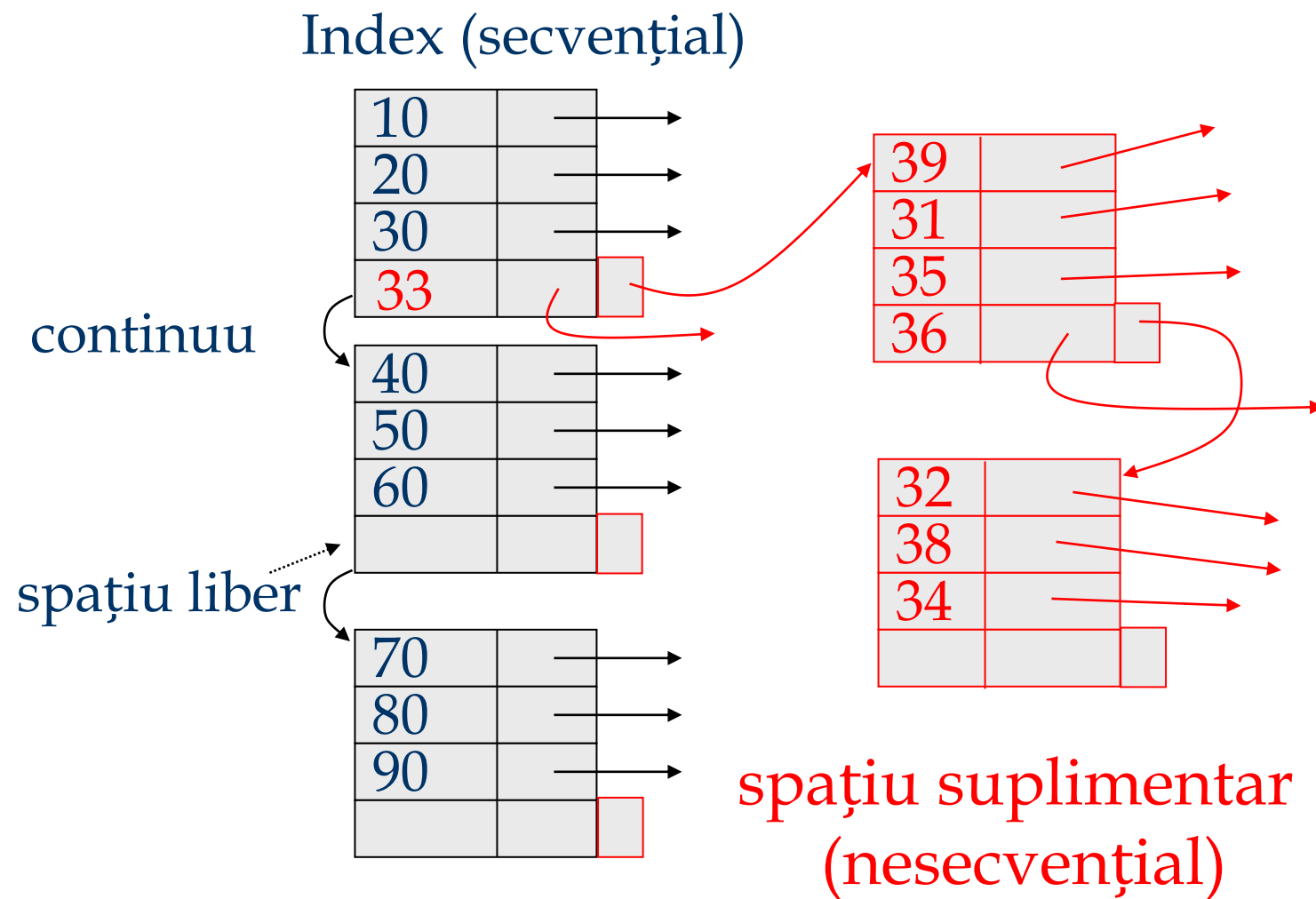
- Avantaje:

- Simpli
- Indexul e un fișier secvențial → potrivit pentru parcurgeri

- Dezavantaje:

- Inserările și/sau ștergerile sunt costisitoare
- Se pierde secvențialitatea & echilibrul

# Exemplu



# Înțelegere *workload*

- Pentru fiecare interogare utilizată:
  - Care sunt relațiile/tabelele accesate?
  - Care sunt atributele/câmpurile returnate?
  - Care dintre atribute sunt implicate în condiții de selecție/join? Cât de selective sunt aceste condiții?
- Pentru fiecare actualizare:
  - Care dintre atribute sunt implicate în condiții de selecție/join? Cât de selective sunt aceste condiții?
  - Ce tip de actualizare este (INSERT/DELETE/UPDATE), și care sunt atributele afectate?

# Alegerea indecșilor

- Ce index trebuie creat?
- Pentru fiecare index, care e varianta de structurare utilizată?
- **Abordare:** Se analizează cele mai importante interogări. Pentru fiecare se consideră cel mai optim plan de execuție folosind indecșii existenți. Vom crea un nou index doar dacă acesta va genera un plan mai bun.
  - Evident, acest lucru implică să înțelegem cum evaluează un SGBD interogările și cum sunt create **planurile de execuție!**
- Înainte de a crea un nou index, trebuie să evaluăm impactul actualizării acestuia!
  - Indecșii accelerează execuția interogărilor, dar încetinesc actualizările. De asemenea, necesită spațiu suplimentar de stocare.

# Alegerea indecșilor

- Atributele din clauza WHERE sunt chei de căutare “candidat”
  - Egalitățile sugerează o structură cu acces direct
  - Intervalele sugerează utilizarea unei structuri arborescente
    - Gruparea e foarte utilă în aceste situații; dar și în cazul egalităților atunci când numărul de duplicate este mare.
- Atunci când clauza WHERE are mai multe condiții se pot lua în considerare chei de căutare compuse
  - Ordinea atributelor e importantă pentru a evalua condițiile cu intervale
  - Pentru interogările importante acești indecși pot determina strategii de execuție **index-only**
- Vor fi aleși indecșii de care beneficiază cele mai multe interogări posibile. Deoarece se poate defini un singur index grupat pe o tabelă, acesta va fi ales astfel încât să beneficieze de acest index o interogare foarte importantă

# Chei de căutare compuse

- Pentru returnarea înregistrărilor din *Employees* cu  $age=30$  AND  $sal=4000$ , un index pe  $\langle age, sal \rangle$  e mai potrivit decât un index pe  $age$  sau unul pe  $sal$ .
- Dacă avem:  $20 < age < 30$  AND  $3000 < sal < 5000$ :
  - Un index arborescent grupat pe  $\langle age, sal \rangle$  sau  $\langle sal, age \rangle$  e foarte bun.
- În cazul:  $age=30$  AND  $3000 < sal < 5000$ :
  - Index grupat  $\langle age, sal \rangle$  e mai bun decât index pe  $\langle sal, age \rangle$
- Indecșii compuși sunt mai voluminoși și sunt actualizați mai des

# Exemple de indecși grupați

- Index arborescent pe *E.age*
  - Cât de selectivă e condiția?
  - Indexul e grupat?
- Utilizare GROUP BY
  - Dacă mai multe înregistrări au *E.age > 10*, un index pe *E.age* urmat de ordonarea înregistrărilor e costisitor
  - Index grupat pe *E.dno* e mai potrivit!
- Egalitate cu duplicate:
  - Index grupat pe *E.hobby*

```
SELECT E.dno  
FROM Employees E  
WHERE E.age>40
```

```
SELECT E.dno, COUNT (*)  
FROM Employees E  
WHERE E.age>10  
GROUP BY E.dno
```

```
SELECT E.dno  
FROM Employees E  
WHERE E.hobby='Stamps'
```

# Planuri *Index-Only*

- Anumite interogări pot fi executate fără a accesa tabelele originale atunci când este disponibil un index potrivit.

*<E.dno>*

```
SELECT D.mgr  
FROM Dept D, Emp E  
WHERE D.dno=E.dno
```

*<E.dno,E.eid> Tree index!*

```
SELECT D.mgr, E.eid  
FROM Dept D, Emp E  
WHERE D.dno=E.dno
```

*<E.dno>*

```
SELECT E.dno, COUNT(*)  
FROM Emp E  
GROUP BY E.dno
```

*<E.dno,E.sal> Tree index!*

```
SELECT E.dno, MIN(E.sal)  
FROM Emp E  
GROUP BY E.dno
```

*<E.age,E.sal> or <E.sal, E.age>  
Tree index!*

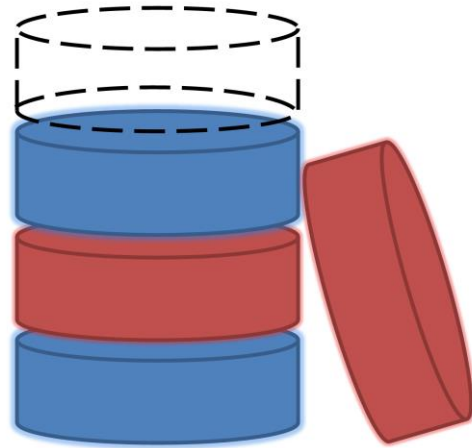
```
SELECT AVG(E.sal)  
FROM Emp E  
WHERE E.age=25 AND  
E.sal BETWEEN 3000 AND 5000
```



# Indexarea azi

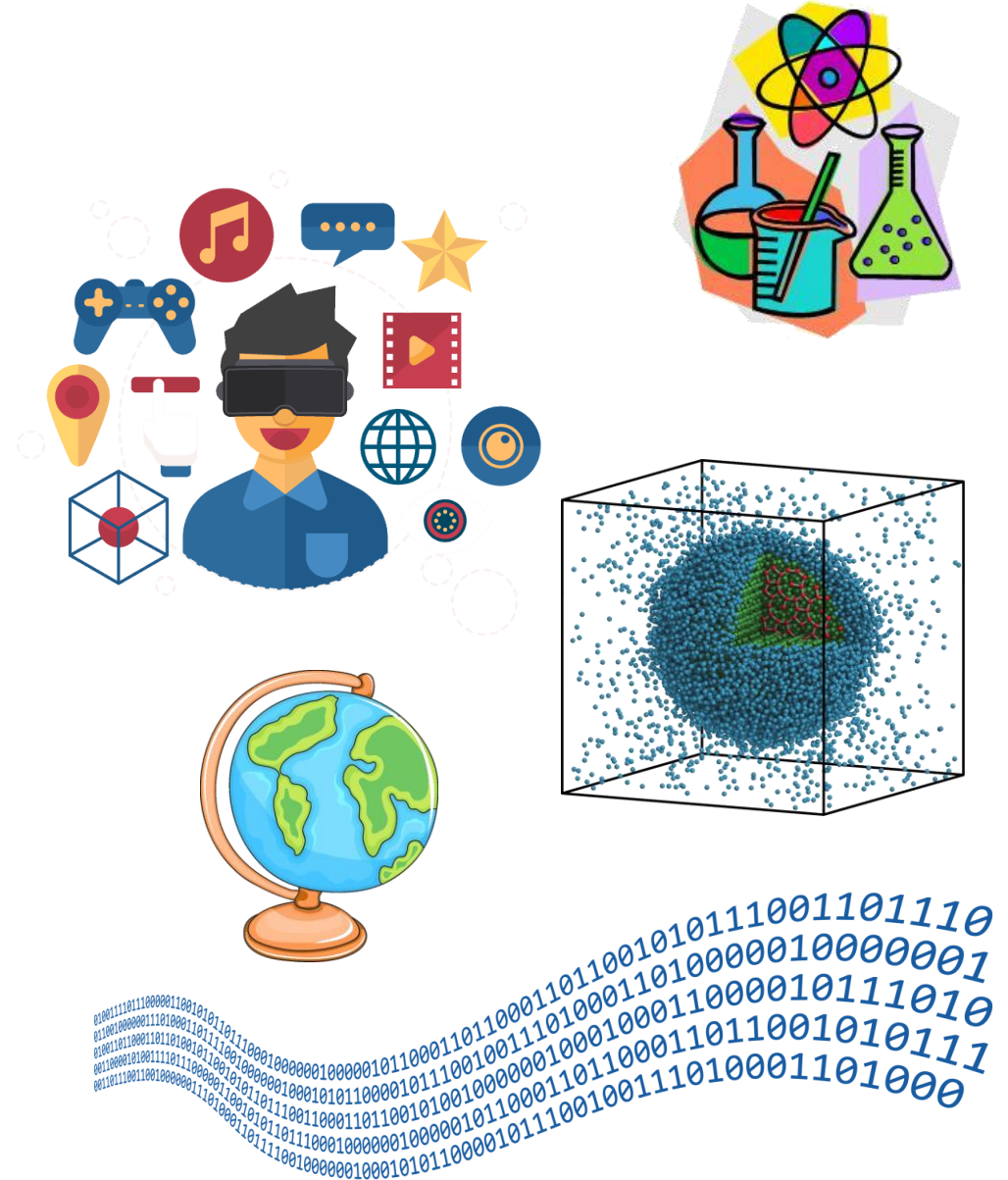
- Indecși unidimensionali
  - B+ Trees, Hash Files
- Indecși multidimensionali
  - pentru baze de date spațiale (GIS)
  - R - Arbori
- Indexare multidimensională
  - Interogări pe intervale
- Indexare avansată
  - Indexarea memoriei interne

# Baze de Date Orientate-Obiect



# Nevoile unui mediu de stocare

- Informații multimedia (imagini, filme)
- Date spațiale (GIS)
- Data biologice
- Proiecte tehnologice (date CAD)
- Lumi virtuale
- Jocuri
- Fluxuri de date
- Tipuri de date definite de utilizator



# Manipularea de categorii noi de date

- Un canal de televiziune necesită stocarea și accesarea rapidă a unor secvențe video, interviuri radio, documente multimedia, informații geografice etc.
- Un producător de filme dorește să stocheze filme întregi și secvențe, date despre actori și cinematografe etc
- Un laborator de cercetări biologice necesită stocarea de date complexe despre molecule, cromozomi și consultarea sau completarea anumitor părți din aceste date.
- Nevoi comerciale complexe.

# Nevoile pentru un SGBD

- Creșterea exponențială a cantității datelor accesate de aplicații în paralel cu reducerea timpului necesar de dezvoltarea a acestor aplicații
  - programare orientată-obiect
  - caracteristici SGBD: optimizare interogări, controlul accesului concurent, recuperarea datelor, indexare etc.
- Subiect de cercetare anii '90: pot fi contopite cele două direcții?

# Dezavantajele bazelor de date relaționale

- Lipsesc attributele de tip colecție
- Lipsește moștenirea
- Lipsesc obiectele complexe, în afară de BLOB (*binary large object*)
- Diferență conceptuală între limbajul de acces la date (declarative: SQL) și limbajul de programare gazdă (procedural: C++, C#, Java etc).

⇒ Ce alte soluții pot fi implementate?

# Baze de date obiectuale

- Baza de date de obiecte – *depozit* de obiecte persistente:
  - Sisteme de baze de date orientate-obiect: alternativă la sistemele relaționale
  - Sisteme de baze de date relațional-obiectuale: extensie a sistemelor relaționale
- Baze de date OO: *ObjectStore, GemStone, Wakanda, Realm*

# Modelul de date obiectual

- *Modelul obiectual* reprezintă fundamentul bazelor de date orientate obiect, așa cum *modelul relațional* reprezintă fundamentul pentru bazele de date relaționale.
- Baza de date conține o colecție de obiecte
- Un obiect are un ID unic (OID) iar colecția obiectelor ce au proprietăți similare se numește clasă.
- Proprietățile unui obiect sunt specificate folosind un ODL (*Object Description Language*) iar obiectele sunt accesate folosind OML (*Object Manipulation Language*).



# Proprietățile obiectelor

- **Attribute:** au tipuri atomice sau structurate (*set, bag, list, array*)
- **Relații:** referință către un obiect sau mulțime de obiecte
- **Metode:** funcții ce pot fi aplicate obiectelor unei clase

# Tipuri abstracte de date

- Funcționalitate cheie: crearea de noi tipuri de date arbitrare de către utilizatori.
- Un tip nou de date este însoțit de metode de accesare corespunzătoare (tip + metode = tip abstract de date).
- De asemenea, un SGBD are tipuri predefinite.

# Încapsularea

- Încapsulare = structuri de date + operații
- Încapsularea permite ascunderea detaliilor interne unui tip abstract de date
- SGBD-ul nu trebuie să cunoască modul de stocare a datelor sau felul în care funcționează metodele unui tip abstract de date. Este necesară doar cunoașterea metodelor disponibile și a detaliilor de apelare a acestora (tipuri de intrare/ieșire)

# Moștenire

|                                                       |
|-------------------------------------------------------|
| O valoare are un tip<br>Un obiect aparține unei clase |
|-------------------------------------------------------|

## ■ Ierarhie de tipuri

- Este permisă definirea de tipuri noi de date pe baza tipurilor existente
- Un *subtip* moștenește toate proprietățile *supertipului*

## ■ Ierarhie de clase

- O subclasă  $C'$  a unei clase  $C$  este o colecție de obiecte în care fiecare obiect al clase  $C'$  este în același timp și obiect al clasei  $C$ .
- Un obiect al clasei  $C'$  moștenește toate proprietățile din  $C$

# Baze de date orientate obiect

- Scopul unui SGBD OO este integrarea “naturală” într-un limbaj de POO ca C++, C#, Java etc.
- ODL = *Object Description Language*, corespunzător DDL din SQL.
- OML = *Object Manipulation Language*, ce înlocuiește SQL DML într-un context orientat-obiect.

# ODL în SGBD-urile Orientate-Obiect

- ODL este utilizat pentru definirea de clase *persistente*, ale căror obiecte pot fi stocate permanent în baza de date.
  - Definirea claselor cu ODL reprezintă o extensie a limbajului orientat-obiect gazdă.

# ODL

- Declarația unei clase include:
  - Numele clasei
  - Declarație opțională de chei
  - Declarația unui *extent* = numele mulțimii tuturor obiectelor ce aparțin clasei.
  - Declarații de elemente. Un *element* poate fi un atribut, o relație sau o metodă.

```
class <name> {  
    <list of element declarations,  
        separated by semicolons>}  
}
```

# Declarații de attribute și metode

- Attributele sunt (de obicei) declarate prin nume și tip, unde tipul nu reprezintă o clasă.

**attribute** <type> <name>;

- Informațiile din declarația unei metode conțin:

- Tipul returnat (dacă este cazul)
- Numele metodei
- Categoria (*in*, *out*, *inout*) și tipul argumentelor (fără nume)
- Excepțiile ce pot fi aruncate de către metodă

**real** grade\_avg(**in** **string**) **raises** (noGrades) ;



# Declarații de relații

- Relațiile conectează un obiect al unei clase cu unul sau mai multe obiecte ale unei alte clase.
- Relațiile sunt memorate ca perechi de pointeri inversați (A îl referă pe B și B îl referă pe A)
- Relațiile sunt întreținute automat de către sistem (dacă A este eliminat, pointerul lui B va fi automat inițializat cu NULL)
- Categoriile de relații: *one-to-one*, *one-to-many*, *many-to-many*

**relationship** <type> <name> **inverse** <relationship>;

# Exemplu

```
class Movie{  
    attribute date start;  
    attribute date end;  
    attribute string movieName;  
    relationship Set<Cinema> shownAt inverse  
                                                Cinema::nowShowing;  
}
```

*tipul relației*



```
class Cinema {  
    attribute string cinemaName;  
    attribute string address;  
    attribute integer ticketPrice;  
    relationship Set <Movie> nowShowing inverse  
                                                Movie::shownAt  
  
    float numshowing() raises(errorCountingMovies) ;  
}
```

*operatorul :: conectează  
un nume unui context*



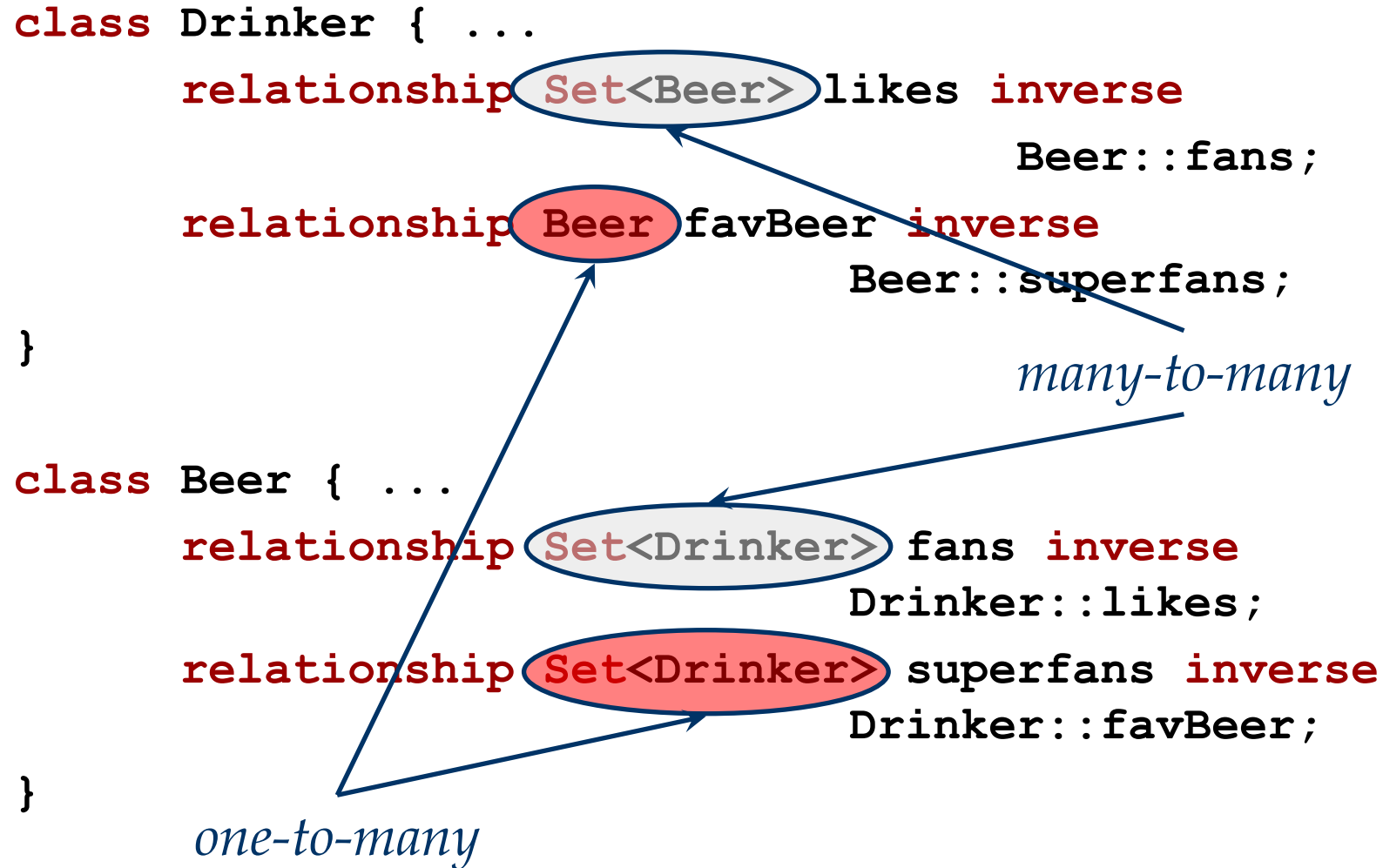
# Tipuri de relații

- Tipul unei relații poate să fie:
  - O clasă, ca *Movie*. În acest caz un obiect cu acest tip de relație poate fi conectat cu un singur obiect *Movie*.
  - **Set**<*Movie*>: obiectul este conectat cu o mulțime de obiecte *Movie*.
  - **Bag**<*Movie*>, **List**<*Movie*>, **Array**<*Movie*>: obiectul este conectat cu o mulțime cu duplicări, listă sau tablou de obiecte *Movie*.

# Multiplicitatea relațiilor

- Toate relațiile ODL sunt binare.
- Relațiile *many-to-many* au *Collection* ca tip al relației și sunt inversate.
- Relațiile *one-to-many* au *Collection*<...> în declarația relației în obiectul “*one*” și doar o clasă în declarația relației obiectului “*many*.”
- Relațiile *one-to-one* au tip de relație clasă în ambele direcții.

# Exemplu



# Exemplu

```
class Person{  
    attribute ...;
```

```
    relationship Person husband inverse wife;  
    relationship Person wife inverse husband;
```

```
    relationship Set<Person> buddies  
        inverse buddies;
```

```
}
```

*husband și wife sunt  
relații one-to-one și una  
reprezintă inversa celeilalte*

*buddies este many-to-many  
și este propria sa inversă*

# Conectarea claselor

- Dacă se dorește conectarea claselor  $X$ ,  $Y$  și  $Z$  printr-o relație  $R$ :
  - Se crează o clasă  $C$ , a căror obiecte reprezintă un triplet de obiecte  $(x, y, z)$  din clasele  $X$ ,  $Y$  și  $Z$ .
  - Se vor crea trei relații *many-to-one* de la  $(x, y, z)$  la fiecare dintre  $x$ ,  $y$  și  $z$ .

# Exemplu: Conectarea claselor

- Fie clasele *BookStore* și *Book*. Dorim să memorăm prețul cu care fiecare librărie (obiect al *BookStore*) vinde o carte.
  - Acest lucru nu se poate modela cu o relație *many-to-many* între *BookStore* și *Book* deoarece nu se pot defini attribute conectate de o relație
- Soluția 1: se crează clasa *Price* și o clasă de conectare *BBP* ce reprezintă relația dintre librărie, carte și preț.
- Soluția 2: deoarece obiectele *Price* conțin doar un simplu număr este poate mai util să:
  - Adăugăm la clasa *BBP* un atribut *price*.
  - Folosim relații *many-to-one* între un obiect *BBP* și obiecte ale *BookStore* și *Book*.



# Exemplu: Conectarea claselor

- Definirea clasei BBP:

```
class BBP {  
    attribute real price;  
    relationship BookStore theBS inverse  
        BookStore::toBBP;  
    relationship Book theBook inverse  
        Book::toBBP; }
```

- *BookStore* și *Book* vor fi ambele modificate prin includerea relației numită *toBBP*, de tipul **Set**<*BBP*>.

# Tipurile ODL

- Tipuri de bază: *int, real/float, string, tipuri enumerare și clase.*
- Tipuri compuse:
  - *Struct* pentru structuri.
  - Tipuri colecții: *Set, Bag, List, Array și Dictionary*
- Tipurile de relații pot fi doar clase sau un tip colecție aplicat unei clase.

# Subclase ODL

- Corespund subclaselor cunoscute din programarea orientată-obiect.

```
class Student:Person
{
    attribute string code;
    . . .
}
```

# Chei și *extensii* în ODL

- Pentru o clasă se pot declara oricâte chei

(**key** <list of keys>)

- Fiecare clasă are un *extent*, ce reprezintă mulțimea tuturor obiectelor clasei respective:

- O extensie se declară după numele clasei împreună cu cheile astfel:

(**extent** <extent name> ... )

- Convenție: se utilizează substantive comune la singular pentru numele claselor, și la plural *extensiile* corespunzătoare.

# Exemplu

```
class Book
```

```
    (key name) { ... }
```

```
class Course
```

```
    (key (dept,number) ,  
        (room, hours)) { ... }
```

```
class Student
```

```
    (extent Students key code) { ... }
```

# OML în SGBD OO

- Implementările OML nu sunt foarte eficiente (optimizările limbajului de interogare sunt modeste)
- Cel mai popular limbaj de interogare este OQL (*Object Query Language*) ce a fost proiectat pentru o sintaxă similară cu SQL.
- OQL poate fi privit ca o extensie a SQL
  - Include clauzele **select**, **from**, **where** și **group by**
  - S-au adăugat elemente ce accesează proprietățile obiectelor și operatori pentru tipuri de date complexe.

# Exemplu

```
class Movie (extent Movies key movieName) {
    attribute date start;
    attribute date end;
    attribute string movieName;
    relationship Set<Cinema> shownAt inverse
                                                Cinema::nowShowing;
}

class Cinema (extent Cinemas key cinemaName) {
    attribute string cinemaName;
    attribute string address;
    attribute integer ticketPrice;
    relationship Set<Movie> nowShowing inverse
                                                Movie::shownAt;

    float numshowing() raises(errorCountingMovies) ;
}
```

# Accesarea proprietăților obiectelor (expresii de cale)

- Fie  $x$  un obiect al clasei  $C$ .
  - Dacă  $a$  este un atribut al  $C$ , atunci  $x.a$  este valoarea acelui atribut.
  - Dacă  $r$  este o relație a lui  $C$ , atunci  $x.r$  este obiectul sau colecția de obiecte cu care  $x$  este conectat prin  $r$ .
  - Dacă  $m$  este o metodă a lui  $C$ , atunci  $x.m(\dots)$  este rezultatul aplicării lui  $m$  la  $x$ .



# OQL: Select-From-Where

- O frază OQL obișnuită are sintaxa:

**SELECT** <list of values>

**FROM** <list of collections and  
names for typical members>

**WHERE** <condition>

- Fiecare termen al clauzei FROM este:

<colecție> <nume membru>

- O colecție poate fi:

- *Extensia* unei clase, sau

- O expresie ce se evaluează la o colecție

- Pentru a schimba denumirea unui câmp, acesta va fi precedat de un nume si “:”

# Exemplu OQL

*Să se returneze cinematografele care proiectează mai mult decât un film și filmele proiectate în aceste cinematografe.*

```
SELECT mname: M.movieName,  
         cname: C.cinemaName  
FROM Movies M, M.shownAt C  
WHERE C.numshowing() >1
```

# Tipul rezultatului unei interogări

- Implicit, tipul rezultatului unei structuri **select-from-where** este un *Bag* de *Struct*.
  - *Struct* are câte un câmp pentru fiecare termen al clauzei SELECT. Numele și tipul sunt preluate de la ultimul element al expresiei de cale.
- Dacă rezultatul interogării are un singur termen, acesta va fi de fapt o structură cu un singur câmp.

# Tipul rezultatului unei interogări

- Se poate adăuga DISTINCT după SELECT iar rezultatul va avea tipul *Set*, duplicatele fiind eliminate.
- La utilizarea clauzei ORDER BY rezultatul va fi o listă de structuri, ordonate după câmpurile enumerate în ORDER BY
  - Ordonarea se face crescător (ASC - implicit) sau descrescător (DESC).
  - Elementele listei pot fi accesate si utilizând indecși ( [1], [2],... ), similar cursorilor SQL.

# Subinterogări

■ O expresie *select-from-where* poate fi utilizată ca subinterogare în mai multe moduri:

- Într-o clauză FROM, ca o colecție.
- Într-o expresie logică folosită în clauza WHERE :

**FOR ALL x IN <collection> : <condition>**

**EXISTS x IN <collection> : <condition>**

# Exemplu

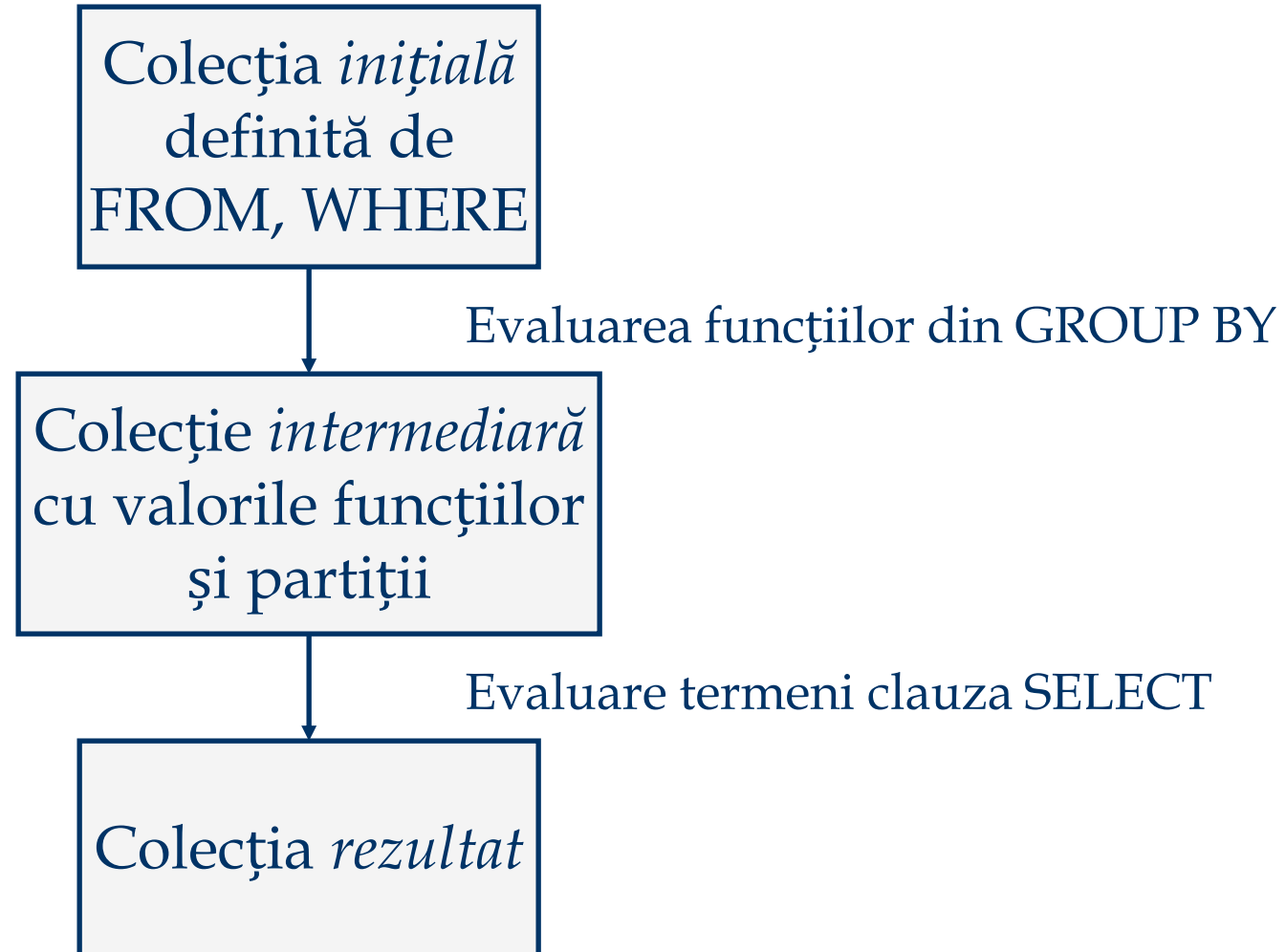
- *Să se returneze numele tuturor filmelor care sunt proiectate în cel puțin un cinematografula un preț de bilet > 5*

```
SELECT m.name
FROM Movies m
WHERE
    EXISTS c IN m.shownAt:
    c.ticketPrice > 5
```

# Gruparea datelor în OQL

- OQL extinde ideea grupării:
  - Toate colecțiile pot fi partiționate în grupuri.
  - Grupările se pot realiza având la bază orice funcție/funcții ale obiectelor ce aparțin colecției inițiale.
- AVG, SUM, MIN, MAX și COUNT se pot aplica tuturor colecțiilor (atunci când este cazul).

# Gruparea datelor în OQL



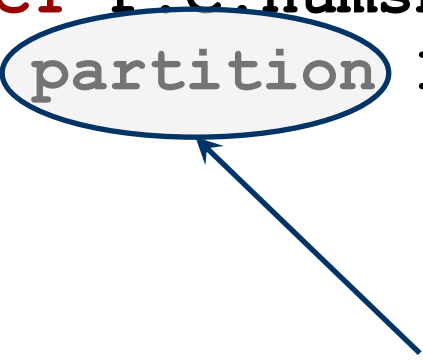


# Exemplu GROUP BY

*Să se returneze toate valorile distincte de preț utilizate de cinematografe și media numărului de filme proiectate cu bilete vândute la acel preț.*

```
SELECT C.ticketPrice,  
        avgNum:AVG(SELECT P.C.numshowing()  
                  FROM partition P)  
FROM Cinemas C  
GROUP BY C.ticketPrice
```

*Partiționarea în OQL*



## Exemplu GROUP BY: Colecția inițială

- Pe baza **FROM** și **WHERE** (care lipsește):  
**FROM Cinemas C**
- Colecția inițială este un *Bag* de structuri cu un singur câmp pentru fiecare element din clauza **FROM**.
- În particular, colecția reprezintă un *Bag* de structuri de forma `Struct(c: obj )`, unde *obj* este un obiect *Cinema*.

# Exemplu GROUP BY: Colecția intermediară

- În general, este un *bag* de structuri cu o componentă pentru fiecare funcție din clauza GROUP BY, și o componentă suplimentară numită invariabil *partition*.
- Valoarea componentei *partition* este dată de mulțimea tuturor obiectelor din colecția inițială care aparțin grupului reprezentat de structură.

```
SELECT C.ticketPrice,  
       avgNum:AVG (  
         SELECT P.C.numshowing()  
         FROM partition P)  
FROM Cinemas C  
GROUP BY C.ticketPrice
```

O funcție de grupare:

- nume - *ticketPrice*,
- tip - *integer*.

Colecția intermediară este un *set* de structuri cu câmpurile

- *ticketPrice* : *integer*, și
- *partition*: *Set<Struct{c: Cinema}>*

## Exemplu GROUP BY: Colecția intermediară

- Un element al colecției intermediare din exemplu este:

`Struct(ticketPrice = 5,  
      partition = {c1, c2, ..., cn })`

- Fiecare element al *partition* e un obiect *c*<sub>*i*</sub> al clasei *Cinema*, pentru care *c*<sub>*i*</sub>.*ticketPrice* = 5.

## Exemplu GROUP BY: Colecția finală

- Colecția rezultat e dată de clauza **SELECT** care este evaluată pe colecția intermediară.

# Exemplu GROUP BY: Colecția finală

```
SELECT C.ticketPrice, avgNum:AVG (  
SELECT P.C.numshowing() FROM partition P)
```

Extrage câmpul *ticketPrice*  
din structura unui  
grup.

Pentru fiecare element *P* din  
*partition*, se accesează atributul *C*  
(obiect al *Cinema* ), de unde  
accesează numărul de proiecții.

Media numerelor returnate  
de funcțiile *numshowing()*  
stocată în câmpul *avgNum*  
al structurilor din colecția  
finală.

Exemplu de element:  
Struct(ticketPrice = 5, avgNum = 9.5)

# Evoluția SGBD-urilor

- SGBD-urile orientate-obiect au eșuat deoarece nu au putut oferi eficiența obținută de SGBD-urile relaționale.
- Extensiile relațional-obiectuale aplicate SGBD-urilor relaționale captează o bună parte din avantajele OO, dar abstractizarea fundamentală rămâne relația.

# Clasificarea SGBD-urilor

